

nobreakspace



从零到 MONERO：第二版

一本关于隐私数字货币的技术指南；面向初学者、爱好者和专家

原版 PUBLISHED APRIL 4, 2020 (v2.0.0)

原著：KOE, KURT M. ALONSO, SARANG NOETHER

中文翻译：KOSMO & KLEO

许可证：Zero to Monero: Second Edition 发布于公共领域。

摘要

密码学。看起来似乎只有数学家和计算机科学家才能触及这门晦涩、深奥、强大而优雅的学科。事实上，许多密码学的基础概念足够简单，任何人都可以学习。

newline 众所周知，密码学被用于保护通信安全，无论是加密信件还是私密的数字交互。密码学的另一个应用是所谓的加密货币。这些数字货币使用密码学来分配和转移资金所有权。为了确保任何一笔钱都无法被复制或随意创造，加密货币通常依赖于“区块链”——一种公共的、分布式的账本，记录着可以被第三方验证的货币交易 [?]

newline 乍看之下，交易似乎需要以明文形式发送和存储才能被公开验证。事实上，使用密码学工具可以隐藏交易的参与者以及涉及的金额，同时仍然允许观察者验证和确认交易 [?]。加密货币 Monero 就是这一能力的典范。

newline 我们在此努力教授任何懂得基本代数和简单计算机科学概念（如数字的“二进制表示”）的人，不仅要深入全面地理解 Monero 的工作原理，还要领略密码学的实用与优美。

newline 对于有经验的读者：Monero 是一种标准的一维有向无环图（DAG）加密货币区块链 [?]，其交易基于椭圆曲线密码学，使用 Ed25519 曲线 [?]，交易输入采用 Schnorr 风格的多层可链接自发匿名群签名（MLSAG）[?] 签名，输出金额（通过 ECDH [?] 传达给接收方）使用 Pedersen 承诺 [?] 隐藏，并通过 Bulletproofs [?] 证明其在合法范围内。本报告的前半部分大部分篇幅用于解释这些概念。

目录

第

1

章 1

引言 (Introduction)

在数字领域，复制信息并对其进行任意修改通常都是轻而易举的事。一种货币要以数字化形式存在并被广泛接受，其使用者必须相信其供应量受到严格限制。货币接收者必须确信自己不会收到伪造的货币，或是已经被发送给其他人的货币。为了在不需要任何第三方（如中央机构）协作的情况下实现这一点，货币的供应量和完整的交易历史必须可以公开验证。

我们可以利用密码学工具，使存储在易于访问的数据库——即 blockchain——中的数据实际上不可篡改和不可伪造，其合法性不容任何一方质疑。

newline 加密货币将 transaction 存储在 blockchain 中，后者充当所有货币操作的公开账本¹。大多数加密货币以明文形式存储 transaction，以便社区用户验证 transaction。

newline 显然，公开的 blockchain 违背了对隐私或可替代性²的基本认知，因为它实际上公开了用户的完整交易历史。

¹ 此处“账本”仅指所有货币创建和交换事件的记录。具体而言，即每次事件中转移了多少金额以及转移给了谁。

² “可替代的指在使用或合同履行中可以相互替代的。……例如：……货币等。”[?] 在 Bitcoin 等公开 blockchain 中，Alice 持有的币可以与 Bob 持有的币通过这些币的“交易历史”加以区分。如果 Alice 的交易历史中包含与所谓恶意为者相关的交易，那么她的币可能被“污染”[?]，从而比 Bob 的币价值更低（即使他们拥有相同数量的币）。知名人士声称，新铸造的 Bitcoin 相比已使用的币有溢价，因为它们没有历史记录 [?]

为了解决隐私缺失的问题，Bitcoin 等加密货币的用户可以通过使用临时中间地址来混淆 transaction [?]。然而，借助适当的工具，仍然可以分析资金流向，并在很大程度上将真正的发送者与接收者关联起来 [?, ?, ?, ?]。

相比之下，加密货币 Monero (Moe-neh-row) 试图通过仅在 blockchain 中存储一次性接收地址来解决隐私问题，并通过环签名来验证每笔 transaction 中资金的分散。采用这些方法，目前没有已知的通用有效方法来关联接收者或追溯资金来源。³

此外，Monero blockchain 中的交易金额隐藏在密码学构造之后，使得资金流向变得不透明。

其结果是一种具有高度隐私性和可替代性的加密货币。

1.1 目标 (Objectives)

Monero 是一种成熟的加密货币，拥有超过五年的开发历史 [?, ?]，并且采用率在稳步增长 [?]。⁴遗憾的是，描述其所用机制的全面文档很少。^{5,6}更糟糕的是，其理论框架的关键部分发表在未经同行评审的论文中，这些论文不完整或包含错误。对于 Monero 理论框架的重要部分，只有源代码才是可靠的信息来源。⁷

此外，对于没有数学背景的人来说，学习椭圆曲线密码学（Monero 大量使用）的基础知识可能是杂乱无章且令人沮丧的尝试。⁸

我们旨在通过介绍理解椭圆曲线密码学所需的基本概念、回顾算法和密码学方案以及收集关于 Monero 内部运作的深入信息来改善这一状况。

为了给读者提供最佳体验，我们精心构建了一个循序渐进的 Monero 加密货币描述。

在本报告的第二版中，我们将注意力集中在 Monero 协议的第 12 版⁹上，对应于 Monero 软件套件 0.15.x.x 版本。此处描述的所有与 transaction 和 blockchain 相关的机制均属于这些版本。^{10,11}已弃用的 transaction 方案没有被探讨，即使它们可能为了向后兼容而被部分支持。弃用的 blockchain 功能亦是如此。第一版 [?] 对应于协议第 7 版和软件套件 0.12.x.x 版本。

³ 根据用户的行为，某些情况下 transaction 可能在一定程度上被分析。相关示例请参见这篇文章：[?]。

⁴ 就市值而言，Monero 相对于其他加密货币一直保持稳定。截至 2018 年 6 月 14 日，其市值排名第 14 位，2020 年 1 月 5 日排名第 12 位；参见 <https://coinmarketcap.com/>。

⁵ 一个文档项目位于 <https://monerodocs.org/>，其中有一些有用的条目，尤其是与命令行界面相关的内容。CLI 是一种可通过控制台/终端访问的 Monero 钱包。它在所有 Monero 钱包中功能最为丰富，但代价是没有用户友好的图形界面。

⁶ 另一个更通用的文档项目名为 Mastering Monero，可在此处找到：[?]。

⁷ Seguias 先生创建了优秀的 Monero Building Blocks 系列 [?]，其中包含了对用于论证 Monero 签名方案的密码学安全证明的深入讨论。与《Zero to Monero: 第一版》[?] 一样，Seguias 的系列专注于协议的 v7 版本。

⁸ 之前一篇解释 Monero 工作原理的尝试 [?] 没有阐明椭圆曲线密码学，内容不完整，而且已过时五年以上。

⁹ “协议”是指在新区块被添加到 blockchain 之前对其进行检验的规则集。这套规则包括“transaction 协议”（目前为第 2 版，即 RingCT），这是关于如何构造 transaction 的通用规则。具体的 transaction 规则可以且确实会变化，而不会改变 transaction 协议的版本号。只有对 transaction 结构的大规模变更才值得提升其版本号。

¹⁰ Monero 代码库的完整性和可靠性建立在假设足够多的人审查了代码以捕获大多数或所有重大错误的基础上。我们希望读者不要将我们的解释视为理所当然，而应自行验证代码是否按预期运行。如果没有，我们希望您会对重大问题负责任的披露 (<https://hackerone.com/monero>)，或对小问题提交 Github pull request (<https://github.com/monero-project/monero>)。

¹¹ 几项值得考虑用于下一代 Monero transaction 的协议正在进行研究和调查，包括 Triptych [?]、RingCT3.0 [?]、Omniring [?] 和 Lelantus [?]。

1.2 读者群体 (Readership)

我们预计许多读者在接触本报告时，对离散数学、代数结构、密码学¹²和 blockchain 几乎没有什么了解。我们力求足够详尽，使来自各方面的外行都能无需外部调研即可学习 Monero。

我们有意省略了某些数学技术细节，或将它们安排在脚注中，以免妨碍理解。我们还在认为某些具体实现细节非必要时应将其省略。我们的目标是在数学密码学和计算机编程之间找到平衡点，力求完整性和概念清晰性。¹³

1.3 Monero 加密货币的起源 (Origins of the Monero cryptocurrency)

加密货币 Monero，最初称为 BitMonero，于 2014 年 4 月作为概念验证货币 CryptoNote 的衍生品被创建 [?]。Monero 在世界语中意为“金钱”，其复数形式为 Moneroj (Moe-neh-rowje，类似于 Moneros 但使用 orange 中的 -ge 发音)。

CryptoNote 是由多位个人设计的加密货币。一份描述它的重要白皮书于 2013 年 10 月以 Nicolas van Saberhagen 的笔名发表 [?]。它通过使用一次性地址提供接收者匿名性，并通过环签名实现发送者模糊性。

自创建以来，Monero 通过实现金额隐藏进一步增强了其隐私方面，正如 Greg Maxwell 等人在 [?] 中所述，并根据 Shen Noether 的建议在 [?] 中将其集成到基于环签名的方案中，随后通过 Bulletproofs [?] 提高了效率。

1.4 大纲 (Outline)

如前所述，我们的目标是提供一个自包含且循序渐进的 Monero 加密货币描述。Zero to Monero 的结构正是为了实现这一目标，引导读者了解该货币内部运作的各个部分。

1.4.1 第一部分：“基础” (Part 1: ‘Essentials’)

在追求全面性的过程中，我们选择介绍理解 Monero 复杂性所需的所有密码学基本要素及其数学基础。在第 ?? 章中，我们阐述了椭圆曲线密码学的基本方面。

第 ?? 章扩展了前一章的 Schnorr 签名方案，并概述了将应用于实现 confidential transaction 的环签名算法。

第 ?? 章探讨了 Monero 如何使用地址来控制资金所有权，以及不同类型的地址。

¹² 一本关于应用密码学的广泛教科书可在此处找到：[?]

¹³ 一些脚注，尤其是与协议相关的章节中的脚注，会剧透后续章节或部分内容。这些脚注旨在第二次阅读时更有意义，因为它们通常涉及只对已掌握 Monero 工作原理的人有用的特定实现细节。

在第 ?? 章中，我们介绍了用于隐藏金额和密码学机制。

在所有组件就位后，我们在第 ?? 章中解释了 Monero 中使用的 transaction 方案。

Monero blockchain 在第 ?? 章中被展开说明。

1.4.2 第二部分：“扩展” (Part 2: ‘Extensions’)

加密货币不仅仅是其协议，在“扩展”部分我们讨论了许多不同的想法，其中许多尚未实现。¹⁴

关于 transaction 的各种信息可以向观察者证明，这些方法是第 ?? 章的内容。

虽然对 Monero 的运行不是必需的，但允许多人协作发送和接收资金的多重签名具有很大的实用价值。第 ?? 章描述了 Monero 当前的多重签名方法，并概述了该领域可能的未来发展。

在在线市场中将多重签名应用于商家和购物者之间的交互极为重要。第 ?? 章构成了我们使用 Monero 多重签名设计的托管市场的原创方案。

首次在此提出的 TxTangle，在第 ?? 章中概述，是一种将多个个体的 transaction 合并为一个的去中心化协议。

1.4.3 附加内容 (Additional content)

附录 ?? 解释了 blockchain 中一个示例 transaction 的结构。附录 ?? 解释了 Monero blockchain 中区块（包括区块头和矿工 transaction）的结构。最后，附录 ?? 通过解释 Monero 创世区块的结构为我们的报告画上句号。这些附录在前述章节中描述的理论元素与其实际实现之间建立了联系。

我们使用旁注来指示在源代码中可以找到 Monero 实现细节的位置。¹⁵通常包含一个文件路径，如 `src/ringct/rctOps.cpp`，和一个函数，如 `ecdhEncode()`。注意：‘-’ 表示拆分的文本，如 `cryptonote` → `cryptonote`，我们在大多数情况下省略命名空间限定符（如 `Blockchain::`）。

1.5 免责声明 (Disclaimer)

所有签名方案、椭圆曲线应用和 Monero 实现细节都应仅视为描述性的。考虑进行严肃实际应用（而非爱好探索）的读者应查阅原始来源和技术规范（我们已在可能的地方引用）。签名方案需要经过严格审查的安全证明，Monero 实现细节可以在 Monero 的源代码中找到。特别是，正如一句老话所说，“不要自己造轮子”。实现密码学原语的代码应经过专家的充分审查并拥有长期可靠的运行记录。此外，本文档中的原创贡献可能未经充分审查且可能未经测试，因此读者在阅读时应自行判断。

¹⁴ 请注意，Monero 未来的协议版本，特别是实现新 transaction 协议的版本，可能使这些想法中任何或全部变得不可能或不切实际。

¹⁵ 我们的旁注在 Monero 软件套件 0.15.x.x 版本中是准确的，但随着代码库的不断变化，可能会逐渐变得不准确。然而，代码存储在 git 仓库中 (<https://github.com/monero-project/monero>)，因此可以查看完整的变更历史。

1.6 Zero to Monero 的历史 (History of Zero to Monero)

Zero to Monero 是 Kurt Alonso 的硕士论文《Monero——区块链中的隐私》[?] 的扩展版本，该论文于 2018 年 5 月发表。第一版于 2018 年 6 月出版 [?]

在第二版中，我们改进了环签名的介绍方式（第 ?? 章），重组了 transaction 的解释方式（新增了关于 Monero 地址的第 ?? 章），更新了传输 output 金额的方法（第 ?? 节），用 Bulletproofs 替代了 Borromean 环签名（第 ?? 节），弃用了 RCTTypeFull（第 ?? 章），更新并详细阐述了 Monero 的动态区块权重和费用系统（第 ?? 章），调查了与 transaction 相关的证明（第 ?? 章），描述了 Monero 多重签名（第 ?? 章），设计了托管市场解决方案（第 ?? 章），提出了一种名为 TxTangle 的新型去中心化联合 transaction 协议（第 ?? 章），更新或添加了各种细节以与最新协议（v12）和 Monero 软件套件（v0.15.x.x）保持一致，并对文档进行了阅读质量的全面润色。¹⁶

1.7 致谢 (Acknowledgements)

由作者 ‘koe’ 撰写。

本报告的诞生离不开 Kurt 的原始硕士论文 [?]，因此我对他深表感谢。Monero 研究实验室 (MRL) 的研究人员 Brandon “Surae Noether” Goodell 和化名 ‘Sarang Noether’ 的研究者（他与我合作了第 ?? 节和第 ?? 章）在两版 Zero to Monero 的开发过程中一直是可靠且知识渊博的资源。化名 ‘moneromooo’ 的开发者是 Monero 项目最高产的核心开发者，可能拥有这个星球上对代码库最广泛的了解，并多次为我指明了正确的方向。当然，许多其他优秀的 Monero 贡献者也花时间回答了我无尽的问题。最后，感谢所有通过校对反馈和鼓励评论联系我们的人！

¹⁶ 两版 Zero to Monero 的 L^AT_EX 源代码可在此处找到（第一版在 ‘ztml’ 分支中）：<https://github.com/UkoeHB/Monero-RCT-report>。

部分 I

基础

第

2

章 2

基本概念 (Basic Concepts)

2.1 关于符号的说明 (A few words about notation)

本报告的一个核心目标是收集、审查、纠正和统一关于 Monero 加密货币内部运作的所有现有信息，同时提供所有必要的细节，以循序渐进的方式呈现材料。

实现这一目标的一个重要工具是确定一套符号约定。其中，我们使用了：

- 小写字母表示简单值、整数、字符串、位表示等
- 大写字母表示曲线点和复杂结构

对于具有特殊含义的元素，我们尽量在全文中使用相同的符号。例如，曲线生成元始终用 G 表示，其阶为 l ，私钥/公钥尽可能分别用 k/K 表示，等等。

除此之外，我们在呈现算法和方案时力求概念化。具有计算机科学背景的读者可能会觉得我们忽略了诸如元素的位表示等问题，或在某些情况下忽略了如何执行具体操作。此外，数学专业的学生可能会发现我们忽略了抽象代数的解释。

然而，我们并不认为这是一种损失。简单的对象如整数或字符串总是可以用位串表示。所谓“字节序”很少相关，对我们的算法来说主要是约定问题。¹

椭圆曲线点通常用数对 (x, y) 表示，因此可以用两个整数来表示。然而，在密码学领域中，通常会应用点压缩技术，允许仅使用一个坐标的空间来表示一个点。对于我们的概念化方法来说，是否使用点压缩通常是次要的，但在大多数情况下是隐式假定的。

我们还自由地使用了密码学 hash 函数，而未指定任何具体算法。在 Monero 的情况下，通常使用的是 *Keccak*² 变体，但如果未明确提及，则对理论并不重要。

密码学 hash 函数（以下简称为“hash 函数”或“hash”）接受任意长度的消息 m ，并返回固定长度的 hash h （或“消息摘要”），对于给定输入，每个可能输出的概率相等。密码学 hash 函数难以逆转，具有一个被称为大雪崩效应的有趣特性，该特性可以使非常相似的消息产生非常不同的 hash，并且很难找到两个具有相同消息摘要的消息。

hash 函数将应用于整数、字符串、曲线点或这些对象的组合。这些情况应被解释为位表示的 hash，或此类表示的拼接。根据上下文，hash 的结果可以是数字、位串，甚至曲线点。这方面的更多细节将在需要时给出。

2.2 模运算 (Modular arithmetic)

大多数现代密码学始于模运算，而模运算又始于取模运算（用 ‘mod’ 表示）。我们只关心正取模，它始终返回一个正整数。

正取模类似于两个数相除后的“余数”，例如 c 是 a/b 的“余数”。让我们想象一条数轴。要计算 $c = a \pmod{b}$ ，我们站在点 a 处，然后以每步 $\text{step} = b$ 的步伐向零走去，直到到达一个 ≥ 0 且 $< b$ 的整数。那就是 c 。例如， $4 \pmod{3} = 1$ ， $-5 \pmod{4} = 3$ ，依此类推。

形式化地，正取模在此定义为 $c = a \pmod{b}$ 满足 $a = bx + c$ ，其中 $0 \leq c < b$ ， x 是一个会被丢弃的带符号整数（ b 是正非零整数）。

注意，如果 $a \leq n$ ，则 $-a \pmod{n}$ 等于 $n - a$ 。

¹ 在计算机内存中，每个字节存储在自己的地址中（地址类似于一个编号的插槽，字节可以存储在其中）。一个给定的“字”或变量通过其字节的最低地址来引用。如果变量 x 有 4 个字节，存储在地址 10-13 中，则使用地址 10 来查找 x 。 x 的字节在其地址集中的组织方式取决于字节序，尽管每个单独的字节在其地址内始终以相同方式存储。基本上， x 的哪一端存储在引用地址中？它可以是大端或小端。给定 $x = 0x12345678$ （十六进制，2 个十六进制数字占用 1 个字节即 8 个二进制位），以及地址数组

10, 11, 12, 13
， x 的大端编码为
12, 34, 56, 78
， 小端编码为
78, 56, 34, 12

。

citeendianness

² Keccak hash 算法构成了 NIST 标准 *SHA-3* [?] 的基础。

2.2.1 模加法和模乘法 (Modular addition and multiplication)

在计算机科学中, 进行模运算时避免大数是很重要的。例如, 如果我们有 $29 + 87 \pmod{99}$, 且不允许使用三位或更多位的变量 (如 $116 = 29 + 87$), 那么我们无法直接计算 $116 \pmod{99} = 17$ 。

要执行 $c = a + b \pmod{n}$, 其中 a 和 b 各自小于模数 n , 我们可以这样做:

- 计算 $x = n - a$ 。如果 $x > b$ 则 $c = a + b$, 否则 $c = b - x$ 。

我们可以利用模加法来实现模乘法 ($a * b \pmod{n} = c$), 使用的算法称为” 双倍加法”。让我们通过示例来演示。假设我们要做 $7 * 8 \pmod{9} = 2$ 。这等同于

$$7 * 8 = 8 + 8 + 8 + 8 + 8 + 8 + 8 \pmod{9}$$

现在将其分成两两一组。

$$(8 + 8) + (8 + 8) + (8 + 8) + 8$$

再分一次, 两两一组。

$$[(8 + 8) + (8 + 8)] + (8 + 8) + 8$$

+ 点运算的总次数从 6 次减少到 4 次, 因为我们只需要找到一次 $(8 + 8)$ 。³

双倍加法是通过将第一个数 (”被乘数” a) 转换为二进制来实现的 (在我们的例子中, $7 \rightarrow [0111]$), 然后遍历二进制数组进行双倍和加法。

让我们建立一个数组 $A = [0111]$ 并按 3,2,1,0 索引。⁴ $A[0] = 1$ 是 A 的第一个元素, 也是最低有效位。我们设置一个结果变量初始为 $r = 0$, 设置一个求和变量初始为 $s = 8$ (更一般地, 我们从 $s = b$ 开始)。我们遵循以下算法:

1. 遍历: $i = (0, \dots, A_{size} - 1)$
 - (a) 如果 $A[i] == 1$, 则 $r = r + s \pmod{n}$ 。
 - (b) 计算 $s = s + s \pmod{n}$ 。
2. 使用最终的 r : $c = r$ 。

在我们的例子 $7 * 8 \pmod{9}$ 中, 出现如下序列:

1. $i = 0$
 - (a) $A[0] = 1$, 所以 $r = 0 + 8 \pmod{9} = 8$
 - (b) $s = 8 + 8 \pmod{9} = 7$
2. $i = 1$

³ 双倍加法的效果在大数时变得明显。例如, 对于 $2^{15} * 2^{30}$, 直接加法需要大约 2^{15} 次 + 运算, 而双倍加法只需要 15 次!

⁴ 这被称为”LSB 0” 编号, 因为最低有效位的索引为 0。本章其余部分将使用”LSB 0”。这里的关键是清晰性, 而非精确的约定。

(a) $A[1] = 1$, 所以 $r = 8 + 7 \pmod{9} = 6$

(b) $s = 7 + 7 \pmod{9} = 5$

3. $i = 2$

(a) $A[2] = 1$, 所以 $r = 6 + 5 \pmod{9} = 2$

(b) $s = 5 + 5 \pmod{9} = 1$

4. $i = 3$

(a) $A[3] = 0$, 所以 r 保持不变

(b) $s = 1 + 1 \pmod{9} = 2$

5. $r = 2$ 即为结果

2.2.2 模幂运算 (Modular exponentiation)

显然 $8^7 \pmod{9} = 8 * 8 * 8 * 8 * 8 * 8 * 8 \pmod{9}$ 。正如双倍加法一样, 我们可以使用平方乘法。对于 $a^e \pmod{n}$:

1. 定义 $e_{\text{scalar}} \rightarrow e_{\text{binary}}$; $A = [e_{\text{binary}}]$; $r = 1$; $m = a$

2. 遍历: $i = (0, \dots, A_{\text{size}} - 1)$

(a) 如果 $A[i] == 1$, 则 $r = r * m \pmod{n}$ 。

(b) 计算 $m = m * m \pmod{n}$ 。

3. 使用最终的 r 作为结果。

2.2.3 模乘逆 (Modular multiplicative inverse)

有时我们需要 $1/a \pmod{n}$, 或者换句话说 $a^{-1} \pmod{n}$ 。某物乘以其逆元的定义是 1 (单位元)。想象 $0.25 = 1/4$, 然后 $0.25 * 4 = 1$ 。

在模运算中, 对于 $c = a^{-1} \pmod{n}$, $ac \equiv 1 \pmod{n}$ 且 $0 \leq c < n$, 其中 a 和 n 互素。⁵互素意味着它们除了 1 以外没有其他公因数 (分数 a/n 不能被约分/简化)。

当 n 是素数时, 我们可以使用平方乘法来计算模乘逆元, 这基于费马小定理:⁶

$$a^{n-1} \equiv 1 \pmod{n}$$

$$a * a^{n-2} \equiv 1 \pmod{n}$$

$$c \equiv a^{n-2} \equiv a^{-1} \pmod{n}$$

更一般地 (也更快地), 所谓的”扩展欧几里得算法” [?] 也可以找到模逆元。

⁵ 在方程 $a \equiv b \pmod{n}$ 中, a 与 b 模 n 同余, 这仅意味着 $a \pmod{n} = b \pmod{n}$ 。

⁶ 模乘逆元有一条规则:

如果 $ac \equiv b \pmod{n}$ 且 a 和 n 互素, 则该线性同余方程的解为 $c = a^{-1}b \pmod{n}$ 。 [?] 这意味着我们可以做 $c = a^{-1}b \pmod{n} \rightarrow ca \equiv b \pmod{n} \rightarrow a \equiv c^{-1}b \pmod{n}$ 。

2.2.4 模方程 (Modular equations)

假设我们有一个方程 $c = 3 * 4 * 5 \pmod{9}$ 。计算这个是直接的。给定两个表达式 A 和 B 之间的某种运算 $\circ$ (例如 $\circ = *$):

$$(A \circ B) \pmod{n} = [A \pmod{n}] \circ [B \pmod{n}] \pmod{n}$$

在我们的例子中, 设 $A = 3 * 4$, $B = 5$, $n = 9$:

$$\begin{aligned} (3 * 4 * 5) \pmod{9} &= [3 * 4 \pmod{9}] * [5 \pmod{9}] \pmod{9} \\ &= [3] * [5] \pmod{9} \\ c &= 6 \end{aligned}$$

现在我们有了解进行模减法的方法。

$$\begin{aligned} A - B \pmod{n} &\rightarrow A + (-B) \pmod{n} \\ &\rightarrow [A \pmod{n}] + [-B \pmod{n}] \pmod{n} \end{aligned}$$

同样的原理适用于像 $x = (a - b * c * d)^{-1}(e * f + g^h) \pmod{n}$ 这样的表达式。⁷

2.3 椭圆曲线密码学 (Elliptic curve cryptography)

2.3.1 什么是椭圆曲线? (What are elliptic curves?)

一个有限域 $\mathbb{F}_q$, 其中 q 是大于 3 的素数, 是由集合 $\{0, 1, 2, \dots, q-1\}$ 构成的域。加法和乘法 $(+, \cdot)$ 以及取反 $(-)$ 是模 q 计算的。

“模 q 计算”意味着对两个域元素之间的任何算术运算实例, 或对单个域元素的取反, 都执行 $\pmod{q}$ 。例如, 给定素域 $\mathbb{F}_p$ 且 $p = 29$, $17 + 20 = 8$ 因为 $37 \pmod{29} = 8$ 。同样, $-13 = -13 \pmod{29} = 16$ 。

通常, 具有给定 (a, b) 数对的椭圆曲线定义为满足 Weierstraß 方程 [?] 的所有坐标为 (x, y) 的点的集合:⁸

$$y^2 = x^3 + ax + b \quad \text{where } a, b, x, y \in \mathbb{F}_q$$

⁷ 大数的取模可以利用模方程。事实上 $254 \pmod{13} \equiv 2 * 10 * 10 + 5 * 10 + 4 \equiv (((2) * 10 + 5) * 10 + 4) \pmod{13}$ 。当 $a > n$ 时, $a \pmod{n}$ 的算法为:

1. 定义 $A \rightarrow [a_{decimal}]$; $r = 0$

2. 对于 $i = A_{size} - 1, \dots, 0$

(a) $r = (r * 10 + A[i]) \pmod{n}$

3. 使用最终的 r 作为结果。

⁸ 符号说明: $a \in \mathbb{F}$ 表示 a 是域 $\mathbb{F}$ 中的某个元素。

加密货币 Monero 使用一种属于所谓扭曲 Edwards 曲线 [?] 类别的特殊曲线, 通常表示为 (对于给定的 (a, d) 数对):

$$ax^2 + y^2 = 1 + dx^2y^2 \quad \text{where } a, d, x, y \in \mathbb{F}_q$$

在下文中我们将偏好第二种形式。它相对于前面提到的 Weierstraß 形式的优势在于, 基本密码学原语需要更少的算术运算, 从而产生更快的密码学算法 (详见 Bernstein 等人在 [?] 中的论述)。

设 $P_1 = (x_1, y_1)$ 和 $P_2 = (x_2, y_2)$ 是属于扭曲 Edwards 椭圆曲线 (以下简称为 EC) 的两个点。我们通过定义 $P_1 + P_2 = (x_1, y_1) + (x_2, y_2)$ 为点 $P_3 = (x_3, y_3)$ 来定义点加法, 其中⁹

$$\begin{aligned} x_3 &= \frac{x_1y_2 + y_1x_2}{1 + dx_1x_2y_1y_2} \pmod{q} \\ y_3 &= \frac{y_1y_2 - ax_1x_2}{1 - dx_1x_2y_1y_2} \pmod{q} \end{aligned}$$

这些加法公式也适用于点倍乘; 即当 $P_1 = P_2$ 时。要减去一个点, 将其坐标沿 y 轴取反, $(x, y) \rightarrow (-x, y)$ [?], 然后使用点加法。回忆一下, $\mathbb{F}_q$ 中的“负”元素 $-x$ 实际上是 $-x \pmod{q}$ 。

每当两个曲线点相加时, P_3 是“原始”椭圆曲线上的一个点, 换句话说, 所有 $x_3, y_3 \in \mathbb{F}_q$ 并满足 EC 方程。

EC 中的每个点 P 都可以使用自身的倍数从 EC 中的其他点生成一个阶 (大小) 为 u 的子群。例如, 某个点 P 的子群可能有阶 5, 包含点 $(0, P, 2P, 3P, 4P)$, 每个都在 EC 中。在 $5P$ 处, 所谓的无穷远点出现, 它类似于 EC 上的“零”位置, 坐标为 $(0, 1)$ 。¹⁰

方便的是, $5P + P = P$ 。这意味着该子群是循环的。¹¹ EC 中的所有 P 都生成一个循环子群。如果 P 生成的子群的阶是素数, 则所有包含的点 (除了无穷远点) 都生成同一个子群。在我们的例子中, 取点 $2P$ 的倍数:

$$2P, 4P, 6P, 8P, 10P \rightarrow 2P, 4P, 1P, 3P, 0$$

另一个例子: 阶为 6 的子群 $(0, P, 2P, 3P, 4P, 5P)$ 。点 $2P$ 的倍数:

$$2P, 4P, 6P, 8P, 10P, 12P \rightarrow 2P, 4P, 0, 2P, 4P, 0$$

这里 $2P$ 的阶为 3。由于 6 不是素数, 并非所有成员点都能重建原始子群。

每个 EC 都有一个等于曲线上总点数 (包括无穷远点) 的阶 N , 由点生成的所有子群的阶都是 N 的因数 (根据拉格朗日定理)。我们可以想象所有 EC 点的集合 $\{0, P_1, \dots, P_{N-1}\}$ 。如果 N 不是素数, 某些点将生成阶等于 N 因数的子群。

要找到任意给定点 P 的子群的阶 u :

⁹ 通常椭圆曲线点在进行点加法等曲线运算之前会被转换为射影坐标, 以避免执行域求逆运算以提高效率。[?]

¹⁰ 事实上, 在上述加法运算下, 椭圆曲线具有阿贝尔群结构, 因为它们的无穷远点是单位元。该概念的简洁定义可在 <https://brilliant.org/wiki/abelian-group/> 找到。

¹¹ 循环子群意味着, 对于 P 的阶为 u 的子群, 以及任意整数 n , $nP = [n \pmod{u}]P$ 。我们可以想象自己站在地球上距某个“零”位置一定距离的一个点上, 每走一步就移动那个距离。过一段时间后, 我们会回到出发点, 尽管可能需要绕很多圈才能精确回到原来的位置。回到完全相同位置所需的步数就是“步进群”的“阶”, 我们所有的脚印都是该群中的唯一点。我们建议将此概念应用到这里讨论的其他想法中。

1. 找到 N (例如使用 *Schoof* 算法)。
2. 找到 N 的所有因数。
3. 对于 N 的每个因数 n , 计算 nP 。
4. 使得 $nP = 0$ 的最小 n 即为子群的阶 u 。

用于密码学的 EC 通常具有 $N = hl$, 其中 l 是某个足够大的素数 (如 160 位), h 是所谓的余因子, 可以小到 1 或 2。¹² 大小为 l 的子群中的一个点通常被选为生成元 G , 这是一种约定。对于该子群中的每个其他点 P , 都存在一个满足 $P = nG$ 的整数 $0 < n \leq l$ 。

让我们扩展我们的理解。假设有一个阶为 N 的点 P' , 其中 $N = hl$ 。任何其他点 P_i 都可以用某个整数 n_i 找到, 使得 $P_i = n_i P'$ 。如果 $P_1 = n_1 P'$ 的阶为 l , 那么任何阶为 l 的 $P_2 = n_2 P'$ 必须与 P_1 在同一个子群中, 因为 $lP_1 = 0 = lP_2$, 并且如果 $l(n_1 P') \equiv l(n_2 P') \equiv NP' = 0$, 那么 n_1 和 n_2 必须都是 h 的倍数。由于 $N = hl$, 只有 l 个 h 的倍数, 意味着只可能有一个大小为 l 的子群。

简单来说, 由 (hP') 的倍数形成的子群始终包含 P_1 和 P_2 。此外, 当 n' 是 l 的倍数时, $h(n'P') = 0$, 并且 $n' \pmod{N}$ 只有 h 种这样的变化 (包括 $n' = hl$ 时的无穷远点), 因为当 $n' = hl$ 时它循环回到 0: $hlP' = 0$ 。所以, EC 中只有 h 个点 P 使得 hP 等于 0。

类似的论证可以应用于任何大小为 u 的子群。任何两个阶为 u 的点 P_1 和 P_2 都在同一个子群中, 该子群由 $(N/u)P'$ 的倍数组成。

有了这个新的理解, 显然我们可以使用以下算法来找到阶为 l 的子群中的 (非无穷远点) 点:

1. 找到椭圆曲线 EC 的 N , 选择子群阶 l , 计算 $h = N/l$ 。
2. 在 EC 中选择一个随机点 P' 。
3. 计算 $P = hP'$ 。
4. 如果 $P = 0$ 则返回第 2 步, 否则 P 在阶为 l 的子群中。

计算任意整数 n 和任意点 P 之间的标量积 nP 并不困难, 而找到满足 $P_1 = nP_2$ 的 n 则被认为是计算困难的。类似于模运算, 这通常被称为离散对数问题 (DLP)。标量乘法可以被视为单向函数, 这为使用椭圆曲线进行密码学奠定了基础。¹³

标量积 nP 等价于 $((P + P) + (P + P) \dots)$ 。虽然不总是最高效的方法, 但我们可以像第 ?? 节中那样使用双倍加法。要获得总和 $R = nP$, 请记住我们使用第 ?? 节中讨论的 $+$ 点运算。

1. 定义 $n_{\text{scalar}} \rightarrow n_{\text{binary}}$; $A = [n_{\text{binary}}]$; $R = 0$, 即无穷远点; $S = P$
2. 遍历: $i = (0, \dots, A_{\text{size}} - 1)$

¹² 具有小余因子的 EC 允许相对更快的点加法等操作。[?]

¹³ 目前没有已知的方程或算法可以高效地 (基于现有技术) 求解 $P_1 = nP_2$ 中的 n , 这意味着仅解开一个标量积就需要很多很多年。

- (a) 如果 $A[i] == 1$, 则 $R += S$ 。
 - (b) 计算 $S += S$ 。
3. 使用最终的 R 作为结果。

注意, 大小为 l 的子群中的 EC 点 (我们将从此使用) 的标量是有限域 $\mathbb{F}_l$ 的成员。这意味着标量之间的算术运算取模 l 。

2.3.2 椭圆曲线公钥密码学 (Public key cryptography with elliptic curves)

公钥密码学算法可以用类似于模运算的方式来设计。

设 k 为满足 $0 < k < l$ 的随机选取的数, 称之为私钥。¹⁴通过标量积 $kG = K$ 计算对应的公钥 K (一个 EC 点)。

由于离散对数问题 (DLP), 我们无法轻易从 K 单独推导出 k 。这一属性允许我们在标准公钥密码学算法中使用值 (k, K) 。

2.3.3 椭圆曲线 Diffie-Hellman 密钥交换 (Diffie-Hellman key exchange with elliptic curves)

Alice 和 Bob 之间的基本 *Diffie-Hellman* [?] 共享密钥交换可以按以下方式进行:

1. Alice 和 Bob 各自生成自己的私钥/公钥对 (k_A, K_A) 和 (k_B, K_B) 。双方公开或交换各自的公钥, 并将私钥保留给自己。
2. 显然, 有

$$S = k_A K_B = k_A k_B G = k_B k_A G = k_B K_A$$

Alice 可以私下计算 $S = k_A K_B$, Bob 计算 $S = k_B K_A$, 从而使他们能够将这个单一值用作共享密钥。

例如, 如果 Alice 有一条消息 m 要发送给 Bob, 她可以对共享密钥进行 hash $h = \mathcal{H}(S)$, 计算 $x = m + h$, 并将 x 发送给 Bob。Bob 计算 $h' = \mathcal{H}(S)$, 计算 $m = x - h'$, 从而获得 m 。

外部观察者由于“Diffie-Hellman 问题” (DHP) 将无法轻易计算共享密钥, DHP 表示从 K_A 和 K_B 找到 S 非常困难。此外, DLP 阻止了他们找到 k_A 或 k_B 。¹⁵

¹⁴ 私钥有时被称为密钥。这使我们可以缩写: pk = 公钥, sk = 密钥。

¹⁵ DHP 被认为至少与 DLP 具有相当的难度, 尽管尚未被证明。[?]

2.3.4 Schnorr 签名和 Fiat-Shamir 变换 (Schnorr signatures and the Fiat-Shamir transform)

1989 年, Claus-Peter Schnorr 发表了一个现在著名的交互式认证协议 [?], 后由 Maurer 在 2009 年进行了推广 [?], 该协议允许某人证明自己知道给定公钥 K 的私钥 k 而不泄露任何关于它的信息 [?]. 它大致如下:

1. 证明者生成一个随机整数 $\alpha \in_R \mathbb{Z}_l$, ¹⁶ 计算 αG , 并将 αG 发送给验证者。
2. 验证者生成一个随机挑战 $c \in_R \mathbb{Z}_l$ 并将 c 发送给证明者。
3. 证明者计算响应 $r = \alpha + c * k$ 并将 r 发送给验证者。
4. 验证者计算 $R = rG$ 和 $R' = \alpha G + c * K$, 并检查 $R \stackrel{?}{=} R'$ 。

验证者可以在证明者之前计算 $R' = \alpha G + c * K$, 所以提供 c 就像是在说, ”我挑战你用 R' 的离散对数来回应。” 这是一个只有知道 k 的证明者才能克服的挑战 (除了可忽略的概率外)。

如果 α 是由证明者随机选择的, 那么 r 是随机分布的 [?], 并且 k 在 r 中是信息论安全的 (仍然可以通过对 K 或 αG 求解 DLP 来找到它)。¹⁷ 然而, 如果证明者重复使用 α 来证明他对 k 的知识, 任何知道 $r = \alpha + c * k$ 和 $r' = \alpha + c' * k$ 中两个挑战的人都可以计算 k (两个方程, 两个未知数)。¹⁸

$$k = \frac{r - r'}{c - c'}$$

如果证明者从一开始就知道 c (例如, 如果验证者秘密地给了她), 她可以生成一个随机响应 r 并计算 $\alpha G = rG - cK$ 。当她后来将 r 发送给验证者时, 她在从未需要知道 k 的情况下”证明”了对 k 的知识。观察证明者和验证者之间事件记录的人完全不会知情。该方案不是公开可验证的。[?]

在验证者作为挑战者的角色中, 他在接收到 αG 后输出一个随机数, 使他等价于一个随机函数。随机函数, 如 hash 函数, 被称为随机预言机, 因为计算一个就像是在向某人请求一个随机数 [?].¹⁹

使用 hash 函数 (而非验证者) 来生成挑战被称为 *Fiat-Shamir* 变换 [?], 因为它使交互式证明变为非交互式且公开可验证 [?].^{20,21}

¹⁶ 符号说明: $\alpha \in_R \mathbb{Z}_l$ 中的 R 表示 α 是从 $\{1, 2, 3, \dots, l-1\}$ 中随机选取的。换句话说, $\mathbb{Z}_l$ 是所有整数 $(\text{mod } l)$ 。我们排除了 ‘0’, 因为无穷远点在这里没有用。

¹⁷ 具有信息论安全性的密码系统是指即使拥有无限计算能力的手也无法破解它, 因为他们根本没有足够的信息。

¹⁸ 如果证明者是一台计算机, 你可以想象有人在计算机生成 α 后”克隆”/复制该计算机, 然后向每个副本提出不同的挑战。

¹⁹ 更一般地, “[在密码学中……预言机是任何可以提供关于系统的额外信息的系统, 而这些信息否则无法获得。”[?]

²⁰ 密码学 hash 函数 $\mathcal{H}$ 的输出在可能的输出范围内均匀分布。也就是说, 对于某个输入 A , $\mathcal{H}(A) \in_R \mathcal{S}_H$, 其中 $\mathcal{S}_H$ 是 $\mathcal{H}$ 的可能输出集合。我们使用 $\in_R$ 表示该函数是确定性随机的。 $\mathcal{H}(A)$ 每次产生相同的结果, 但其输出等价于一个随机数。

²¹ 注意, 非交互式类 Schnorr 证明 (和签名) 需要使用固定的生成元 G , 或将生成元包含在挑战 hash 中。包含生成元的方式被称为密钥前缀, 我们在后面会进一步讨论 (第 ?? 和 ?? 节)。

非交互式证明 (Non-interactive proof)

1. 生成随机数 $\alpha \in_R \mathbb{Z}_l$, 并计算 αG 。
2. 使用密码学安全 hash 函数计算挑战, $c = \mathcal{H}([\alpha G])$ 。
3. 定义响应 $r = \alpha + c * k$ 。
4. 公开证明对 $(\alpha G, r)$ 。

验证 (Verification)

1. 计算挑战: $c' = \mathcal{H}([\alpha G])$ 。
2. 计算 $R = rG$ 和 $R' = \alpha G + c' * K$ 。
3. 如果 $R = R'$, 则证明者必须知道 k (除了可忽略的概率外)。

为什么有效 (Why it works)

$$\begin{aligned}
 rG &= (\alpha + c * k)G \\
 &= (\alpha G) + (c * kG) \\
 &= \alpha G + c * K \\
 R &= R'
 \end{aligned}$$

任何证明/签名方案的一个重要部分是验证它们所需的资源。这包括存储证明的空间和验证所花费的时间。在此方案中, 我们存储一个 EC 点和一个整数, 并需要知道公钥——另一个 EC 点。由于 hash 函数计算速度相对较快, 请记住验证时间主要是椭圆曲线运算的函数。

2.3.5 对消息签名 (Signing messages)

通常, 密码学签名是对消息的密码学 hash 而非消息本身执行的, 这有助于对不同大小的消息进行签名。然而, 在本报告中, 我们将宽泛地使用术语“消息”及其符号 m 来指代消息本身和/或其 hash 值, 除非另有说明。

对消息签名是互联网安全的基础, 它让消息接收者确信其内容与签名者的意图一致。一种常见的签名方案叫做 ECDSA。参见 [?], ANSI X9.62 和 [?]

我们在此提出的签名方案是之前变换后的 Schnorr 证明的另一种表述。以这种方式思考签名为我们在下一章探索环签名做好了准备。

签名 (Signature)

假设 Alice 拥有私钥/公钥对 (k_A, K_A) 。为了明确地签署任意消息 m ，她可以执行以下步骤：

1. 生成随机数 $\alpha \in_R \mathbb{Z}_l$ ，并计算 αG 。
2. 使用密码学安全 hash 函数计算挑战， $c = \mathcal{H}(m, [\alpha G])$ 。
3. 定义响应 r 使得 $\alpha = r + c * k_A$ 。换句话说， $r = \alpha - c * k_A$ 。
4. 公开签名 (c, r) 。

验证 (Verification)

任何知道 EC 域参数（指定使用了哪条椭圆曲线）、签名 (c, r) 和签名方法、 m 和 hash 函数以及 K_A 的第三方都可以验证签名：

1. 计算挑战： $c' = \mathcal{H}(m, [rG + c * K_A])$ 。
2. 如果 $c = c'$ ，则签名通过。

在此签名方案中，我们存储两个标量，并需要一个公钥。

为什么有效 (Why it works)

这源于以下事实

$$\begin{aligned}
 rG &= (\alpha - c * k_A)G \\
 &= \alpha G - c * K_A \\
 \alpha G &= rG + c * K_A \\
 \mathcal{H}_n(m, [\alpha G]) &= \mathcal{H}_n(m, [rG + c * K_A]) \\
 c &= c'
 \end{aligned}$$

因此 k_A 的所有者 (Alice) 为 m 创建了 (c, r) ：她对消息进行了签名。其他人——一个没有 k_A 的伪造者——能够制作 (c, r) 的概率可以忽略不计，因此验证者可以确信消息没有被篡改。

2.4 Ed25519 曲线 (Curve Ed25519)

Monero 使用一种特定的扭曲 Edwards 椭圆曲线进行密码学运算，即 *Ed25519*，它是 Montgomery 曲线 *Curve25519* 的双有理等价曲线²²。

²² 不作进一步详述，双有理等价可以被理解为可用有理项表达的同构。

Curve25519 和 Ed25519 均由 Bernstein 等人发布 [?, ?, ?]。²³

该曲线定义在素域 $\mathbb{F}_{2^{255}-19}$ 上 (即 $q = 2^{255} - 19$), 通过以下方程:

$$-x^2 + y^2 = 1 - \frac{121665}{121666}x^2y^2$$

这条曲线解决了密码学社区提出的许多问题。²⁴众所周知, NIST²⁵ 标准算法存在问题。例如, 最近已清楚 NIST 标准随机数生成算法 PNRG (基于椭圆曲线的版本) 存在缺陷并包含潜在的后门 [?]. 从更广泛的视角来看, NIST 等标准化机构导致了密码学的单一文化, 引入了中心化问题。一个很好的例子是 NSA 利用其对 NIST 的影响力来削弱国际密码学标准 [?].

Ed25519 曲线不受任何专利约束 (参见 [?] 关于此主题的讨论), 其背后的团队已经开发并调整了以效率为导向的基本密码学算法 [?].

扭曲 Edwards 曲线的阶可表示为 $N = 2^cl$, 其中 l 是素数, c 是正整数。在 Ed25519 曲线的情况下, 其阶是一个 76 位数 (l 为 253 位): ²⁶

$$2^3 \cdot 7237005577332262213973186563042994240857116359379907606001950938285454250989$$

2.4.1 二进制表示 (Binary representation)

$\mathbb{F}_{2^{255}-19}$ 的元素被编码为 256 位整数, 因此可以用 32 字节表示。由于每个元素只需要 255 位, 最高有效位始终为零。

因此, Ed25519 中的任何点都可以用 64 字节表示。然而, 通过应用下面描述的点压缩技术, 可以将此量减半至 32 字节。

2.4.2 点压缩 (Point compression)

Ed25519 曲线具有其点可以被轻松压缩的特性, 使得表示一个点仅消耗一个坐标的空间。我们将不深入证明这一特性所需的数学, 但可以简要介绍其工作原理 [?]. Ed25519 曲线的点压缩首次在 [?] 中描述, 而该概念首次在 [?] 中引入。

这种点压缩方案源于扭曲 Edwards 曲线方程的变换 (假设 $a = -1$, 这对 Monero 成立): $x^2 = (y^2 - 1)/(dy^2 + 1)$, ²⁷这表明对于每个 y 有两个可能的 x 值 (+ 或 -)。域元素 x 和 y 是 $(\text{mod } q)$ 计算的, 因此没有实际的负值。然而, 对 $-x$ 取 $(\text{mod } q)$ 会在奇偶之间改变值, 因为 q 是奇数。例如: $3 \pmod{5} = 3$, $-3 \pmod{5} = 2$ 。换句话说, 域元素 x 和 $-x$ 具有不同的奇偶分配。

²³ Bernstein 博士还开发了一种名为 ChaCha 的加密方案 [?, ?], Monero 的主要实现使用它来加密与用户钱包相关的某些敏感信息。

²⁴ 即使一条曲线表面上没有密码学安全问题, 创建它的人/组织也可能知道一个秘密问题, 该问题只在非常罕见的曲线中出现。这样的人可能需要随机生成许多曲线, 才能找到一条具有隐藏弱点且没有已知弱点的曲线。如果需要为曲线参数提供合理的解释, 那么找到密码学社区可以接受的弱曲线就更加困难了。Ed25519 曲线被称为“完全刚性”曲线, 这意味着其生成过程得到了充分解释。 [?]

²⁵ 美国国家标准与技术研究院, <https://www.nist.gov/>

²⁶ 这意味着 Ed25519 中的私钥为 253 位。

²⁷ 这里 $d = -\frac{121665}{121666}$ 。

如果我们有一个曲线点并知道其 x 是偶数，但给定其 y 值，变换后的曲线方程输出一个奇数，那么我们就知道对该数取反将给出正确的 x 。一位可以传达这一信息，而方便的是 y 坐标有一个额外的位。

假设我们想要压缩一个点 (x, y) 。

编码 (Encoding) 我们将 y 的最高有效位设为：如果 x 是偶数则为 0，如果是奇数则为 1。得到的值 y' 将代表该曲线点。

解码 (Decoding)

1. 获取压缩后的点 y' ，然后将其最高有效位复制到奇偶位 b ，再将其设为 0。现在它又变回了原始的 y 。
2. 设 $u = y^2 - 1 \pmod{q}$ 和 $v = dy^2 + 1 \pmod{q}$ 。这意味着 $x^2 = u/v \pmod{q}$ 。
3. 计算²⁸ $z = uv^3(uv^7)^{(q-5)/8} \pmod{q}$ 。
 - (a) 如果 $yz^2 = u \pmod{q}$ 则 $x' = z$ 。
 - (b) 如果 $yz^2 = -u \pmod{q}$ 则计算 $x' = z * 2^{(q-1)/4} \pmod{q}$ 。
4. 使用第一步中的奇偶位 b ，如果 $b \neq x'$ 的最低有效位，则 $x = -x' \pmod{q}$ ，否则 $x = x'$ 。
5. 返回解压后的点 (x, y) 。

Ed25519 的实现（如 Monero）通常使用生成元 $G = (x, 4/5)$ [?]²⁹，其中 x 是基于 $y = 4/5 \pmod{q}$ 点解压的”偶数”或 $b = 0$ 变体。

2.4.3 EdDSA 签名算法 (EdDSA signature algorithm)

Bernstein 和他的团队基于 Ed25519 曲线开发了许多基本算法。²⁹ 为了说明，我们将描述一种高度优化和安全的 ECDSA 签名方案替代方案，据作者称，使用普通的 Intel Xeon 处理器每秒可产生超过 100,000 个签名 [?]。该算法也可以在互联网 RFC8032 [?] 中找到描述。注意这是一种非常类似 Schnorr 的签名方案。

除了其他方面，它不使用每次生成随机整数，而是使用从签名者私钥和消息本身派生的 hash 值。这规避了与随机数生成器实现相关的安全缺陷。此外，该算法的另一个目标是避免访问秘密或不可预测的内存位置，以防止所谓的缓存时序攻击 [?]

我们在此提供该算法执行步骤的概述。完整的描述和 Python 语言的示例实现可在 [?] 中找到。

²⁸ 由于 $q = 2^{255} - 19 \equiv 5 \pmod{8}$ ， $(q-5)/8$ 和 $(q-1)/4$ 是整数。

²⁹ 参见 [?] 了解扭曲 Edwards EC 中的高效群运算（即点加法、倍乘、混合加法等）。参见 [?] 了解高效的模运算。

签名 (Signature)

1. 设 h_k 为签名者私钥 k 的 hash $\mathcal{H}(k)$ 。计算 α 为私钥 hash 和消息的 hash $\alpha = \mathcal{H}(h_k, \mathbf{m})$ 。根据实现, $\mathbf{m}$ 可以是实际消息或其 hash [?]
2. 计算 αG 和挑战 $ch = \mathcal{H}([\alpha G], K, \mathbf{m})$ 。
3. 计算响应 $r = \alpha + ch \cdot k$ 。
4. 签名为对 $(\alpha G, r)$ 。

验证 (Verification)

验证按如下方式执行

1. 计算 $ch' = \mathcal{H}([\alpha G], K, \mathbf{m})$ 。
2. 如果等式 $2^c r G \stackrel{?}{=} 2^c \alpha G + 2^c ch' \cdot K$ 成立, 则签名有效。

2^c 项来自 Bernstein 等人的 EdDSA 算法通用形式 [?]。根据该论文, 虽然对于充分验证来说并非必需, 但去除 2^c 可以提供更强的方程。

公钥 K 可以是任何 EC 点, 但我们只想使用生成元 G 的子群中的点。乘以余因子 2^c 可确保所有点都在该子群中。或者, 验证者可以检查 $lK \stackrel{?}{=} 0$, 这仅在 K 在子群中时才有效。我们不知道这些预防措施能捕获的任何弱点, 不过正如我们将看到的, 后一种方法在 Monero 中很重要 (第 ?? 节)。

在此签名方案中, 我们存储一个 EC 点和一个标量, 并有一个公钥。

为什么有效 (Why it works)

$$\begin{aligned} 2^c r G &= 2^c (\alpha + \mathcal{H}([\alpha G], K, \mathbf{m}) \cdot k) \cdot G \\ &= 2^c \alpha G + 2^c \mathcal{H}([\alpha G], K, \mathbf{m}) \cdot K \end{aligned}$$

二进制表示 (Binary representation)

默认情况下, EdDSA 签名需要 $64 + 32$ 字节用于 EC 点 αG 和标量 r 。然而, RFC8032 假设点 αG 被压缩, 这将空间需求减少到仅 $32 + 32$ 字节。我们包含公钥 K , 这意味着总共 $32 + 32 + 32$ 字节。

2.5 二元运算符 XOR (Binary operator XOR)

二元运算符 XOR 是一个有用的工具，将在第 ?? 和 ?? 节中出现。它接受两个参数，如果其中一个（但不是两个）为真则返回真 [?]。以下是其真值表：

A	B	A XOR B
T	T	F
T	F	T
F	T	T
F	F	F

在计算机科学的语境中，XOR 等价于模 2 的位加法。例如，两对位的 XOR：

$$\begin{aligned}\text{XOR}(\{1, 1\}, \{1, 0\}) &= \{1 + 1, 1 + 0\} \pmod{2} \\ &= \{0, 1\}\end{aligned}$$

以下也产生 $\{0, 1\}$ ： $\text{XOR}(\{1, 0\}, \{1, 1\})$ 、 $\text{XOR}(\{0, 0\}, \{0, 1\})$ 和 $\text{XOR}(\{0, 1\}, \{0, 0\})$ 。对于具有 b 位的 XOR 输入，有 $2^b - 1$ 种其他输入组合可以产生相同的输出。这意味着如果 $C = \text{XOR}(A, B)$ 且输入 $A \in_R \{0, \dots, 2^b - 1\}$ ，知道 C 的观察者不会获得关于 B 的任何信息。

同时，任何知道 $\{A, B, C\}$ 中两个元素的人（其中 $C = \text{XOR}(A, B)$ ），都可以计算第三个元素，例如 $A = \text{XOR}(B, C)$ 。XOR 指示两个元素是不同还是相同，所以知道 C 和 B 就足以暴露 A 。仔细检查真值表可以揭示这一关键特性。³⁰

³⁰ XOR 的一个有趣应用（与 Monero 无关）是在不使用第三个寄存器的情况下交换两个位寄存器。我们使用符号 $\oplus$ 表示 XOR 运算。 $A \oplus A = 0$ ，所以在寄存器之间进行三次 XOR 运算后： $\{A, B\} \rightarrow \{[A \oplus B], B\} \rightarrow \{[A \oplus B], B \oplus [A \oplus B]\} = \{[A \oplus B], A \oplus 0\} = \{[A \oplus B], A\} \rightarrow \{[A \oplus B] \oplus A, A\} = \{B, A\}$ 。

第

3

章 3

高级类 Schnorr 签名 (Advanced Schnorr-like Signatures)

一个基本的 Schnorr 签名只有一个签名密钥。事实上，我们可以应用其核心概念来创建各种复杂度递增的签名方案。其中一种方案 MLSAG 将在 Monero 的 transaction 协议中具有核心重要性。

3.1 跨多个基底证明离散对数知识 (Prove knowledge of a discrete logarithm across multiple bases)

证明同一个私钥被用于在不同“基底”密钥上构建公钥通常很有用。例如，我们可以有一个普通公钥 kG ，以及与某个人的公钥的 Diffie-Hellman 共享密钥 kR （回顾第 ?? 节），其中基底密钥是 G 和 R 。正如我们将很快看到的，我们可以证明知道 kG 中的离散对数 k ，证明知道 kR 中的 k ，并证明两种情况下的 k 是相同的（全部不泄露 k ）。

非交互式证明 (Non-interactive proof)

假设我们有一个私钥 k 和 d 个基底密钥 $\mathcal{J} = \{J_1, \dots, J_d\}$ 。对应的公钥为 $\mathcal{K} = \{K_1, \dots, K_d\}$ 。我们在所有基底上进行类 Schnorr 证明（回顾第 ?? 节）。¹假设存在一个 hash 函数 $\mathcal{H}_n$ 映射到 0 到 $l-1$ 之间的整数。²

1. 生成随机数 $\alpha \in_R \mathbb{Z}_l$ ，并计算所有 $i \in (1, \dots, d)$ 的 αJ_i 。
2. 计算挑战，

$$c = \mathcal{H}_n(\mathcal{J}, \mathcal{K}, [\alpha J_1], [\alpha J_2], \dots, [\alpha J_d])$$

3. 定义响应 $r = \alpha - c * k$ 。
4. 公开签名 (c, r) 。

验证 (Verification)

假设验证者知道 $\mathcal{J}$ 和 $\mathcal{K}$ ，他执行以下操作。

1. 计算挑战：

$$c' = \mathcal{H}(\mathcal{J}, \mathcal{K}, [rJ_1 + c * K_1], [rJ_2 + c * K_2], \dots, [rJ_d + c * K_d])$$

2. 如果 $c = c'$ ，则签名者必须知道跨所有基底的离散对数，且每种情况下都是同一个离散对数（一如既往，除了可忽略的概率外）。

为什么有效 (Why it works)

如果 d 个基底密钥只有一个，这个证明显然与我们原始的 Schnorr 证明相同（第 ?? 节）。我们可以将每个基底密钥单独来看，发现多基底证明只是一系列连接在一起的 Schnorr 证明。此外，通过对所有这些证明使用唯一的挑战 and 响应，它们必须具有相同的离散对数 k 。要获得适用于多个密钥的单一响应，挑战需要在为每个密钥定义 α 之前就被知道，但 c 是 α 的函数！

3.2 一个证明中的多个私钥 (Multiple private keys in one proof)

与多基底证明非常相似，我们可以组合多个使用不同私钥的 Schnorr 证明。这样做证明了我们知道一组公钥的所有私钥，并通过为所有证明只生成一个挑战来减少存储需求。

¹ 虽然我们说的是“证明”，但通过在挑战 hash 中包含消息 m 可以轻易地将其变为签名。在此语境中，这两个术语可以互换使用。

² 在 Monero 中，hash 函数 $\mathcal{H}_n(x) = \text{sc_reduce32}(\text{Keccak}(x))$ ，其中 Keccak 是 SHA3 的基础，sc_reduce32() 将 256 位结果放在 0 到 $l-1$ 的范围内（虽然严格来说应该是 1 到 $l-1$ ）。

非交互式证明 (Non-interactive proof)

假设我们有 d 个私钥 $k_1, \dots, k_d$, 以及基底密钥 $\mathcal{J} = \{J_1, \dots, J_d\}$ 。³ 对应的公钥为 $\mathcal{K} = \{K_1, \dots, K_d\}$ 。我们同时为所有密钥进行类 Schnorr 证明。

1. 为所有 $i \in (1, \dots, d)$ 生成随机数 $\alpha_i \in_R \mathbb{Z}_l$, 并计算所有 $\alpha_i J_i$ 。
2. 计算挑战,

$$c = \mathcal{H}_n(\mathcal{J}, \mathcal{K}, [\alpha_1 J_1], [\alpha_2 J_2], \dots, [\alpha_d J_d])$$

3. 定义每个响应 $r_i = \alpha_i - c * k_i$ 。
4. 公开签名 $(c, r_1, \dots, r_d)$ 。

验证 (Verification)

假设验证者知道 $\mathcal{J}$ 和 $\mathcal{K}$, 他执行以下操作。

1. 计算挑战:

$$c' = \mathcal{H}(\mathcal{J}, \mathcal{K}, [r_1 J_1 + c * K_1], [r_2 J_2 + c * K_2], \dots, [r_d J_d + c * K_d])$$

2. 如果 $c = c'$, 则签名者必须知道 $\mathcal{K}$ 中所有公钥的私钥 (除了可忽略的概率外)。

3.3 自发匿名群 (SAG) 签名 (Spontaneous Anonymous Group (SAG) signatures)

群签名是一种证明签名者属于某个群组而不必识别他的方式。最初 (Chaum 在 [?] 中), 群签名方案要求系统由可信人员设置, 并在某些情况下由其管理, 以防止非法签名, 并在少数方案中裁决争议。这些方案依赖于群密钥, 这是不理想的, 因为它造成了可能破坏匿名性的泄露风险。此外, 要求群成员之间的协调 (即设置和管理) 对于小群组或公司内部以外的场景是不可扩展的。

Liu 等人在 [?] 中基于 Rivest 等人 在 [?] 中的工作提出了一个更有趣的方案。作者详细介绍了一种名为 LSAG 的群签名算法, 具有三个特性: 匿名性、可链接性和自发性。这里我们讨论 SAG, 即 LSAG 的不可链接版本, 以获得概念上的清晰性。我们将可链接性的概念留到后面的章节。

具有匿名性和自发性的方案通常被称为“环签名”。在 Monero 的语境中, 它们最终将实现不可伪造的、签名者模糊的 transaction, 使资金流向在很大程度上不可追踪。

³ 这里没有理由 $\mathcal{J}$ 不能包含重复的基底密钥, 或者所有基底密钥都相同 (例如 G)。对于多基底证明来说重复是冗余的, 但现在我们处理的是不同的私钥。

签名 (Signature)

环签名由一个环和一个签名组成。每个环是一组公钥，其中一个属于签名者，其余的与之无关。签名使用该密钥环生成，任何验证它的人都无法分辨哪个环成员是实际的签名者。

我们在第 ?? 节中的类 Schnorr 签名方案可以被视为单密钥环签名。通过不立即定义 r ，而是生成一个诱饵 r' 并创建新的挑战来定义 r ，我们得到了双密钥版本。

设 m 为待签名的消息， $\mathcal{R} = \{K_1, K_2, \dots, K_n\}$ 为一组不重复的公钥（一个群/环）， k_π 为签名者的私钥，对应其公钥 $K_\pi \in \mathcal{R}$ ，其中 π 是秘密索引。

1. 生成随机数 $\alpha \in_R \mathbb{Z}_l$ 和伪响应 $r_i \in_R \mathbb{Z}_l$ ，其中 $i \in \{1, 2, \dots, n\}$ 但排除 $i = \pi$ 。

2. 计算

$$c_{\pi+1} = \mathcal{H}_n(\mathcal{R}, m, [\alpha G])$$

3. 对于 $i = \pi + 1, \pi + 2, \dots, n, 1, 2, \dots, \pi - 1$ 计算，将 $n + 1 \rightarrow 1$,

$$c_{i+1} = \mathcal{H}_n(\mathcal{R}, m, [r_i G + c_i K_i])$$

4. 定义真实响应 r_π 使得 $\alpha = r_\pi + c_\pi k_\pi \pmod{l}$ 。

环签名包含签名 $\sigma(m) = (c_1, r_1, \dots, r_n)$ 和环 $\mathcal{R}$ 。

验证 (Verification)

验证意味着证明 $\sigma(m)$ 是由 $\mathcal{R}$ 中某个公钥对应的私钥创建的有效签名（不必知道是哪一个），方式如下：

1. 对于 $i = 1, 2, \dots, n$ 迭代计算，将 $n + 1 \rightarrow 1$,

$$c'_{i+1} = \mathcal{H}_n(\mathcal{R}, m, [r_i G + c_i K_i])$$

2. 如果 $c_1 = c'_1$ ，则签名有效。注意 c'_1 是最后计算的项。

在此方案中，我们存储 $(1+n)$ 个整数并使用 n 个公钥。

为什么有效 (Why it works)

我们可以通过一个例子来非正式地说服自己该算法是有效的。考虑环 $R = \{K_1, K_2, K_3\}$ ，其中 $k_\pi = k_2$ 。首先签名：

1. 生成随机数: α, r_1, r_3
2. 初始化签名循环:

$$c_3 = \mathcal{H}_n(\mathcal{R}, \mathbf{m}, [\alpha G])$$

3. 迭代:

$$c_1 = \mathcal{H}_n(\mathcal{R}, \mathbf{m}, [r_3 G + c_3 K_3])$$

$$c_2 = \mathcal{H}_n(\mathcal{R}, \mathbf{m}, [r_1 G + c_1 K_1])$$

4. 通过响应来闭合循环: $r_2 = \alpha - c_2 k_2 \pmod{l}$

我们可以将 α 代入 c_3 来看看”环”这个词的由来:

$$c_3 = \mathcal{H}_n(\mathcal{R}, \mathbf{m}, [(r_2 + c_2 k_2)G])$$

$$c_3 = \mathcal{H}_n(\mathcal{R}, \mathbf{m}, [r_2 G + c_2 K_2])$$

然后使用 $\mathcal{R}$ 和 $\sigma(\mathbf{m}) = (c_1, r_1, r_2, r_3)$ 进行验证:

1. 我们使用 r_1 和 c_1 来计算

$$c'_2 = \mathcal{H}_n(\mathcal{R}, \mathbf{m}, [r_1 G + c_1 K_1])$$

2. 根据创建签名时的情况, 我们看到 $c'_2 = c_2$ 。使用 r_2 和 c'_2 我们计算

$$c'_3 = \mathcal{H}_n(\mathcal{R}, \mathbf{m}, [r_2 G + c'_2 K_2])$$

3. 通过将 c_2 替换为 c'_2 , 我们可以很容易看出 $c'_3 = c_3$ 。使用 r_3 和 c'_3 我们得到

$$c'_1 = \mathcal{H}_n(\mathcal{R}, \mathbf{m}, [r_3 G + c'_3 K_3])$$

不出意外: 如果我们将 c_3 替换为 c'_3 , 则 $c'_1 = c_1$ 。

3.4 Back 的可链接自发匿名群 (bLSAG) 签名 (Back's Linkable Spontaneous Anonymous Group (bLSAG) signatures)

从这里开始讨论的环签名方案展示了几个对产生 confidential transaction 有用的特性。⁴注意, ”签名者模糊性”和”不可伪造性”也适用于 SAG 签名。

签名者模糊性 (Signer Ambiguity) 观察者应该能够确定签名者必须是环的成员 (除了可忽略的概率外), 但不能确定是哪个成员。⁵Monero 用此来混淆每笔 transaction 中资金的来源。

⁴ 请记住, 所有健壮的签名方案都有包含各种特性的安全模型。这里提到的特性也许与理解 Monero 环签名的目的最为相关, 但不是对可链接环签名特性的全面概述。

⁵ 操作的匿名性通常用”匿名集”来衡量, 即”所有可能执行该操作的人”。最大的匿名集是”全人类”, 对于 Monero 来说是环的大小, 例如所谓的”混合级别” v 加上真实签名者。混合 (Mixin) 指的是每个环签名有多少个假成员。如果混合为 $v = 4$, 则有 5 个可能的签名者。扩大匿名集使得追踪真实行为者变得越来越困难。

可链接性 (Linkability) 如果一个私钥被用来签署两个不同的消息，那么这些消息将被链接在一起。⁶正如我们将要展示的，此属性用于防止 Monero 中的双花攻击（除了可忽略的概率外）。

不可伪造性 (Unforgeability) 没有攻击者能够在除了可忽略的概率外伪造签名。⁷这用于防止没有相应私钥的人盗窃 Monero 资金。

在 LSAG 签名方案 [?] 中，私钥的所有者可以为每个环产生一个匿名的、不可链接的签名。⁸在本节中，我们提出了 LSAG 算法的增强版本，其中可链接性独立于环的诱饵成员。⁹

该修改在 [?] 中被揭示，基于 Adam Back [?] 关于 CryptoNote [?] 环签名算法的出版物（此前在 Monero 中使用，现已弃用；参见第 ?? 节），而这又受到 Fujisaki 和 Suzuki 在 [?] 中工作的启发。

签名 (Signature)

与 SAG 一样，设 $\mathbf{m}$ 为待签名的消息， $\mathcal{R} = \{K_1, K_2, \dots, K_n\}$ 为一组不重复的公钥， k_π 为签名者的私钥，对应其公钥 $K_\pi \in \mathcal{R}$ ，其中 π 是秘密索引。假设存在一个 hash 函数 $\mathcal{H}_p$ ，映射到 EC 中的曲线点。^{10,11}

1. 计算 key image $\tilde{K} = k_\pi \mathcal{H}_p(K_\pi)$ 。¹²
2. 生成随机数 $\alpha \in_R \mathbb{Z}_l$ 和随机数 $r_i \in_R \mathbb{Z}_l$ ，其中 $i \in \{1, 2, \dots, n\}$ 但排除 $i = \pi$ 。
3. 计算

$$c_{\pi+1} = \mathcal{H}_n(\mathbf{m}, [\alpha G], [\alpha \mathcal{H}_p(K_\pi)])$$
4. 对于 $i = \pi + 1, \pi + 2, \dots, n, 1, 2, \dots, \pi - 1$ 计算，将 $n + 1 \rightarrow 1$ ，

$$c_{i+1} = \mathcal{H}_n(\mathbf{m}, [r_i G + c_i K_i], [r_i \mathcal{H}_p(K_i) + c_i \tilde{K}])$$
5. 定义 $r_\pi = \alpha - c_\pi k_\pi \pmod{l}$ 。

签名为 $\sigma(\mathbf{m}) = (c_1, r_1, \dots, r_n)$ ，附带 key image $\tilde{K}$ 和环 $\mathcal{R}$ 。

⁶ 可链接性属性不适用于非签名公钥。也就是说，公钥已被用于不同环签名的环成员不会导致链接。

⁷ 某些环签名方案，包括 Monero 中的方案，对于自适应选择消息和自适应选择公钥攻击具有强安全性。攻击者即使能够获取所选消息的合法签名，并且这些签名对应于他选择的环中的特定公钥，也无法发现如何伪造哪怕一条消息的签名。这被称为存在不可伪造性；参见 [?] 和 [?]

⁸ 在 LSAG 方案中，可链接性仅适用于使用相同成员且相同顺序的环的签名，即“完全相同的环”。实际上是“每个环成员每个环一个匿名签名”。即使为不同的消息制作，签名也可以被链接。

⁹ LSAG 在本报告第一版中有讨论。[?]

¹⁰ $\mathcal{H}_p$ 产生的点是否压缩并不重要。它们总是可以被解压的。

¹¹ Monero 使用一个直接返回曲线点的 hash 函数，而不是计算某个整数然后乘以 G 。如果有人发现了找到 n_x 使得 $n_x G = \mathcal{H}_p(x)$ 的方法， $\mathcal{H}_p$ 就会被破解。参见 [?] 中的算法描述。根据 CryptoNote 白皮书 [?]，其来源是这篇论文：[?]

¹² 在 Monero 中，使用 hash 到点函数来计算 key image 而不是另一个基底点非常重要，这样线性性就不会导致由相同地址（即使是一次性地址）创建的签名被链接。参见 [?] 第 18 页。

验证 (Verification)

验证意味着证明 $\sigma(\mathbf{m})$ 是由 $\mathcal{R}$ 中某个公钥对应的私钥创建的有效签名，方式如下：

1. 检查 $l\tilde{K} \stackrel{?}{=} 0$ 。

2. 对于 $i = 1, 2, \dots, n$ 迭代计算，将 $n + 1 \rightarrow 1$,

$$c'_{i+1} = \mathcal{H}_n(\mathbf{m}, [r_i G + c_i K_i], [r_i \mathcal{H}_p(K_i) + c_i \tilde{K}])$$

3. 如果 $c_1 = c'_1$ ，则签名有效。

在此方案中，我们存储 $(1+n)$ 个整数，有一个 EC key image，并使用 n 个公钥。

我们必须检查 $l\tilde{K} \stackrel{?}{=} 0$ ，因为可以将大小为 h （余因子）的子群中的 EC 点加到 $\tilde{K}$ 上，并且如果所有 c_i 都是 h 的倍数（我们可以使用不同的 α 和 r_i 值通过自动试错来实现），就可以使用相同的环和签名密钥制作 h 个不可链接的有效签名。¹³这是因为 EC 点乘以其子群的阶等于零。¹⁴

明确地说，给定阶为 l 的子群中的某个点 K ，阶为 h 的某个点 K^h ，以及一个能被 h 整除的整数 c ：

$$\begin{aligned} c * (K + K^h) &= cK + cK^h \\ &= cK + 0 \end{aligned}$$

我们可以用与更简单的 SAG 签名方案类似的方式展示正确性（即“它是如何工作的”）。

我们的描述力求忠实于 bLSAG 的原始解释，该解释在计算 c_i 的 hash 中不包含 $\mathcal{R}$ 。将密钥包含在 hash 中被称为“密钥前缀”。最近的研究 [?] 表明这可能不是必需的，尽管添加前缀是类似签名方案的标准做法（LSAG 使用密钥前缀）。

可链接性 (Linkability)

给定两个在某些方面不同的有效签名（例如不同的伪响应、不同的消息、不同的整体环成员），

$$\sigma(\mathbf{m}) = (c_1, r_1, \dots, r_n) \text{ with } \tilde{K}, \text{ and}$$

$$\sigma'(\mathbf{m}') = (c'_1, r'_1, \dots, r'_{n'}) \text{ with } \tilde{K}',$$

如果 $\tilde{K} = \tilde{K}'$ ，则显然两个签名来自同一个私钥。

虽然观察者可以将 σ 和 σ' 链接起来，但他不一定知道 $\mathcal{R}$ 或 $\mathcal{R}'$ 中的哪个 K_i 是罪魁祸首，除非它们之间只有一个共同密钥。如果有多个共同环成员，他唯一的办法就是求解 DLP 或以某种方式审计环（例如学习所有 $i \neq \pi$ 的 k_i ，或学习 k_π ）。¹⁵

¹³ 我们不关心来自其他子群的点，因为 $\mathcal{H}_n$ 的输出限制在 $\mathbb{Z}_l$ 中。对于 EC 阶 $N = hl$ ， N 的所有因数（以及因此可能的子群）要么是 l （素数）的倍数，要么是 h 的因数。

¹⁴ 在 Monero 的早期历史中，这没有被检查。幸运的是，在 2017 年 4 月修复实现（协议第 5 版）之前，这个漏洞没有被利用 [?]

¹⁵ LSAG 与 bLSAG 非常相似，是不可伪造的，这意味着没有攻击者可以在不知道私钥的情况下制作有效的环签名。如果他发明了一个假的 $\tilde{K}$ 并用 $c_{\pi+1}$ 初始化他的签名计算，那么，由于不知道 k_π ，他无法计算一个数 $r_\pi = \alpha - c_\pi k_\pi$ 来产生 $[r_\pi G + c_\pi K_\pi] = \alpha G$ 。验证者会拒绝他的签名。Liu 等人证明，能够通过验证的伪造行为概率极低 [?]

3.5 多层可链接自发匿名群 (MLSAG) 签名 (Multilayer Linkable Spontaneous Anonymous Group (MLSAG) signatures)

为了签署 transaction, 必须使用多个私钥进行签名。在 [?] 中, Shen Noether 等人描述了一种 bLSAG 签名方案的多层推广, 适用于我们有 $n \cdot m$ 个密钥集合的情况; 即集合

$$\mathcal{R} = \{K_{i,j}\} \quad \text{for } i \in \{1, 2, \dots, n\} \quad \text{and } j \in \{1, 2, \dots, m\}$$

其中我们知道对应于子集 $\{K_{\pi,j}\}$ 的 m 个私钥 $\{k_{\pi,j}\}$, 对于某个索引 $i = \pi$ 。如果我们推广可链接性的概念, 这样的算法将满足我们的需求。

可链接性 (Linkability) 如果任何私钥 $k_{\pi,j}$ 被用于 2 个不同的签名中, 则这些签名将被自动链接。

签名 (Signature)

1. 为所有 $j \in \{1, 2, \dots, m\}$ 计算 key image $\tilde{K}_j = k_{\pi,j} \mathcal{H}_p(K_{\pi,j})$ 。
2. 生成随机数 $\alpha_j \in_R \mathbb{Z}_l$, 以及 $r_{i,j} \in_R \mathbb{Z}_l$, 其中 $i \in \{1, 2, \dots, n\}$ (排除 $i = \pi$) 和 $j \in \{1, 2, \dots, m\}$ 。
3. 计算¹⁶

$$c_{\pi+1} = \mathcal{H}_n(\mathbf{m}, [\alpha_1 G], [\alpha_1 \mathcal{H}_p(K_{\pi,1})], \dots, [\alpha_m G], [\alpha_m \mathcal{H}_p(K_{\pi,m})])$$

4. 对于 $i = \pi + 1, \pi + 2, \dots, n, 1, 2, \dots, \pi - 1$ 计算, 将 $n + 1 \rightarrow 1$,

$$c_{i+1} = \mathcal{H}_n(\mathbf{m}, [r_{i,1}G + c_i K_{i,1}], [r_{i,1} \mathcal{H}_p(K_{i,1}) + c_i \tilde{K}_1], \dots, [r_{i,m}G + c_i K_{i,m}], [r_{i,m} \mathcal{H}_p(K_{i,m}) + c_i \tilde{K}_m])$$

5. 定义所有 $r_{\pi,j} = \alpha_j - c_{\pi} k_{\pi,j} \pmod{l}$ 。

签名为 $\sigma(\mathbf{m}) = (c_1, r_{1,1}, \dots, r_{1,m}, \dots, r_{n,1}, \dots, r_{n,m})$, 附带 key image $(\tilde{K}_1, \dots, \tilde{K}_m)$ 。

验证 (Verification)

签名的验证按 ([?] 的方式进行:

1. 对所有 $j \in \{1, \dots, m\}$ 检查 $l \tilde{K}_j \stackrel{?}{=} 0$ 。
2. 对于 $i = 1, \dots, n$ 计算, 将 $n + 1 \rightarrow 1$,

$$c'_{i+1} = \mathcal{H}_n(\mathbf{m}, [r_{i,1}G + c_i K_{i,1}], [r_{i,1} \mathcal{H}_p(K_{i,1}) + c_i \tilde{K}_1], \dots, [r_{i,m}G + c_i K_{i,m}], [r_{i,m} \mathcal{H}_p(K_{i,m}) + c_i \tilde{K}_m])$$
3. 如果 $c_1 = c'_1$, 则签名有效。

¹⁶ Monero MLSAG 使用密钥前缀。每个挑战包含如下显式公钥 (添加了 bLSAG 中缺失的 K 项; key image 包含在签名的消息中):

$$c_{\pi+1} = \mathcal{H}_n(\mathbf{m}, K_{\pi,1}, [\alpha_1 G], [\alpha_1 \mathcal{H}_p(K_{\pi,1})], \dots, K_{\pi,m}, [\alpha_m G], [\alpha_m \mathcal{H}_p(K_{\pi,m})])$$

为什么有效 (Why it works)

正如 SAG 算法一样, 我们可以容易地观察到

- 如果 $i \neq \pi$, 则显然值 c'_{i+1} 是按照签名算法中描述的方式计算的。
- 如果 $i = \pi$, 由于 $r_{\pi,j} = \alpha_j - c_{\pi}k_{\pi,j}$ 闭合了循环,

$$r_{\pi,j}G + c_{\pi}K_{\pi,j} = (\alpha_j - c_{\pi}k_{\pi,j})G + c_{\pi}K_{\pi,j} = \alpha_jG$$

并且

$$r_{\pi,j}\mathcal{H}_p(K_{\pi,j}) + c_{\pi}\tilde{K}_j = (\alpha_j - c_{\pi}k_{\pi,j})\mathcal{H}_p(K_{\pi,j}) + c_{\pi}\tilde{K}_j = \alpha_j\mathcal{H}_p(K_{\pi,j})$$

换句话说, $c'_{\pi+1} = c_{\pi+1}$ 也成立。

可链接性 (Linkability)

如果私钥 $k_{\pi,j}$ 被重复使用来制作任何签名, 签名中提供的对应 key image $\tilde{K}_j$ 将揭示这一点。这一观察符合我们对可链接性的推广定义。¹⁷

空间需求 (Space requirements)

在此方案中, 我们存储 $(1+m \cdot n)$ 个整数, 有 m 个 EC key image, 并使用 $m \cdot n$ 个公钥。

3.6 简洁可链接自发匿名群 (CLSAG) 签名 (Concise Linkable Spontaneous Anonymous Group (CLSAG) signatures)

CLSAG [?] ¹⁸是介于 bLSAG 和 MLSAG 之间的一种方案。假设你有一个”主”密钥, 与之相关联的是几个”辅助”密钥。证明知道所有私钥很重要, 但可链接性仅适用于主密钥。这种可链接性收缩允许比 MLSAG 更小、更快的签名。

与 MLSAG 一样, 我们有一个 $n \cdot m$ 个密钥的集合 (n 是环的大小, m 是签名密钥的数量), 主密钥位于索引 1。换句话说, 有 n 个主密钥, 第 π 个主密钥及其辅助密钥将进行签名。

$$\mathcal{R} = \{K_{i,j}\} \quad \text{for } i \in \{1, 2, \dots, n\} \quad \text{and } j \in \{1, 2, \dots, m\}$$

我们知道对应于子集 $\{K_{\pi,j}\}$ 的私钥 $\{k_{\pi,j}\}$, 对于某个索引 $i = \pi$ 。

¹⁷ 与 bLSAG 一样, 链接的 MLSAG 签名不会指明使用了哪个公钥进行签名。然而, 如果链接的 key image 的子循环环只有一个共同密钥, 那么罪魁祸首就是显而易见的。如果罪魁祸首被识别, 两个签名的所有其他签名成员都会被揭露, 因为它们共享罪魁祸首的索引。

¹⁸ 本节所依据的论文是一份正在定稿以供外部评审的预印本。CLSAG 作为 MLSAG 在未来协议版本中的替代方案很有前景, 但尚未实现, 也可能不会在未来实现。

签名 (Signature)

1. 为所有 $j \in \{1, 2, \dots, m\}$ 计算 key image $\tilde{K}_j = k_{\pi,j} \mathcal{H}_p(K_{\pi,1})$ 。注意基底密钥始终相同，因此 $j > 1$ 的 key image 是“辅助 key image”。为简化符号，我们统称它们为 $\tilde{K}_j$ 。
2. 生成随机数 $\alpha \in_R \mathbb{Z}_l$ ，以及 $r_i \in_R \mathbb{Z}_l$ ，其中 $i \in \{1, 2, \dots, n\}$ （排除 $i = \pi$ ）。
3. 为 $i \in \{1, 2, \dots, n\}$ 计算聚合公钥 W_i 和聚合 key image $\tilde{W}$ ¹⁹

$$W_i = \sum_{j=1}^m \mathcal{H}_n(T_j, \mathcal{R}, \tilde{K}_1, \dots, \tilde{K}_m) * K_{i,j}$$

$$\tilde{W} = \sum_{j=1}^m \mathcal{H}_n(T_j, \mathcal{R}, \tilde{K}_1, \dots, \tilde{K}_m) * \tilde{K}_j$$

其中 $w_\pi = \sum_j \mathcal{H}_n(T_j, \dots) * k_{\pi,j}$ 是聚合私钥。

4. 计算

$$c_{\pi+1} = \mathcal{H}_n(T_c, \mathcal{R}, \mathbf{m}, [\alpha G], [\alpha \mathcal{H}_p(K_{\pi,1})])$$

5. 对于 $i = \pi + 1, \pi + 2, \dots, n, 1, 2, \dots, \pi - 1$ 计算，将 $n + 1 \rightarrow 1$,

$$c_{i+1} = \mathcal{H}_n(T_c, \mathcal{R}, \mathbf{m}, [r_i G + c_i W_i], [r_i \mathcal{H}_p(K_{i,1}) + c_i \tilde{W}])$$

6. 定义 $r_\pi = \alpha - c_\pi w_\pi \pmod{l}$ 。

因此 $\sigma(\mathbf{m}) = (c_1, r_1, \dots, r_n)$ ，附带主 key image $\tilde{K}_1$ 和辅助 image $(\tilde{K}_2, \dots, \tilde{K}_m)$ 。

验证 (Verification)

签名的验证按以下方式进行：

1. 对所有 $j \in \{1, \dots, m\}$ 检查 $l\tilde{K}_j \stackrel{?}{=} 0$ 。²⁰
2. 为 $i \in \{1, 2, \dots, n\}$ 计算聚合公钥 W_i 和聚合 key image $\tilde{W}$

$$W_i = \sum_{j=1}^m \mathcal{H}_n(T_j, \mathcal{R}, \tilde{K}_1, \dots, \tilde{K}_m) * K_{i,j}$$

$$\tilde{W} = \sum_{j=1}^m \mathcal{H}_n(T_j, \mathcal{R}, \tilde{K}_1, \dots, \tilde{K}_m) * \tilde{K}_j$$

¹⁹ CLSAG 论文建议使用不同的 hash 函数来进行域分离，我们通过在每个 hash 前加标签来建模 [?]，例如 $T_1 = \text{"CLSAG_1"}$ ， $T_c = \text{"CLSAG_c"}$ 等。域分离的 hash 函数即使输入相同也产生不同的输出。我们这里也使用密钥前缀（将包含所有密钥的 $\mathcal{R}$ 包含在 hash 中）。域分离是 Monero 开发的新策略，未来添加的所有 hash 函数应用 (v13+) 都可能会这样做。历史上的 hash 函数使用可能将保持不变。

²⁰ 在 Monero 中，我们只为主 key image 检查 $l * \text{tilde}K_1$

$\text{stackrel{rel}{?}}=0$ 。辅助密钥将存储为 $(1/8) *$

$\text{tilde}K_j$ ，在验证时乘以 8（回顾第 ?? 节），这更高效。方法差异是一种实现选择，因为可链接的 key image 非常重要，不应被激进地修改，而另一种方法在之前的协议版本中使用过。

3. 对于 $i = 1, \dots, n$ 计算, 将 $n + 1 \rightarrow 1$,

$$c_{i+1} = \mathcal{H}_n(T_c, \mathcal{R}, \mathbf{m}, [r_i G + c_i W_i], [r_i \mathcal{H}_p(K_{i,1}) + c_i \tilde{W}])$$

4. 如果 $c_1 = c'_1$, 则签名有效。

为什么有效 (Why it works)

在这种简洁签名中最大的危险是密钥抵消, 即报告的 key image 不是合法的, 但仍然求和得到合法的聚合值。这就是聚合系数 $\mathcal{H}_n(T_j, \mathcal{R}, \tilde{K}_1, \dots, \tilde{K}_m)$ 发挥作用的地方, 它将每个密钥锁定到其预期值。我们将追踪伪造 key image 的循环影响作为练习留给读者 (也许可以先想象这些系数不存在)。辅助 key image 是证明主 image 合法性的副产品, 因为包含所有私钥的聚合私钥 w_π 被应用于基底点 $\mathcal{H}_p(K_{\pi,1})$ 。

可链接性 (Linkability)

如果私钥 $k_{\pi,1}$ 被重复使用来制作任何签名, 签名中提供的对应主 key image $\tilde{K}_1$ 将揭示这一点。辅助 key image 被忽略, 因为它们的存在只是为了促进 CLSAG 的”简洁”部分。

空间需求 (Space requirements)

我们存储 $(1+n)$ 个整数, 有 m 个 key image, 并使用 $m * n$ 个公钥。

第

4

章 4

门罗币地址 (Monero Addresses)

存储在区块链中的数字货币的所有权由所谓的”地址”控制。地址是用来接收资金的，只有地址的持有者才能花费这些资金。¹

更具体地说，一个地址拥有某些交易的”output”，这些 output 类似于票据，赋予地址持有者花费一定”金额”资金的权利。这样的一张票据可能写着”地址 C 现在拥有 5.3 XMR”。

要花费一个已拥有的 output，地址持有者将它作为新交易的 input 引用。这个新交易的 output 由其他地址拥有（或者如果发送者愿意，也可以由发送者自己的地址拥有）。交易的 input 总金额等于其 output 总金额，而且一旦被花费，已拥有的 output 就不能被再次花费。拥有地址 C 的 Carol 可以在新交易中引用那个旧的 output（例如”在这个交易中，我想花费那个旧的 output”），并添加一张票据说”地址 B 现在拥有 5.3 XMR”。

一个地址的余额是其未花费 output 中所含金额的总和。²

我们将在第 ?? 章讨论如何对观察者隐藏金额，在第 ?? 章讨论交易的结构（包括如何证明你正在花费一个已拥有的、之前未花费的 output，而不暴露具体花费的是哪个 output），在第 ?? 章讨论货币创造过程和观察者的角色。

¹ 概率可忽略不计。

² 被称为”钱包”的计算机应用程序用于查找和组织地址拥有的 output，保管其私钥以授权新交易，并将这些交易提交给网络进行验证和纳入区块链。

4.1 用户密钥 (User keys)

与比特币不同，门罗币用户有两组私钥/公钥， (k^v, K^v) 和 (k^s, K^s) ，生成方式如第 ?? 节所述。

用户的地址就是公钥对 (K^v, K^s) 。她的私钥就是对应的私钥对 (k^v, k^s) 。³

使用两组密钥可以实现功能隔离。其原理将在本章后面变得清晰，但现在让我们称私钥 k^v 为查看密钥 (view key)，称 k^s 为花费密钥 (spend key)。一个人可以使用他们的查看密钥来判断自己的地址是否拥有某个 output，而花费密钥则允许他们在交易中花费该 output（并追溯地确定它已被花费）。⁴

4.2 一次性地址 (One-time addresses)

要接收资金，门罗币用户可以将自己的地址分发给其他用户，然后其他用户就可以通过交易的 output 向该地址发送资金。

地址永远不会被直接使用。⁵取而代之的是，对其应用类似于 Diffie-Hellman 的交换，为将要支付给用户的每个交易 output 创建一个唯一的一次性地址 (one-time address)。这样一来，即使知道所有用户地址的外部观察者也无法用它们来识别哪个用户拥有任何给定的交易 output。⁶

让我们从一个非常简单的模拟交易开始，它只包含一个 output——一笔从 Alice 到 Bob 的“0”金额支付。

Bob 拥有私钥/公钥 (k_B^v, k_B^s) 和 (K_B^v, K_B^s) ，Alice 知道他的公钥（他的地址）。模拟交易可以如下进行 [?]:

1. Alice 生成一个随机数 $r \in_R \mathbb{Z}_l$ ，并计算一次性地址⁷

$$K^o = \mathcal{H}_n(rK_B^v)G + K_B^s$$

2. Alice 将 K^o 设为收款地址，将 output 金额“0”和值 rG 添加到交易数据中，并提交给网络。

³ 为了将地址传达给其他用户，最常见的方式是将其编码为 base-58，这是一种最初为比特币创建的二进制到文本编码方案 [?]。更多详情请参见 [?]

⁴ 它目前最常见的做法是让查看密钥 k^v 等于 $\mathcal{H}_n(k^s)$ 。这意味着一个人只需要保存他们的花费密钥 k^s ，就可以访问（查看和花费）他们拥有的所有 output（已花费和未花费的）。花费密钥通常表示为 25 个单词的序列（其中第 25 个单词是校验和）。其他不太流行的方法包括：将 k^v 和 k^s 作为独立的随机数生成，或者生成一个随机的 12 助记词 a ，其中 $k^s = \text{sc_reduce32}(\text{Keccak}(a))$ ， $k^v = \text{sc_reduce32}(\text{Keccak}(\text{Keccak}(a)))$ 。[?]

⁵ 此处描述的方法不是由协议强制执行的，而是由钱包实现标准强制执行的。这意味着另一个钱包可以遵循比特币的风格，将接收者的地址直接包含在交易数据中。这样一个不合规的钱包将产生其他钱包无法使用的交易 output，而且由于 key image 的唯一性，每个类比特币风格的地址只能使用一次。

⁶ 概率可忽略不计。

⁷ 在门罗币中， rK_B^vG 的每个实例（包括当它用于交易的其他部分时）都乘以辅因子 8，所以在这种情况下 Alice 计算 $8 * rK_B^v$ ，Bob 计算 $8 * k_B^v rG$ 。据我们所知，这并没有什么实际用途（但它确实是用户必须遵守的规则）。乘以辅因子确保结果点在 G 的子群中，但如果 R 和 K^v 本身就不共享子群，那么无论如何，离散对数 r 和 k^v 都不能用来生成共享密钥。

3. Bob 接收到数据, 看到值 rG 和 K^o 。他可以计算 $k_B^v rG = rK_B^v$ 。然后他可以计算 $K_B^s = K^o - \mathcal{H}_n(rK_B^v)G$ 。当他看到 $K_B^s = K_B^s$ 时, 他就知道这个 output 是发给他的。⁸

私钥 k_B^v 被称为“查看密钥”, 因为任何拥有它 (以及 Bob 的公钥花费密钥 K_B^s) 的人都可以为区块链 (交易记录) 中的每个交易 output 计算 K_B^s , 并“查看”哪些属于 Bob。

4. 该 output 的一次性密钥为:

$$\begin{aligned} K^o &= \mathcal{H}_n(rK_B^v)G + K_B^s = (\mathcal{H}_n(rK_B^v) + k_B^s)G \\ k^o &= \mathcal{H}_n(rK_B^v) + k_B^s \end{aligned}$$

要在新交易中花费他的“0”金额 output [原文如此], Bob 所需要做的就是通过使用一次性密钥 K^o 签名消息来证明所有权。私钥 k_B^s 是“花费密钥”, 因为证明 output 所有权需要它; 而 k_B^v 是“查看密钥”, 因为它可以用来找到 Bob 可花费的 output。

正如将在第 ?? 章中变得清楚的那样, 没有 k^o Alice 无法计算该 output 的 key image, 因此她永远无法确定 Bob 是否花费了她发送给他的 output。⁹

Bob 可以将他的查看密钥交给第三方。这样的第三方可以是受信任的托管人、审计员、税务机关等。可以被允许读取用户交易历史而没有其他权限的人。这个第三方也将能够解密 output 的金额 (将在第 ?? 节中解释)。有关 Bob 提供其交易历史信息的其他方式, 请参见第 ?? 章。

4.2.1 多 output 交易 (Multi-output transactions)

大多数交易将包含多个 output。至少, 要将“找零”转回给发送者。^{10,11}

门罗币发送者通常只生成一个随机值 r 。曲线点 rG 通常被称为交易公钥 (transaction public key), 与其他交易数据一起发布在区块链中。

为了确保具有 p 个 output 的交易中所有一次性地址都不同, 即使同一收款地址被使用两次, 门罗币使用了输出索引。交易的每个 output 都有一个索引 $t \in \{0, \dots, p-1\}$ 。通过在 hash 之前将此值附加到共享密钥中, 可以确保生成的一次性地址是唯一的:

$$\begin{aligned} K_t^o &= \mathcal{H}_n(rK_t^v, t)G + K_t^s = (\mathcal{H}_n(rK_t^v, t) + k_t^s)G \\ k_t^o &= \mathcal{H}_n(rK_t^v, t) + k_t^s \end{aligned}$$

⁸ K_B^s 是通过 `derive_subaddress_public_key()` 计算的, 因为普通地址的花费密钥存储在花费密钥查找表的索引 0 处, 而子地址存储在索引 1、2……处。这很快就会有意义: 见第 ?? 节。

⁹ 想象 Alice 产生了两个交易, 每个都包含 Bob 可以花费的同一个一次性 output 地址 K^o 。由于 K^o 仅依赖于 r 和 K_B^v , 她没有理由不能这样做。Bob 只能花费其中一个 output, 因为每个一次性地址只有一个 key image, 所以如果不小心的话 Alice 可能会欺骗他。她可以创建一个给 Bob 大量资金的交易 1, 然后创建一个给 Bob 少量资金的交易 2。如果他花费了交易 2 中的资金, 他就永远无法花费交易 1 中的资金。事实上, 没有人能花费交易 1 中的资金, 实际上就“烧毁”了它。门罗币钱包已被设计为在此场景中忽略较小金额。

¹⁰ 实际上, 从协议 v12 起, 每笔 (非矿工) 交易必须有两个 output, 即使这意味着一个 output 的金额为 0 (`HF_VERSION_MIN_2_OUTPUTS`)。这通过将 1-output 情况与更常见的 2-output 交易混合来提高交易的不可区分性。在 v12 之前, 核心钱包软件已经在创建 0 值的 output。核心实现将 0 金额的 output 发送到随机地址。

¹¹ 在 Bulletproofs 于协议 v8 中实现后, 每笔交易被限制为不超过 16 个 output (`BULLETPROOF_MAX_OUTPUTS`)。此前没有限制, 只是对交易大小 (字节数) 有限制。

4.3 子地址 (Subaddresses)

门罗币用户可以从每个地址生成子地址 [?]¹²。发送到子地址的资金可以使用其主地址的查看密钥和花费密钥来查看和花费。打个比方：一个在线银行账户可能有多个对应于信用卡和存款的余额，但它们都可以从同一个视角——账户持有人——访问和花费。

子地址对于在用户不想通过发布/使用同一地址来关联其活动时，将资金接收到同一地点非常方便。正如我们将看到的，大多数观察者将不得不解决离散对数问题才能确定一个给定的子地址是从任何特定地址派生的 [?]¹²。

它们对于区分接收到的 output 也很有用。例如，如果 Alice 想在周二从 Bob 那里买一个苹果，Bob 可以写一张描述购买的收据并为该收据创建一个子地址，然后让 Alice 在向他发送资金时使用该子地址。这样 Bob 就可以将他收到的钱与他卖出的苹果关联起来。我们将在下一节中探索另一种区分接收到的 output 的方法。

Bob 从他的地址生成第 i 个子地址 ($i = 1, 2, \dots$) 作为公钥对 $(K^{v,i}, K^{s,i})$ ：¹³

$$\begin{aligned} K^{s,i} &= K^s + \mathcal{H}_n(k^v, i)G \\ K^{v,i} &= k^v K^{s,i} \end{aligned}$$

所以，

$$\begin{aligned} K^{v,i} &= k^v (k^s + \mathcal{H}_n(k^v, i))G \\ K^{s,i} &= (k^s + \mathcal{H}_n(k^v, i))G \end{aligned}$$

4.3.1 向子地址发送 (Sending to a subaddress)

假设 Alice 再次向 Bob 发送“0”金额，这次是通过他的子地址 $(K_B^{v,1}, K_B^{s,1})$ 。

1. Alice 生成一个随机数 $r \in_R \mathbb{Z}_l$ ，并计算一次性地址

$$K^o = \mathcal{H}_n(rK_B^{v,1}, 0)G + K_B^{s,1}$$

2. Alice 将 K^o 设为收款地址，将 output 金额“0”和值 $rK_B^{s,1}$ 添加到交易数据中，并提交给网络。
3. Bob 接收到数据，看到值 $rK_B^{s,1}$ 和 K^o 。他可以计算 $k_B^v rK_B^{s,1} = rK_B^{v,1}$ 。然后他可以计算 $K_B'^s = K^o - \mathcal{H}_n(rK_B^{v,1}, 0)G$ 。当他看到 $K_B'^s = K_B^{s,1}$ 时，他就知道该交易是发给他的。¹⁴

Bob 只需要他的私有查看密钥 k_B^v 和子地址的公钥花费密钥 $K_B^{s,1}$ 就可以找到发送到他子地址的交易 output。

¹² 在子地址之前（在与协议 v7 对应的软件更新中添加 [?]¹²），门罗币用户可以简单地生成许多普通地址。要查看每个地址的余额，你需要对区块链记录进行单独扫描。这非常低效。有了子地址，用户维护一个（已 hash 的）花费密钥查找表，因此对 1 个子地址或 10,000 个子地址，扫描区块链所需的时间是相同的。

¹³ 实际上子地址的 hash 是域分离的，所以实际上是 $\mathcal{H}_n(T_{sub}, k^v, i)$ ，其中 $T_{sub} = \text{“SubAddr”}$ 。为了简洁起见，我们在本文档中省略 T_{sub} 。

¹⁴ 高级攻击者可能能够关联子地址 [?]（即 Janus 攻击）。攻击者认为可能相关的子地址 K_B^1, K_B^2 （其中一个可以是普通地址），他创建一个交易 output，其中 $K^o = \mathcal{H}_n(rK_B^{v,2}, 0)G + K_B^{s,1}$ ，并包含交易公钥 $rK_B^{s,2}$ 。Bob 计算 $rK_B^{v,2}$ 来找到 $K_B^{s,1}$ ，但无法知道使用的是他的另一个（子）地址的密钥！如果他告诉攻击者他收到了发往 K_B^1 的资金，攻击者

4. 该 output 的一次性密钥为:

$$\begin{aligned} K^o &= \mathcal{H}_n(rK_B^{v,1}, 0)G + K_B^{s,1} = (\mathcal{H}_n(rK_B^{v,1}, 0) + k_B^{s,1})G \\ k^o &= \mathcal{H}_n(rK_B^{v,1}, 0) + k_B^{s,1} \end{aligned}$$

现在, Alice 的交易公钥是 Bob 特有的 ($rK_B^{s,1}$ 而不是 rG)。如果她创建一个包含 p 个 output 的交易, 且至少有一个 output 是发往子地址的, Alice 需要为每个 output $t \in \{0, \dots, p-1\}$ 创建一个唯一的交易公钥。换句话说, 如果 Alice 向 Bob 的子地址 ($K_B^{v,1}, K_B^{s,1}$) 和 Carol 的地址 (K_C^v, K_C^s) 发送, 她将在交易数据中放置两个交易公钥 $\{r_1 K_B^{s,1}, r_2 G\}$ 。¹⁵

就会知道 K_B^2 是一个相关的子地址 (或普通地址)。由于子地址不在协议范围内, 缓解措施取决于钱包实现者。目前没有已知的钱包这样做过, 而且任何缓解措施只对合规钱包有效。潜在的缓解措施包括: 不向攻击者通报收到的资金、在交易数据中包含加密的交易私钥 r 、使用 $K^{s,1}$ 而不是 G 作为基点对共享密钥进行 Schnorr 签名、在交易数据中包含 rG 并使用 $rK^{s,1} \stackrel{?}{=} (k^s + \mathcal{H}_n(k^v, 1)) * rG$ 验证共享密钥 (需要私有花费密钥)。普通地址收到的 output 也应该被验证。有关最后一种缓解措施的讨论, 请参见 [?]

¹⁵ 在门罗币中, 子地址以“8”为前缀, 与以“4”为前缀的地址区分开来。这有助于发送者在构建交易时选择正确的过程。

门罗币金额隐藏 (Monero Amount Hiding)

在大多数像比特币这样的加密货币中，交易 output 票据——赋予花费一定“金额”资金的权利——以明文传达这些金额。这使得观察者可以轻松验证花费的金额等于发送的金额。

在门罗币中，我们使用承诺 (commitment) 来对除发送者和接收者以外的所有人隐藏 output 金额，同时仍然让观察者确信交易发送的金额不多也不少。正如我们将看到的，金额承诺还必须有相应的“范围证明 (range proof)”来证明隐藏的金额在合法范围内。

5.1 承诺 (Commitments)

一般来说，密码学中的承诺方案 (commitment scheme) 是一种在不揭示值本身的情况下对一个值做出承诺的方式。一旦对某事做出承诺，你就必须坚守它。

例如，在一个抛硬币游戏中，Alice 可以通过将她承诺的值与秘密数据进行 hash 并发布 hash 值来私下承诺一个结果（即“猜正反”）。在 Bob 抛过硬币之后，Alice 宣布她承诺了哪个值，并通过揭示秘密数据来证明。然后 Bob 可以验证她的声明。

换句话说，假设 Alice 有一个秘密字符串 *blah*，她想要承诺的值是 *heads*。她计算 $h = \mathcal{H}(\text{blah}, \text{heads})$ 并将 h 给 Bob。Bob 抛硬币，然后 Alice 告诉 Bob 秘密字符串 *blah* 并且她承诺了 *heads*。Bob 计算 $h' = \mathcal{H}(\text{blah}, \text{heads})$ 。如果 $h' = h$ ，那么他就知道 Alice 在抛硬币之前就猜了 *heads*。

Alice 使用所谓的”盐 (salt)” *blah*，这样 Bob 就不能在抛硬币之前直接猜 $\mathcal{H}(\text{heads})$ 和 $\mathcal{H}(\text{tails})$ ，然后推断出她承诺了 *heads*。¹

5.2 Pedersen commitment

Pedersen commitment [?] 是一种具有加法同态 (additively homomorphic) 性质的承诺。如果 $C(a)$ 和 $C(b)$ 分别表示值 a 和 b 的承诺，那么 $C(a + b) = C(a) + C(b)$ 。²这个性质在承诺交易金额时非常有用，因为人们可以证明，例如，input 等于 output，而无需揭示手头的金额。

幸运的是，Pedersen commitment 很容易用椭圆曲线密码学实现，因为以下等式显然成立：

$$aG + bG = (a + b)G$$

显然，如果将承诺简单地定义为 $C(a) = aG$ ，我们可以很容易地创建承诺的作弊表来帮助我们识别 a 的常见值。

要获得信息论隐私，需要添加一个秘密盲因子 (blinding factor) 和另一个生成元 H ，使得对于哪个 γ 值以下等式成立是未知的： $H = \gamma G$ 。离散对数问题的困难性确保了从 H 计算 γ 是不可行的。³

然后我们可以将对值 a 的承诺定义为 $C(x, a) = xG + aH$ ，其中 x 是盲因子 (又名”掩码 (mask)”)，用于防止观察者猜测 a 。

承诺 $C(x, a)$ 在信息论上是私密的，因为有许多可能的 x 和 a 组合会输出相同的 C 。⁴如果 x 是真正随机的，攻击者将完全没有方法推断出 a [?, ?]。

5.3 金额承诺 (Amount commitments)

在门罗币中，output 金额以 Pedersen commitment 的形式存储在交易中。我们将对 output 金额 b 的承诺定义为：

$$C(y, b) = yG + bH$$

接收者应该能够知道他们拥有的每个 output 中有多少钱，以及重建金额承诺，以便它们可以作为新交易的 input。这意味着盲因子 y 和金额 b 必须传达给接收者。

采用的解决方案是使用 Diffie-Hellman 共享密钥 rK_B^v (回顾第 ?? 节中的”交易公钥”)。对于区块链中的任何给定交易，其每个 output $t \in \{0, \dots, p - 1\}$ 都有一个掩码 y_t (发送者和接收者可以私

¹ 如果承诺的值很难猜测和验证，例如如果它是一个看似随机的椭圆曲线点，那么给承诺加盐就不是必要的。

² 在这个上下文中，加法同态意味着当你将标量通过标量 x 的映射 $x \rightarrow xG$ 变换为 EC 点时，加法得以保持。

³ 在门罗币的情况下， $H = 8 * \text{to_point}(\mathcal{H}_n(G))$ 。这与 $\mathcal{H}_p$ hash 函数的不同之处在于，它直接将 $\mathcal{H}_n(G)$ 的输出解释为压缩点坐标，而不是通过数学方法派生曲线点 (参见 [?])。这种差异的历史原因我们不得而知，实际上这是唯一不使用 $\mathcal{H}_p$ 的地方 (Bulletproofs 也使用 $\mathcal{H}_p$)。注意有一个 *8 操作，以确保结果点在我们的 l 子群中 ($\mathcal{H}_p$ 也这样做)。

⁴ 基本上，有许多 x' 和 a' 使得 $x' + a'\gamma = x + a\gamma$ 。承诺者知道一种组合，但攻击者无法知道是哪一种。这种性质也被称为”完美隐藏 (perfect hiding)” [?]。此外，即使是承诺者也不能在不解决 γ 的离散对数问题的情况下找到另一种组合，这种性质称为”计算绑定 (computational binding)” [?]

下计算) 和一个存储在交易数据中的金额。虽然 y_t 是椭圆曲线标量, 占 32 字节, 但 b 将被范围证明限制为 8 字节, 因此只需要存储一个 8 字节的值。^{5,6}

$$y_t = \mathcal{H}_n(\text{"commitment_mask"}, \mathcal{H}_n(rK_B^v, t))$$

$$\text{amount}_t = b_t \oplus_8 \mathcal{H}_n(\text{"amount"}, \mathcal{H}_n(rK_B^v, t))$$

这里, $\oplus_8$ 表示对每个操作数的前 8 字节执行 XOR 操作 (第 ?? 节) (b_t 已经是 8 字节, $\mathcal{H}_n(\dots)$ 是 32 字节)。接收者可以对 amount_t 执行相同的 XOR 操作来揭示 b_t 。

接收者 Bob 将能够使用交易公钥 rG 和他的查看密钥 k_B^v 计算盲因子 y_t 和金额 b_t 。他还可以检查交易数据中提供的承诺 $C(y_t, b_t)$ (此后记为 C_t^b) 是否与手头的金额一致。

更一般地说, 任何可以访问 Bob 查看密钥的第三方都可以解密他的 output 金额, 还可以确保它们与关联的承诺一致。

5.4 RingCT 简介 (RingCT introduction)

一笔交易将包含对其他交易 output 的引用 (告诉观察者哪些旧的 output 将被花费), 以及它自己的 output。output 的内容包括一个一次性地址 (分配 output 的所有权) 和一个隐藏金额的 output 承诺 (以及第 ?? 节中的编码 output 金额)。

虽然交易的验证者不知道每个 input 和 output 中包含多少钱, 但他们仍然需要确保 input 金额之和等于 output 金额之和。门罗币使用一种叫做 RingCT [?] 的技术来实现这一点, 该技术于 2017 年 1 月首次实现 (协议 v4)。

如果我们有一笔包含金额 $a_1, \dots, a_m$ 的 m 个 input 的交易, 以及金额 $b_0, \dots, b_{p-1}$ 的 p 个 output, 那么观察者有理由期望:⁷

$$\sum_j a_j - \sum_t b_t = 0$$

由于承诺是加法的, 我们不知道 γ , 我们可以轻松地通过使 input 和 output 金额的承诺之和为零 (即通过设置 output 盲因子之和等于旧 input 盲因子之和) 来向观察者证明我们的 input 等于 output:⁸

$$\sum_j C_{j,in} - \sum_t C_{t,out} = 0$$

⁵ 与第 ?? 节中的一次性地址 K^o 一样, output 索引 t 在 hash 前被附加到共享密钥中。这确保了发往同一地址的 output 不会有相似的掩码和金额, 概率可忽略不计。也和之前一样, 项 rK_B^v 乘以 8, 所以实际上是 $8rK_B^v$ 。

⁶ 这个解决方案 (在协议 v10 中实现) 替代了之前使用更多数据的方法, 从而导致交易类型从 v3 (RCTTypeBulletproof) 变为 v4 (RCTTypeBulletproof2)。本报告的第一版讨论了之前的方法 [?]

⁷ 如果预期的 output 总金额不精确等于所拥有的 output 的任何组合, 那么交易作者可以添加一个“找零” output 将多余的钱发回给自己。打个比方, 用一张 20 美元的钞票支付 15 美元的开销, 你会从收银员那里收到 5 美元。

⁸ 回忆第 ?? 节, 我们可以通过反转坐标然后相加来减去一个点。如果 $P = (x, y)$, $-P = (-x, y)$ 。还请回忆, 域元素的取反是计算

$p \bmod q$, 所以 $(-x$

$p \bmod q)$ 。

为了避免发送者可识别性，我们使用一种略有不同的方法。被花费的金额对应于之前交易的 output，这些 output 有承诺

$$C_j^a = x_j G + a_j H$$

发送者可以创建对相同金额但使用不同盲因子的新承诺；即，

$$C_j'^a = x_j' G + a_j H$$

显然，她会知道两个承诺之差的私钥：

$$C_j^a - C_j'^a = (x_j - x_j') G$$

因此，她将能够使用这个值作为零承诺 (commitment to zero)，因为私钥 $(x_j - x_j') = z_j$ 可以用来做签名，从而证明和中没有 H 分量（假设 γ 是未知的）。换句话说，证明 $C_j^a - C_j'^a = z_j G + 0H$ ，我们在讨论 RingCT 交易结构时将在第 ?? 章中实际执行这一操作。

让我们称 $C_j'^a$ 为伪 output 承诺 (pseudo output commitment)。伪 output 承诺包含在交易数据中，每个 input 一个。

在将交易提交到区块链之前，网络将想要验证其金额是平衡的。伪承诺和 output 承诺的盲因子被选择为使得

$$\sum_j x_j' - \sum_t y_t = 0$$

这允许我们证明 input 金额等于 output 金额：

$$(\sum_j C_j'^a - \sum_t C_t^b) = 0$$

幸运的是，选择这样的盲因子很容易。在当前版本的门罗币中，除了第 m 个伪 output 承诺外，所有盲因子都是随机的，其中 x_m' 简单地是

$$x_m' = \sum_t y_t - \sum_{j=1}^{m-1} x_j'$$

5.5 范围证明 (Range proofs)

加法承诺的一个问题是，如果我们有承诺 $C(a_1)$ 、 $C(a_2)$ 、 $C(b_1)$ 和 $C(b_2)$ ，并且我们打算用它们来证明 $(a_1 + a_2) - (b_1 + b_2) = 0$ ，那么如果等式中有一个值是“负的”，这些承诺仍然成立。

例如，我们可以有 $a_1 = 6$ 、 $a_2 = 5$ 、 $b_1 = 21$ 和 $b_2 = -10$ 。

$$(6 + 5) - (21 + -10) = 0$$

where

$$21G + -10G = 21G + (l - 10)G = (l + 11)G = 11G$$

由于 $-10 = l - 10$ ，我们实际上创造了比我们投入的多 l 个门罗币（超过 7.2×10^{74} 个！）。

解决门罗币中这个问题的方案是使用 Benedikt Bünz 等人在 [?] 中首次描述的 Bulletproofs 证明方案（也参见 [?, ?] 中的解释）来证明每个 output 金额在某个范围内（从 0 到 $2^{64} - 1$ ）。⁹鉴于 Bulletproofs 的复杂和精密性质，本文档不对其进行阐述。此外，我们认为引用的材料充分地呈现了其概念。¹⁰

Bulletproof 证明算法以 output 金额 b_t 和承诺掩码 y_t 作为输入，输出所有 C_t^b 和一个 n 元组聚合证明 $\Pi_{BP} = (A, S, T_1, T_2, \tau_x, \mu, \mathbb{L}, \mathbb{R}, a, b, t)$ ^{11,12}。该单一证明用于同时证明所有 output 金额在范围内，聚合它们大大减少了空间需求（尽管确实增加了验证时间）。¹³验证算法以所有 C_t^b 和 Π_{BP} 作为输入，如果所有承诺的金额都在 0 到 $2^{64} - 1$ 的范围内则输出 `true`。

n 元组 Π_{BP} 占用 $(2 \cdot \lceil \log_2(64 \cdot p) \rceil + 9) \cdot 32$ 字节的存储空间。

⁹ 可以想象，如果几个 output 都在合法范围内，它们金额之和可能会溢出并导致类似的问题。然而，当最大 output 远小于 l 时，需要大量的 output 才会发生这种情况。例如，如果范围是 0-5, $l = 99$ ，那么要使用一个输入 2 来伪造货币，我们需要 $5 + 5 + \dots + 5 + 1 = 101 \equiv 2 \pmod{99}$ ，总共需要 21 个 output。在门罗币中 l 大约比可用范围大 2^{189} 倍，这意味着需要荒谬的 2^{189} 个 output 才能伪造货币。

¹⁰ 在协议 v8 之前，范围证明是通过 Borromean 环签名完成的，在 Zero to Monero 的第一版中有所解释 [?]

¹¹ 向量 $\mathbb{L}$ 和 $\mathbb{R}$ 各包含 $\lceil \log_2(64 \cdot p) \rceil$ 个元素。 $\lceil \cdot \rceil$ 表示 $\log$ 函数向上取整。由于它们的构造方式，某些 Bulletproofs 使用“虚拟 output”作为填充，以确保 p 加上虚拟 output 的数量是 2 的幂。这些虚拟 output 可以在验证期间生成，不与证明数据一起存储。

¹² Bulletproof 中的变量与本文档中的其他变量无关。符号重叠纯属巧合。注意群元素 $A, S, T_1, T_2, \mathbb{L}$ 和 $\mathbb{R}$ 在存储前乘以 $1/8$ ，在验证期间乘以 8。这确保它们都是 l 子群的成员（回顾第 ?? 节）。

¹³ 实际上，多个单独的 Bulletproofs 可以被一起“批处理”，这意味着它们被同时验证。这样做改善了验证它们所需的时间，目前在门罗币中 Bulletproofs 是按区块批处理的，尽管理论上批处理多少没有限制。每笔交易只允许有一个 Bulletproof。

门罗币环机密交易 (RingCT) (Monero Ring Confidential Transactions (RingCT))

在第 ?? 章和第 ?? 章中，我们逐步建立了门罗币交易的多个方面。到目前为止，一个简单的单 input、单 output 的交易——从某个未知作者到某个未知接收者——听起来像这样：

”我的交易使用交易公钥 rG 。我将花费旧的 output X （注意它有一个隐藏的金额 A_X ，在 C_X 中做出承诺）。我将给它一个伪 output 承诺 C'_X 。我将创建一个 output Y ，可由一次性地址 K_Y^o 花费。它有一个隐藏的金额 A_Y 在 C_Y 中做出承诺，为接收者加密，并通过 Bulletproofs 风格的范围证明证明在范围内。请注意 $C'_X - C_Y = 0$ 。”

还有一些问题。作者是否真的拥有 X ？伪 output 承诺 C'_X 是否真的对应于 C_X ，使得 $A_X = A'_X = A_Y$ ？是否有人篡改了交易，也许将 output 指向了原始作者未打算的接收者？

如第 ?? 节所述，我们可以通过使用一次性地址签名消息来证明对 output 的所有权（谁拥有该地址的密钥就拥有该 output）。我们还可以通过证明对零承诺的私钥的知识来证明它与伪 output 承诺有相同的金额（ $C_X - C'_X = z_X G$ ）。此外，如果该消息是所有交易数据（签名本身除外），那么验证者可以确信一切都是按作者的意图（签名只对原始消息有效）。MLSAG 签名允许我们做到所有这些，同时还将实际花费的 output 混入区块链中的其他 output 中，这样观察者就无法确定正在花费的是哪一个。

6.1 交易类型 (Transaction types)

门罗币是一种持续发展中的加密货币。交易结构、协议和密码方案总是会随着新的创新、目标或威胁的发现而演变。

在本报告中，我们将注意力集中在环机密交易 (Ring Confidential Transactions)，即 RingCT，因为它们是在当前版本的门罗币中实现的。RingCT 对所有新的门罗币交易都是强制性的，因此我们不会描述任何已弃用的交易方案，即使它们仍然被部分支持。¹我们到目前为止讨论的交易类型，以及将在本章中进一步详细说明的，是 RCTTypeBulletproof2。²

我们在第 ?? 节中给出交易的概念性总结。

6.2 类型为 RCTTypeBulletproof2 的环机密交易 (Ring Confidential Transactions of type RCTTypeBulletproof2)

目前（协议 v12）所有新交易都必须使用这种交易类型，其中每个 input 是单独签名的。一个 RCTTypeBulletproof2 交易的实际示例，包含所有组件，可以在附录 ?? 中查看。

6.2.1 金额承诺与交易手续费 (Amount commitments and transaction fees)

假设一个交易发送者之前收到了各种 output，金额为 $a_1, \dots, a_m$ ，地址为一次性地址 $K_{\pi,1}^o, \dots, K_{\pi,m}^o$ ，金额承诺为 $C_{\pi,1}^a, \dots, C_{\pi,m}^a$ 。

该发送者知道对应于一次性地址的私钥 $k_{\pi,1}^o, \dots, k_{\pi,m}^o$ （第 ?? 节）。发送者还知道承诺 $C_{\pi,j}^a$ 中使用的盲因子 x_j （第 ?? 节）。

通常，交易 output 的总额低于交易 input，以提供手续费来激励矿工将交易纳入区块链。³交易手续费金额 f 以明文存储在发送给网络的交易数据中。矿工可以为自己创建一个额外的 output 来收取手续费（见第 ?? 节）。

一笔交易由 input $a_1, \dots, a_m$ 和 output $b_0, \dots, b_{p-1}$ 组成，使得 $\sum_{j=1}^m a_j - \sum_{t=0}^{p-1} b_t - f = 0$ 。⁴

发送者为 input 金额计算伪 output 承诺 $C_{\pi,1}^a, \dots, C_{\pi,m}^a$ ，并为预期的 output 金额 $b_0, \dots, b_{p-1}$ 创建承诺。设这些新承诺为 $C_0^b, \dots, C_{p-1}^b$ 。

¹ RingCT 于 2017 年 1 月首次实现（协议 v4）。它在 2017 年 9 月（协议 v6）被强制要求用于所有新交易 [?]。RingCT 是门罗币交易协议的第 2 版。

² 在 RingCT 时代有三种已弃用的交易类型：RCTTypeFull、RCTTypeSimple 和 RCTTypeBulletproof。前两种在 RingCT 的第一次迭代中并存，本报告的第一版 [?] 中有探讨，然后随着 Bulletproofs 的出现（协议 v8）RCTTypeFull 被弃用，RCTTypeSimple 被升级为 RCTTypeBulletproof。RCTTypeBulletproof2 由于对 output 承诺的掩码和金额进行加密的小改进（v10）而到来。

³ 在门罗币中，有一个最低基础手续费，按交易权重缩放。它是半强制性的，因为虽然你可以创建包含微小手续费交易的新区块，但大多数门罗币节点不会将此类交易转发给其他节点。结果是很少有矿工会尝试将它们纳入区块。如果交易作者愿意，可以提供高于最低值的矿工手续费。我们在第 ?? 节中对此进行了更详细的讨论。

⁴ Output 在被分配索引之前由核心实现随机打乱，因此观察者无法围绕其顺序建立启发式规则。Input 在交易数据中按 key image 排序。

他知道零承诺 $(C_{\pi,1}^a - C_{\pi,1}^b), \dots, (C_{\pi,m}^a - C_{\pi,m}^b)$ 的私钥 $z_1, \dots, z_m$ 。

为了让验证者确认交易金额之和为零，手续费金额必须转换为承诺。解决方案是在没有任何盲因子的掩蔽效果下计算手续费 f 的承诺。即 $C(f) = fH$ 。

现在我们可以证明 input 金额等于 output 金额：

$$(\sum_j C_j^a - \sum_t C_t^b) - fH = 0$$

6.2.2 签名 (Signature)

发送者从区块链中选择 m 组、每组 v 个额外的不相关一次性地址及其承诺，对应于看似未花费的 output。^{5,6}要签名 input j ，她将一个 v 大小的集合搅入一个环 (ring) 中，同时加入她自己第 j 个未花费的一次性地址（放在唯一索引 π 处），以及虚假的零承诺，如下：

⁵ 在门罗币中，“额外不相关地址”的集合通常从 Gamma 分布中选择，覆盖历史 output 的范围（仅限 RingCT output，对于 pre-RingCT output 使用三角分布）。此方法使用分箱程序来平滑区块密度的差异。首先计算 RingCT output 过去一年内的平均每区块 output 数量（平均时间 = $[\text{\#outputs}/\text{\#blocks}] \times \text{blocktime}$ ）。通过 Gamma 分布选择一个 output，然后在其区块内查找并抓取一个随机 output 作为集合成员。[?]

⁶ 从协议 v12 起，所有交易 input 必须至少有 10 个区块的确认时间（CRYPTONOTE_DEFAULT_TX_SPENDABLE_AGE）。在 v12 之前，核心实现默认使用 10 个区块，但不是强制要求，因此替代钱包可以做出不同选择，有些确实这样做了 [?]

第

7

章 7

门罗币区块链 (The Monero Blockchain)

互联网时代为人类体验带来了新的维度。我们可以与地球上每一个角落的人通信，难以想象的信息财富就在我们指尖。交换商品和服务是和平繁荣社会的基础 [?], 在数字领域我们可以向全世界提供我们的生产力。

交换媒介（货币）是必不可少的，它为我们提供了衡量极其多样的经济商品的参照点，否则这些商品将不可能被评估，并使拥有共同点的人之间实现互利互动 [?]. 纵观历史，有许多种货币，从贝壳到纸张到黄金。那些是当面交换的，现在货币可以电子交换了。

在当前最为普遍的模式中，电子交易由第三方金融机构处理。这些机构被赋予货币托管权，并被信任在被要求时进行转移。此类机构必须调解争议，其支付是可逆的，并且它们可以被强大的组织审查或控制。[?]

为了缓解这些缺点，人们设计了去中心化数字货币。¹

7.1 数字货币 (Digital currency)

设计一种数字货币并非易事。有三种类型：个人型、中心化型或分布式型。请记住，数字货币只是一组消息的集合，这些消息中记录的”金额”被解释为货币数量。

¹ 本章比前面的章节包含更多实现细节，因为区块链的性质在很大程度上取决于其具体结构。

在**电子邮件模型**中，任何人都可以制造硬币（例如一条消息说“我拥有 5 个硬币”），任何人都可以将他们的硬币一遍又一遍地发送给任何有电子邮件地址的人。它没有有限的供应量，也不防止重复花费同一硬币（双花）。

在**电子游戏模型**中，整个货币存储/记录在一个中央数据库上，用户依赖于托管人是诚实的。货币的供应对观察者来说是不可验证的，托管人可以随时更改规则，或者被强大的外部力量审查。

7.1.1 分布式/共享版本的事件 (Distributed/shared version of events)

在数字“共享”货币中，许多计算机各自拥有每笔货币交易的记录。当一台计算机上进行新交易时，它会被广播给其他计算机，如果符合预定义的规则则被接受。

只有当其他用户接受硬币进行交换时，用户才能从硬币中获益，而用户只接受他们认为合法的硬币。为了最大化硬币的效用，用户自然倾向于在一个没有中央权威的情况下，达成一个被共同接受的规则集。²

规则 1：货币只能在明确定义的场景中创造。

规则 2：交易花费的是已经存在的货币。

规则 3：一个人只能花费一笔钱一次。

规则 4：只有拥有一笔钱的人才能花费它。

规则 5：交易输出的金额等于花费的金额。

规则 6：交易格式正确。

规则 2-6 由第 ?? 章讨论的交易方案涵盖，该方案增加了可替代性和隐私相关的好处：模糊签名、匿名接收资金和不可读的金额转移。我们将在本章后面解释规则 1。³交易使用密码学，因此我们称其内容为加密货币 (cryptocurrency)。

如果两台计算机在有机会互相发送信息之前收到了花费同一笔钱的不同合法交易，它们如何决定哪个是正确的？货币中出现了“分叉 (fork)”，因为存在遵循相同规则的两个不同副本。

显然，花费一笔钱的最早合法交易应该是规范的。说起来容易做起来难。正如我们将看到的，获得交易历史的共识构成了区块链技术存在的理由。

7.1.2 简单区块链 (Simple blockchain)

首先我们需要所有计算机（此后称为节点 (nodes)）就交易顺序达成一致。

假设一种货币以一个“创世 (genesis)”声明开始：“让 SampleCoin 开始吧！”。我们称这条消息为一个“区块 (block)”，它的区块 hash 为

² 在政治学中，这被称为谢林点 [?]、社会最小值或社会契约。

³ 在像黄金这样的商品货币中，这些规则由物理现实来满足。

$$BH_G = \mathcal{H}(\text{"Let the SampleCoin begin!"})$$

每当一个节点收到一些交易，它使用这些交易的 hash TH 作为消息，连同前一个区块的 hash，计算新的区块 hash

$$\begin{aligned} BH_1 &= \mathcal{H}(BH_G, TH_1, TH_2, \dots) \\ BH_2 &= \mathcal{H}(BH_1, TH_3, TH_4, \dots) \end{aligned}$$

依此类推，每产生一个新的消息区块就发布它。每个新区块引用前一个最近发布的区块。这样，一个清晰的事件顺序一直延伸/链接到创世消息。我们就有了一个非常简单的”区块链”。⁴

节点可以在其区块中包含时间戳以辅助记录。如果大多数节点的时间戳是诚实的，那么区块链提供了每笔交易何时被记录的相当清晰的图景。

如果引用同一前一个区块的不同区块同时发布，那么节点网络将分叉，因为每个节点在收到另一个新区块之前先收到其中一个（为简单起见，想象大约一半节点最终拥有分叉的每一侧）。

7.2 难度 (Difficulty)

如果节点可以随时发布新区块，网络可能会分裂并分化为许多不同的、同样合法的链。假设需要 30 秒来确保网络中的每个人都能收到一个新区块。如果每隔 31 秒、15 秒、10 秒等发送新区块呢？

我们可以控制整个网络产生新区块的速度。如果产生一个新区块的时间远高于前一个区块到达大多数节点的时间，网络将趋于保持完整。

7.2.1 挖掘区块 (Mining a block)

密码学 hash 函数的输出是均匀分布的，且显然独立于输入。这意味着，给定一个潜在输入，其 hash 等可能地是每一个可能的输出。此外，计算一个 hash 需要一定的时间。

让我们想象一个 hash 函数 $\mathcal{H}_i(x)$ ，它输出 1 到 100 之间的数字： $\mathcal{H}_i(x) \in_R^D \{1, \dots, 100\}$ 。⁵ 给定某个 x ， $\mathcal{H}_i(x)$ 每次计算时都选择相同的”随机”数字，从 $\{1, \dots, 100\}$ 中选取。计算 $\mathcal{H}_i(x)$ 需要 1 分钟。

假设我们被给定一条消息 m ，并要求找到一个”nonce” n （某个整数），使得 $\mathcal{H}_i(m, n)$ 输出一个小于或等于目标 (target) $t = 5$ 的数字（即 $\mathcal{H}_i(m, n) \in \{1, \dots, 5\}$ ）。

由于只有 $\mathcal{H}_i(x)$ 输出的 $1/20$ 会满足目标，大约需要 20 次对 n 的猜测才能找到一个有效的（因此需要 20 分钟的计算时间）。

搜索有用的 nonce 被称为挖矿 (mining)，发布带 nonce 的消息是一种工作量证明 (proof of work)，因为它证明我们找到了一个有用的 nonce（即使我们很幸运只用一次 hash 就找到了，或者甚至盲目地发布了一个好的 nonce），任何人都可以通过计算 $\mathcal{H}_i(m, n)$ 来验证。

⁴ 区块链在技术上是”有向无环图” (DAG)，比特币风格的区块链是一维变体。DAG 包含有限数量的节点和连接节点的单向边（向量）。如果你从一个节点开始，无论走什么路径都不会循环回原节点。[?]

⁵ 我们用 $\in_R^D$ 表示输出是确定性随机的。

现在假设我们有一个用于生成工作量证明的 hash 函数 $\mathcal{H}_{PoW} \in_R^D \{0, \dots, m\}$, 其中 m 是其最大可能输出。给定一条消息 m (一个信息区块), 一个待挖的 nonce n , 以及一个目标 t , 我们可以定义预期的平均 hash 次数——难度 (difficulty) d ——如下: $d = m/t$ 。如果 $\mathcal{H}_{PoW}(m, n) * d \leq m$, 则 $\mathcal{H}_{PoW}(m, n) \leq t$ 且 n 是可接受的。⁶

目标越小, 难度越大, 计算机需要越来越多的 hash, 因此需要越来越长的时间来找到有用的 nonce。

7

7.2.2 挖矿速度 (Mining speed)

假设所有节点同时挖矿, 但当它们从网络收到新区块时就放弃”当前”区块。它们立即开始挖掘引用新区块的新区块。

假设我们从区块链中收集一组 b 个最近的区块 (比如说, 索引为 $u \in \{1, \dots, b\}$), 每个区块都有难度 d_u 。现在假设挖掘它们的节点是诚实的, 所以每个区块时间戳 TS_u 是准确的。⁸最早区块和最近区块之间的总时间为 $totalTime = TS_b - TS_1$ 。挖掘所有区块大约需要的 hash 次数为 $totalDifficulty = \sum_u d_u$ 。

现在我们可以猜测网络及其所有节点计算 hash 的速度有多快。如果在区块组产生期间实际速度没有变化多少, 它应该是有效的⁹

$$hashSpeed \approx totalDifficulty / totalTime$$

如果我们想设置挖掘新区块的目标时间, 使区块以

(一个区块)/(目标时间) 的速率产生, 那么我们计算网络应该花多少 hash 来完成那个时间的挖掘。注意: 我们向上取整, 使难度永远不会为零。

$$newDifficulty = hashSpeed * targetTime$$

不能保证下一个区块需要 $newDifficulty$ 数量的总网络 hash 来挖掘, 但随着时间推移, 经过许多区块和不断的重新校准, 难度将与网络的实际 hash 速度保持一致, 区块将趋于花费 $targetTime$ 。¹⁰

7.2.3 共识: 最大累积难度 (Consensus: largest cumulative difficulty)

现在我们可以解决链分叉之间的冲突。

⁶ 在门罗币中只记录/计算难度, 因为 $\mathcal{H}_{PoW}(m, n) * d \leq m$ 不需要 t 。

⁷ 挖矿和验证是不对称的, 因为无论难度如何, 验证工作量证明 (工作量证明算法的一次计算) 都需要相同的时间。

⁸ 时间戳在矿工开始挖矿时确定, 因此它们可能滞后于实际发布时刻。下一个区块立即开始挖矿, 因此出现在给定区块之后的时间戳表明矿工花了多长时间在上面。

⁹ 如果节点 1 尝试 nonce $n = 23$, 后来节点 2 也尝试 $n = 23$, 节点 2 的努力是浪费的, 因为网络已经”知道” $n = 23$ 不行 (否则节点 1 就会发布那个区块了)。网络的有效 hash 速率取决于它为给定消息区块 hash 唯一 nonce 的速度。正如我们将看到的, 由于矿工在其区块中包含一个具有一次性地址 $K^o \in_{ER} \mathbb{Z}_l$ ($ER =$ 有效随机) 的矿工交易, 除了可忽略不计的概率外, 矿工之间的区块总是唯一的, 所以尝试相同的 nonce 无关紧要。

¹⁰ 如果我们假设网络 hash 速率在持续、逐渐地增加, 那么由于新难度依赖于过去的 hash (即在 hash 速率微增之前), 我们应该预期实际区块时间平均略低于 $targetTime$ 。这对排放计划 (第 ?? 节) 的影响可能被增加区块权重的惩罚所抵消, 我们在第 ?? 节中探讨。

按照惯例，具有最高累积难度（来自链中所有区块）的链，因此也是花费最多工作量（网络 hash）构建的链，被认为是真实的、合法的版本。如果链分裂且每个分叉具有相同的累积难度，节点继续在它们先收到的分支上挖掘。当一个分支领先于另一个时，它们放弃（“孤立”）较弱的分支。

如果节点希望更改或升级基本协议——即节点在决定区块链副本或新区块是否合法时考虑的规则集——它们可以轻松地通过分叉链来做到这一点。新分支是否对用户有影响取决于有多少节点切换以及多少软件基础设施被修改。¹¹

对于攻击者来说，要让诚实节点改变交易历史——也许是为了重新花费/取消花费资金——他必须创建一个在当前协议上具有比主链（与此同时继续增长）更高总难度的链分叉。这非常困难，除非你控制超过 50% 的网络 hash 速度并能超越其他矿工。[?]

7.2.4 门罗币中的挖矿 (Mining in Monero)

为了确保链分叉在公平的基础上竞争，我们不采样最近的区块（用于计算新难度），而是将我们的区块组 b 滞后 l 。例如，如果链中有 29 个区块（区块 1, ..., 29）， $b = 10$ ， $l = 5$ ，我们采样区块 15-24 来计算区块 30 的难度。

如果挖矿节点不诚实，它们可以操纵时间戳使新难度与网络的实际 hash 速度不匹配。我们通过按时间顺序排列时间戳，然后截掉前 o 个异常值和后 o 个异常值来解决这个问题。现在我们有一个区块“窗口” $w = b - 2 * o$ 。从前面的例子，如果 $o = 3$ 且时间戳是诚实的，那么我们将截掉区块 15-17 和 22-24，留下区块 18-21 来计算区块 30 的难度。

在截掉异常值之前，我们对时间戳进行了排序，但只是时间戳。区块难度保持不排序。我们使用每个区块的累积难度，即该区块的难度加上链中所有先前区块的难度。

使用截断后的 w 个已排序时间戳和未排序累积难度的数组（索引从 1, ..., w ），我们定义

$$\begin{aligned} totalTime &= choppedSortedTimestamps[w] - choppedSortedTimestamps[1] \\ totalDifficulty &= choppedCumulativeDifficulties[w] - choppedCumulativeDifficulties[1] \end{aligned}$$

在门罗币中，目标时间是 120 秒（2 分钟）， $l = 15$ （30 分钟）， $b = 720$ （一天）， $o = 60$ （2 小时）。^{12,13}

区块难度不存储在区块链中，因此下载区块链副本并验证所有区块合法性的人需要从记录的时间戳重新计算难度。前 $b + l = 735$ 个区块需要考虑一些规则。

规则 1： 完全忽略创世区块（区块 0， $d = 1$ ）。区块 1 和 2 的 $d = 1$ 。

规则 2： 在截掉异常值之前，尝试获得窗口 w 来计算总计。

¹¹ 门罗币开发者已成功更改其协议 11 次，几乎所有用户和矿工都采用了每个分叉：v1 2014 年 4 月 18 日（创世版本）[?]; v2 2016 年 3 月; v3 2016 年 9 月; v4 2017 年 1 月; v5 2017 年 4 月; v6 2017 年 9 月; v7 2018 年 4 月; v8 和 v9 2018 年 10 月; v10 和 v11 2019 年 3 月; v12 2019 年 11 月。核心 git 仓库的 README 包含每个版本协议更改的摘要。

¹² 在 2016 年 3 月（协议 v2），门罗币从 1 分钟目标出块时间改为 2 分钟目标出块时间 [?]。其他难度参数一直相同。

¹³ 门罗币的难度算法与最先进的算法相比可能是次优的 [?]。幸运的是它“对自私挖矿有相当强的抵抗力” [?]，这是一个必要的特性。

规则 3: w 个区块之后, 截掉高和低异常值, 按比例调整截掉的数量直到 b 个区块。如果先前区块的数量 (减去 w) 是奇数, 比高异常值多截掉一个低异常值。

规则 4: b 个区块之后, 采样最早的 b 个区块直到 $b+l$ 个区块, 此后一切正常进行——滞后 l 。

门罗币工作量证明 (PoW) (Monero proof of work (PoW))

门罗币在不同的协议版本中使用了几种不同的工作量证明 hash 算法 (输出为 32 字节)。最初的算法称为 Cryptonight, 被设计为在 GPU、FPGA 和 ASIC 架构上相对低效 [?], 与 SHA256 等标准 hash 函数相比。在 2018 年 4 月 (协议 v7), 新区块被要求开始使用一种略微修改的变体, 以对抗 Cryptonight ASIC 的出现 [?]. 另一种略作修改的变体, 名为 Cryptonight V2, 在 2018 年 10 月 (v8) 实现 [?], Cryptonight-R (基于 Cryptonight 但比简单调整有更实质性变化) 从 2019 年 3 月 (v10) 开始用于新区块 [?]. 一个全新的工作量证明称为 RandomX [?] 被设计并在 2019 年 11 月 (v12) 强制用于新区块, 旨在长期抵抗 ASIC [?].

7.3 货币供应 (Money supply)

在基于区块链的加密货币中, 创造货币有两种基本机制。

第一, 货币的创造者可以在创世消息中凭空创造硬币并分发给人们。这通常被称为”空投 (airdrop)”。有时创造者在所谓的”预挖 (pre-mine)”中给自己大量硬币。 [?]

第二, 货币可以作为挖掘区块的奖励自动分发, 很像挖掘黄金。这里有两种类型。在比特币模型中, 总可能供应量是有上限的。区块奖励缓慢下降到零, 之后不再创造更多货币。在通货膨胀模型中, 供应量无限增加。

门罗币基于一种名为 Bytecoin 的货币, 该货币有相当大的预挖, 之后是区块奖励 [?]. 门罗币没有预挖, 正如我们将看到的, 其区块奖励缓慢下降到一个较小的金额, 之后所有新区块都奖励相同的金额, 使门罗币成为通货膨胀型货币。

7.3.1 区块奖励 (Block reward)

区块矿工在挖掘 nonce 之前, 制作一笔没有 input 且至少有一个 output 的”矿工交易”。¹⁴总 output 金额等于区块奖励加上将被纳入区块的所有交易的手续费, 并以明文传达。收到已挖掘区块的节点必须验证区块奖励是否正确, 并且可以通过将所有过去的区块奖励加起来计算当前的货币供应量。

除了分发货币, 区块奖励还激励挖矿。如果没有区块奖励 (也没有其他机制), 为什么有人会挖新区块? 也许是利他主义或好奇心。然而, 矿工太少使得恶意行为者容易聚集超过 50% 的网络 hash 速率并轻松重写最近的链历史。¹⁵这也是为什么在门罗币中区块奖励不会降到零。

¹⁴ 矿工交易可以有任意数量的 output, 但目前核心实现只能创建一个。此外, 与普通交易不同, 对矿工交易的权重没有明确限制。它们受最大区块权重的功能性限制。

¹⁵ 随着攻击者获得越来越高的 hash 速率份额 (超过 50%), 重写越来越旧的区块所需的时间就越少。给定一个 x 天前的区块, 拥有 hash 速率 v , 诚实 hash 速率 v_h ($v > v_h$), 重写需要 $y = x * (v_h / (v - v_h))$ 天。

有了区块奖励，矿工之间的竞争推动总 hash 速率上升，直到增加更多 hash 速率的边际成本高于获得相应比例挖掘区块的边际收益（区块以恒定速率出现）（加上一些溢价，如风险和机会成本）。这意味着随着加密货币变得更有价值，其总 hash 速率将增加，聚集超过 50% 会变得越来越困难和昂贵。

位移 (Bit shifting)

位移用于计算基础区块奖励（正如我们将在第 ?? 节中看到的，实际区块奖励有时可以低于基础金额）。

假设我们有一个整数 $A = 13$ ，位表示为 $[1101]$ 。如果我们使用位右移运算符将 A 的位向下移动 2 位，记为 $A \gg 2$ ，我们得到 $[0011].01$ ，等于 3.25。实际上最后一部分被丢弃了——“移入”了虚无，留下 $[0011] = 3$ 。¹⁶

计算门罗币的基础区块奖励 (Calculating base block reward for Monero)

让我们称当前总货币供应为 M ，货币供应的“上限”为 $L = 2^{64} - 1$ （二进制为 $[11\dots11]$ ，64 位）。¹⁷在门罗币开始时，基础区块奖励为

$\text{term} B = (L - M) \gg 20$ 。如果 $M = 0$ ，那么，以十进制格式，

$$\begin{aligned} L &= 18,446,744,073,709,551,615 \\ B_0 &= (L - 0) \gg 20 = 17,592,186,044,415 \end{aligned}$$

这些数字以“原子单位”表示——1 原子单位的门罗币不可分割。显然原子单位是荒谬的—— L 超过 18 万亿亿！我们可以将所有数字除以 10^{12} 来移动小数点，得到门罗币的标准单位（又名 XMR，门罗币的所谓“股票代码”）。

$$\begin{aligned} \frac{L}{10^{12}} &= 18,446,744.073709551615 \\ B_0 &= \frac{(L - 0) \gg 20}{10^{12}} = 17.592186044415 \end{aligned}$$

就这样，第一个区块奖励——分发给化名 `thankful_for_today`（负责启动门罗币项目的人）——大约是 17.6 个门罗币！参见附录 ?? 自行确认。¹⁸

随着区块被挖掘， M 增长，后续区块奖励降低。最初（自 2014 年 4 月的创世区块起）门罗币区块每分钟挖掘一次，但在 2016 年 3 月变为每两分钟一个区块 [?]。为了保持货币创造速率——即“排放计划”¹⁹——不变，区块奖励翻倍。这只意味着更改后，我们对新区块使用 $(L - M) \gg 19$ 而不是 $\gg 20$ 。目前基础区块奖励为

$$B = \frac{(L - M) \gg 19}{10^{12}}$$

¹⁶ 位右移 n 位等同于整数除以 2^n 。

¹⁷ 也许现在清楚了为什么范围证明（第 ?? 节）将交易金额限制为 64 位。

¹⁸ 门罗币金额在区块链中以原子单位格式存储。

¹⁹ 关于门罗币和比特币排放计划的有趣比较，请参见 [?]。

7.3.2 动态区块权重 (Dynamic block weight)

如果能立即将每笔新交易都挖入区块就好了。但如果有人恶意提交大量交易怎么办？存储每笔交易的区块链将迅速变得庞大。

一种缓解措施是固定的区块大小（字节数），这样每个区块的交易数量受到限制。如果诚实的交易量上升怎么办？每位交易作者会通过向矿工提供手续费来竞标新区块中的位置。矿工会专注于挖掘手续费最高的交易。随着交易量增加，小额交易（如 Alice 从 Bob 那里买苹果）的手续费将变得令人望而却步。只有愿意出价高于其他所有人的才能将他们的交易纳入区块链。²⁰

门罗币通过动态区块权重避免了这些极端（无限 vs 固定）。

大小 vs 权重 (Size vs Weight)

由于 Bulletproofs 的添加 (v8)，交易和区块大小不再被严格考虑。现在使用的术语是交易权重 (transaction weight)。矿工交易（见第 ?? 节）或具有两个 output 的普通交易的交易权重等于字节大小。当普通交易有两个以上的 output 时，权重略高于大小。

回顾第 ?? 节，一个 Bulletproof 占用 $(2 \cdot \lceil \log_2(64 \cdot p) \rceil + 9) \cdot 32$ 字节，因此随着更多 output 被添加，范围证明的额外存储是次线性的。然而，Bulletproof 验证是线性的，所以人为增加交易权重将“定价”额外的验证时间（称为“clawback”）。

假设我们有一笔包含 p 个 output 的交易，如果 p 不是 2 的幂，我们创建足够的虚拟 output 来填补差距。我们计算实际 Bulletproof 大小与所有 Bulletproof 大小（如果那些 $p +$ “虚拟 output”在 2-output 交易中）之间的差异（如果 $p = 2$ 则为 0）。我们只 claw back 差异的 80%。²¹

$$\text{transaction_clawback} = 0.8 * [(23 * (p + \text{num_dummy_outs})/2) \cdot 32 - (2 \cdot \lceil \log_2(64 \cdot p) \rceil + 9) \cdot 32]$$

因此交易权重为

$$\text{transaction_weight} = \text{transaction_size} + \text{transaction_clawback}$$

区块的权重等于其组成交易的权重之和加上矿工交易的权重。

长期区块权重 (Long term block weight)

如果允许动态区块快速增长，区块链可能很快变得不可管理 [?]。为了缓解这一点，最大区块权重受到长期区块权重 (long term block weights) 的约束。每个区块除了其正常权重外，还有一个基

²⁰ 比特币有交易量过载的历史。这个网站 (<https://bitcoinfees.info/>) 记录了遇到的荒谬手续费水平（最高时每笔交易相当于 35 美元）。

²¹ 注意

$\text{term} \log_2(64 \cdot 2) = 7, 2 * 7 + 9 = 23。$

于前一个区块有效中位数长期权重计算的”长期权重”。²²一个区块的有效中位数长期权重与最近 100,000 个区块的长期权重中位数（包括其自身）相关。^{23,24}

$$\text{longterm_block_weight} = \min\{\text{block_weight}, 1.4 * \text{previous_effective_longterm_median}\}$$

$$\text{effective_longterm_median} = \max\{300\text{kB}, \text{median_100000blocks_longterm_weights}\}$$

如果正常区块权重长期保持较大，那么有效中位数长期权重至少需要 50,000 个区块（约 69 天）才能上升 40%（那是给定长期权重成为中位数所需的时间）。

累积中位数权重 (Cumulative median weight)

交易量可以在短时间内剧烈变化，特别是在节假日前后 [?]。为适应这一点，门罗币允许区块权重的短期灵活性。为了平滑瞬时变异性，区块的累积中位数使用最近 100 个区块的正常区块权重中位数（包括其自身）。

$$\text{cumulative_weights_median} = \max\{300\text{kB}, \min\{\max\{300\text{kB}, \text{median_100blocks_weights}\}, 50 * \text{effective_longterm_median}\}\}$$

将要添加到区块链的下一个区块以此方式受限：²⁵

$$\text{max_next_block_weight} = 2 * \text{cumulative_weights_median}$$

虽然最大区块权重在几百个区块后可以升至有效中位数长期权重的 100 倍，但在接下来的 50,000 个区块中不能超过其 40% 以上。因此，长期区块权重增长受长期权重约束，短期权重可能会激增至超过其稳态值。

7.3.3 区块奖励惩罚 (Block reward penalty)

要挖掘大于累积中位数的区块，矿工必须以减少区块奖励的形式付出代价或惩罚。这意味着最大区块权重内实际上有两个区域：无惩罚区和惩罚区。中位数可以缓慢上升，允许逐渐增大的区块无需惩罚。

如果预期区块权重大于累积中位数，那么，给定基础区块奖励 B，区块奖励惩罚为

$$P = B * ((\text{block_weight} / \text{cumulative_weights_median}) - 1)^2$$

²² 与区块难度类似，区块权重和长期区块权重由区块链验证者计算和存储，而不是包含在区块链数据中。

²³ 在长期权重被实现之前创建的区块的长期权重等于其正常权重，因此我们无需担心创世区块或早期区块的细节。一个全新的链可以轻松做出明智的选择。

²⁴ 在门罗币开始时，“300kB”项是 20kB，然后在 2016 年 3 月（协议 v2）增加到 60kB [?]，自 2017 年 4 月（协议 v5）起一直是 300kB [?]。动态区块权重中位数中的这个非零“底价”在绝对交易量低时有助于应对瞬时交易量变化，特别是在门罗币采用的早期阶段。

²⁵ 累积中位数在协议 v8 中替代了“M100”（类似的中位数项）。本报告第一版 [?] 中描述的惩罚和手续费使用了 M100。

实际区块奖励因此为²⁶

$$B^{\text{actual}} = B - P$$

$$B^{\text{actual}} = B * (1 - ((\text{block_weight}/\text{cumulative_weights_median}) - 1)^2)$$

使用 $\wedge^2$ 操作意味着惩罚是次比例于区块权重的。区块权重大于先前 `cumulative_weights_median` 10% 只有 1% 的惩罚，大 50% 是 25% 惩罚，大 90% 是 81% 惩罚，依此类推。[?]

我们可以预期，当添加另一笔交易的手续费大于产生的惩罚时，矿工工会创建大于累积中位数的区块。

7.3.4 动态最低手续费 (Dynamic minimum fee)

为了防止恶意行为者用可能用于污染环签名的交易充斥区块链，以及一般性地不必要地膨胀它，门罗币有一个每字节交易数据的最低手续费。²⁷最初这是 0.01 XMR/KiB（在协议 v1 早期添加）[?]，然后在 2016 年 9 月（v3）变为 0.002 XMR/KiB。²⁸

在 2017 年 1 月（v4），添加了一个动态每 KiB 手续费算法 [?, ?, ?, ?]，²⁹然后随着 Bulletproofs（v8）带来的交易权重减少，它从每 KiB 变为每字节。该算法最重要的特性是它防止最低可能总手续费超过区块奖励（即使在小区块奖励和大区块权重的情况下），据信这会导致不稳定 [?, ?, ?]。³⁰

手续费算法 (The fee algorithm)

我们围绕一个参考交易 [?] 来构建手续费算法，该交易权重为 3000 字节（类似于基本的 `RCTTypeBulletproof2` 2-input、2-output 交易，通常约 2600 字节）³¹，以及在中位数处于最小值（最小的无惩罚区，300kB）时抵消惩罚所需的手续费 [?]。换句话说，303kB 区块权重引起的惩罚。

首先，将权重为 TW 的交易添加到权重为 BW 的区块时，平衡边际惩罚 MP 的手续费 F 为

²⁶ 在机密交易 (RingCT) 实现之前 (v4)，所有金额以明文传达，在某些早期协议版本中分成多个面额（例如 1244 → 1000 + 200 + 40 + 4）。为了减小矿工交易大小，核心实现在 v2-v3 中截掉了区块奖励的最低有效数字（低于 0.0001 门罗币的任何金额；见 `BASE_REWARD_CLAMP_THRESHOLD`）。多余的小额没有丢失，只是留给了未来的区块奖励。更一般地说，自 v2 起，此处的区块奖励计算只是矿工交易 output 中可以分发的实际区块奖励的上限。还值得注意的是，非常早期交易中明文金额未分面额的 output 不能在当前实现中用于环签名，因此要花费它们，它们被迁移到分面额的“可混合”output 中，然后通过使用相同金额的其他面额创建环，在普通 RingCT 交易中花费。关于这些古老 pre-RingCT output 的确切现代协议规则尚不清楚。

²⁷ 这个最低值由节点共识协议而不是区块链协议强制执行。如果交易的手续费低于最低值，大多数节点不会将交易转发给其他节点（至少部分原因是只有可能被某人挖掘的交易才会被传递 [?])，但它们会接受包含该交易的新区块。特别是，这意味着不需要维护与手续费算法的向后兼容性。

²⁸ 单位 KiB (kibibyte, 1 KiB = 1024 字节) 与 kB (kilobyte, 1 kB = 1000 字节) 不同。

²⁹ 基础手续费在 2017 年 4 月（协议 v5）从 0.002 XMR/KiB 变为 0.0004 XMR/KiB [?]。本报告第一版描述了原始的动态手续费算法 [?]

³⁰ 本节概念主要归功于 Francisco Cabañas（又名“ArticMine”），门罗币动态区块和手续费系统的架构师。参见 [?, ?, ?]。

³¹ 一个基本的 1-input、2-output 比特币交易是 250 字节 [?]，2-input/2-output 为 430 字节。

$$F = MP = B * (([BW + TW]/\text{cumulative_median} - 1)^2 - B * ((BW/\text{cumulative_median} - 1)^2$$

定义区块权重因子 $WF_b = (BW/\text{cumulative_median}-1)$, 交易权重因子 $WF_t = (TW/\text{cumulative_median})$, 使我们能够简化

$$F = B * (2 * WF_b * WF_t + WF_t^2)$$

使用一个 300kB 的区块（累积中位数在默认值 300kB）和我们的 3000 字节参考交易，

$$F_{\text{ref}} = B * (2 * 0 * WF_t + WF_t^2)$$

$$F_{\text{ref}} = B * WF_t^2$$

$$F_{\text{ref}} = B * \left(\frac{TW_{\text{ref}}}{\text{cumulative_median}_{\text{ref}}} \right)^2$$

这个手续费分摊在惩罚区的 1%（300000 中的 3000）。我们可以用广义参考交易将相同的手续费分摊在任何惩罚区的 1% 中。

$$\begin{aligned} \frac{TW_{\text{ref}}}{\text{cumulative_median}_{\text{ref}}} &= \frac{TW_{\text{general-ref}}}{\text{cumulative_median}_{\text{general}}} \\ 1 &= \left(\frac{TW_{\text{general-ref}}}{\text{cumulative_median}_{\text{general}}} \right) * \left(\frac{\text{cumulative_median}_{\text{ref}}}{TW_{\text{ref}}} \right) \\ F_{\text{general-ref}} &= F_{\text{ref}} \\ &= F_{\text{ref}} * \left(\frac{TW_{\text{general-ref}}}{\text{cumulative_median}_{\text{general}}} \right) * \left(\frac{\text{cumulative_median}_{\text{ref}}}{TW_{\text{ref}}} \right) \\ F_{\text{general-ref}} &= B * \left(\frac{TW_{\text{general-ref}}}{\text{cumulative_median}_{\text{general}}} \right) * \left(\frac{TW_{\text{ref}}}{\text{cumulative_median}_{\text{ref}}} \right) \end{aligned}$$

现在我们可以根据给定中位数下的实际交易权重来缩放手续费，例如如果交易是惩罚区的 2%，手续费就翻倍。

$$\begin{aligned} F_{\text{general}} &= F_{\text{general-ref}} * \frac{TW_{\text{general}}}{TW_{\text{general-ref}}} \\ F_{\text{general}} &= B * \left(\frac{TW_{\text{general}}}{\text{cumulative_median}_{\text{general}}} \right) * \left(\frac{TW_{\text{ref}}}{\text{cumulative_median}_{\text{ref}}} \right) \end{aligned}$$

这重新排列为我们一直在追求的默认每字节手续费。

$$\begin{aligned} f_{\text{default}}^B &= F_{\text{general}}/TW_{\text{general}} \\ f_{\text{default}}^B &= B * \left(\frac{1}{\text{cumulative_median}_{\text{general}}} \right) * \left(\frac{3000}{300000} \right) \end{aligned}$$

当交易量低于中位数时，没有真正的理由让手续费处于参考水平 [?]. 我们将最低值设为默认值的 1/5。

$$\begin{aligned} f_{\text{min}}^B &= B * \left(\frac{1}{\text{cumulative_weights_median}} \right) * \left(\frac{3000}{300000} \right) * \left(\frac{1}{5} \right) \\ f_{\text{min}}^B &= B * \left(\frac{1}{\text{cumulative_weights_median}} \right) * 0.002 \end{aligned}$$

手续费中位数 (The fee median)

事实证明，对费用使用累积中位数使得垃圾邮件攻击成为可能。通过将短期中位数提升到最高值（50 倍长期中位数），攻击者可以使用最低手续费以极低的成本维持高区块权重（相对于有机交易量）。

为避免这种情况，我们限制交易的费用使用最小可用中位数，这在所有情况下都倾向于更高的费用。³²

$$\text{smallest_median} = \max\{300\text{kB}, \min\{\text{median_100blocks_weights}, \text{effective_longterm_median}\}\}$$

在交易量上升期间倾向于更高的费用也有助于调整短期中位数并确保交易不被搁置，因为矿工更有可能挖掘到惩罚区。

因此实际最低手续费为

³² 攻击者可以花费刚好够的费用使短期中位数达到 50* 长期中位数。按照当前（撰写本文时）2 XMR 的区块奖励，优化的攻击者可以每 50 个区块增加短期中位数 17%，约 1300 个区块（约 43 小时）后达到上限，每区块花费 0.39*2 XMR，总设置成本约 1000 XMR（按当前估值约 6.5 万美元），然后恢复到最低手续费。当手续费中位数等于无惩罚区时，填满无惩罚区的最低总手续费为 0.004 XMR（按当前估值约 0.26 美元）。如果手续费中位数等于长期中位数，在垃圾邮件场景中它将是无惩罚区的 1/50。因此只需短期中位数情况的 50 倍，每区块 0.2 XMR（每区块 13 美元）。这折合每天 2.88 XMR 对比每天 144 XMR（持续 69 天，直到长期中位数上升 40%）来维持每个区块 50* 长期中位数的区块权重。1000 XMR 的设置成本在前一种情况下是值得的，但在后一种情况下不是。这在排放尾部将减少到 300 XMR 设置和 43 XMR 维护。

部分 II

扩展

Monero 交易相关知识证明 (Monero Transaction-Related Knowledge Proofs)

Monero 是一种货币，如同任何货币一样，其用途是复杂的。从企业财务、市场交易到法律仲裁，不同的利益相关方可能希望了解交易的详细信息。

你如何确定收到的钱来自特定的人？或者在对方否认的情况下，证明你确实向某人发送了某个 output 或交易？在 Monero 的公共账本中，发送者和接收者都是模糊的。你如何在不泄露私钥的情况下证明你拥有一定数量的资金？在 Monero 中，金额对观察者来说是完全隐藏的。

我们考虑了几种类型的交易断言，其中一些已在 Monero 中实现，可通过内置钱包工具使用。我们还概述了一个审计个人或组织所持全部余额的框架，该框架不会泄露其未来交易的信息。

8.1 Monero 中的交易证明 (Transaction proofs in Monero)

Monero 的交易证明正在进行更新 [?]。目前已实现的证明都是「版本 1」，不包含域分离。我们仅描述最先进的证明，无论它们是目前已实现的、计划在未来版本中实现的 [?]，还是可能会也可能不会被实现的假设性证明（第 ?? 节 [?] 和第 ?? 节）。

8.1.1 多基 Monero 交易证明 (Multi-base Monero transaction proofs)

有一些细节需要注意。大多数 Monero 交易证明涉及多基证明（回忆第 ?? 节）。在相关之处，域分隔符为 $T_{txprf2} = \mathcal{H}_n(\text{"TXPROOF_V2"})$ 。¹被签名的消息通常（除非另有说明）为 $m = \mathcal{H}_n(\text{tx_hash}, \text{message})$ ，其中 tx_hash 是相关交易的 ID（第 ?? 节）， message 是证明者或第三方可以提供的可选消息，以确保证明者确实制作了证明而非窃取了他人的。

证明以 base-58 编码，这是一种最初为 Bitcoin 引入的二进制到文本的编码方案 [?]。验证这些证明始终涉及先将其从 base-58 解码回二进制。注意验证者还需要访问区块链，以便使用交易 ID 引用来获取一次性地址等信息。²

证明中密钥前缀的结构有些不对称，部分原因是累积的更新尚未重新组织。2 基「版本 2」证明的挑战按以下格式组装，如果「基密钥 1」是 G ，则其在挑战中的位置用 32 个零字节填充，

$$c = \mathcal{H}_n(m, \text{public key 2}, \text{proof part 1}, \text{proof part 2}, T_{txprf2}, \text{public key 1}, \text{base key 2}, \text{base key 1})$$

8.1.2 证明交易 input 的创建 (SpendProofV1)

假设我们进行了一笔交易，并想要证明它。显然，通过在新消息上重新制作交易 input 的签名，任何验证者都只能得出结论：原始交易确实是我们制作的。重新制作交易所有 input 的签名意味着我们必须制作了整笔交易（回忆第 ?? 节），或者至少完全为其提供了资金。³

所谓的「SpendProof」包含对交易所有 input 的重新制作的签名。重要的是，SpendProof 环签名重新使用了原始环成员，以避免通过环交集识别真正的签名者。

SpendProof 已在 Monero 中实现。为了编码传输给验证者，证明者将前缀字符串「SpendProofV1」与签名列表连接起来。注意前缀字符串不在 base-58 编码中，也不需要编码/解码，因为其目的是为了人类可读性。

SpendProof

SpendProof 出人意料地不使用 MLSAG，而是使用 Monero 最初的环签名方案，该方案用于最早的交易协议（RingCT 之前）[?]

1. 计算 key image $\tilde{K} = k_\pi^o \mathcal{H}_p(K_\pi^o)$ 。
2. 生成随机数 $\alpha \in_R \mathbb{Z}_l$ ，以及对 $i \in \{1, 2, \dots, n\}$ 但排除 $i = \pi$ 的随机数 $c_i, r_i \in_R \mathbb{Z}_l$ 。
3. 计算

$$c_{tot} = \mathcal{H}_n(m, [r_1 G + c_1 K_1^o], [r_1 \mathcal{H}_p(K_1^o) + c_1 \tilde{K}], \dots, [\alpha G], [\alpha \mathcal{H}_p(K_\pi^o)], \dots, \text{etc.})$$

¹正如第 ?? 节所述，哈希函数应通过前缀标签进行域分离。当前 Monero 交易证明的实现没有域分离，因此本章中的所有标签都是尚未实现的功能。

²交易 ID 通常与证明分开传达。

³正如我们将在第 ?? 章中看到的，制作了一个 input 签名的人不一定制作了所有 input 签名。

4. 定义真正的挑战

$$c_{\pi} = c_{tot} - \sum_{i=1, i \neq \pi}^n c_i$$

5. 定义 $r_{\pi} = \alpha - c_{\pi} * k_{\pi}^o \pmod{l}$ 。

签名为 $\sigma = (c_1, r_1, c_2, r_2, \dots, c_n, r_n)$ 。

验证 (Verification)

验证给定交易上的 SpendProof 时，验证者使用相关引用交易中的信息（例如 key image，以及从其他交易获取一次性地址的 output 偏移量）确认所有环签名有效。

1. 计算

$$c_{tot} = \mathcal{H}_n(\mathbf{m}, [r_1 G + c_1 K_1^o], [r_1 \mathcal{H}_p(K_1^o) + c_1 \tilde{K}], \dots, [r_n G + c_n K_n^o], [r_n \mathcal{H}_p(K_n^o) + c_n \tilde{K}])$$

2. 检查

$$c_{tot} \stackrel{?}{=} \sum_{i=1}^n c_i$$

原理 (Why it works)

注意当只有一个环成员时，该方案与 bLSAG（第 ?? 节）相同。添加伪造成员时，不是将挑战 $c_{\pi+1}$ 传入新的挑战哈希，而是将成员添加到原始哈希中。由于以下方程

$$c_s = c_{tot} - \sum_{i=1, i \neq s}^n c_i$$

对任意索引 s 都平凡成立，验证者无法识别真正的挑战。此外，如果不知道 k_{π}^o ，证明者将永远无法正确定义 r_{π} （概率可忽略不计的情况除外）。

8.1.3 证明交易 output 的创建 (OutProofV2)

现在假设我们向某人发送了资金（一个 output），并想要证明它。交易 output 的核心包含三个组成部分：接收者地址、发送金额和交易私钥。金额是编码过的，因此我们实际上只需要地址和交易私钥即可开始。任何删除或丢失了交易私钥的人都将无法制作 OutProof，因此从这个意义上说，OutProof 是所有 Monero 交易证明中最不可靠的。⁴

我们这里的任务是展示一次性地址是从接收者的地址生成的，并允许验证者重构 output 承诺。我们通过提供发送者-接收者共享秘密 rK^v 来做到这一点，然后通过基密钥 G 和 K^v 上签名一个 2 基签名（第 ?? 节）来证明我们创建了它，并且它与交易公钥和接收者地址相对应。验证者可以使用共享秘密来检查接收者（第 ?? 节）、解码金额（第 ?? 节），并重构 output 承诺（第 ?? 节）。我们提供普通地址和子地址的详细信息。

⁴ 我们可以将「OutProof」理解为展示一个 output 从证明者处「发出」。相应的「InProof」（第 ?? 节）展示的是「进入」证明者地址的 output。

OutProof

为向地址 (K^v, K^s) 或子地址 $(K^{v,i}, K^{s,i})$ 发送的 output 生成证明, 交易私钥为 r , 发送者-接收者共享秘密为 rK^v 。回忆存储在交易数据中的交易公钥为 rG 或 $rK^{s,i}$, 取决于接收者是否使用子地址 (第 ?? 节)。

1. 生成随机数 $\alpha \in_R \mathbb{Z}_l$, 并计算

- (a) 普通地址: αG 和 αK^v
- (b) 子地址: $\alpha K^{s,i}$ 和 $\alpha K^{v,i}$

2. 计算挑战

- (a) 普通地址: ⁵

$$c = \mathcal{H}_n(\mathbf{m}, [rK^v], [\alpha G], [\alpha K^v], [T_{txprf2}], [rG], [K^v], [0])$$

- (b) 子地址:

$$c = \mathcal{H}_n(\mathbf{m}, [rK^{v,i}], [\alpha K^{s,i}], [\alpha K^{v,i}], [T_{txprf2}], [rK^{s,i}], [K^{v,i}], [K^{s,i}])$$

3. 定义响应⁶ $r^{resp} = \alpha - c * r$ 。

4. 签名为 $\sigma^{outproof} = (c, r^{resp})$ 。

证明者可以生成一批 OutProof, 然后一起发送给验证者。他将前缀字符串「OutProofV2」与证明列表连接起来, 其中每项 (以 base-58 编码) 包含发送者-接收者共享秘密 rK^v (子地址则为 $rK^{v,i}$) 及其对应的 $\sigma^{outproof}$ 。我们假设验证者知道每条证明对应的地址。

验证 (Verification)

1. 计算挑战

- (a) 普通地址:

$$c' = \mathcal{H}_n(\mathbf{m}, [rK^v], [r^{resp}G + c * rG], [r^{resp}K^v + c * rK^v], [T_{txprf2}], [rG], [K^v], [0])$$

- (b) 子地址:

$$c' = \mathcal{H}_n(\mathbf{m}, [rK^{v,i}], [r^{resp}K^{s,i} + c * rK^{s,i}], [r^{resp}K^{v,i} + c * rK^{v,i}], [T_{txprf2}], [rK^{s,i}], [K^{v,i}], [K^{s,i}])$$

2. 若 $c = c'$, 则证明者知道 r , 且 rK^v 是 rG 和 K^v 之间的合法共享秘密 (概率可忽略不计的情况除外)。
3. 验证者应检查所提供的接收者地址是否能用于从相关交易生成一次性地址 (普通地址和子地址的计算方式相同)

$$K^s \stackrel{?}{=} K_t^o - \mathcal{H}_n(rK^v, t)$$

⁵ 此处的 '0' 值是 32 字节的零字节编码。

⁶ 由于可用符号有限, 我们遗憾地用 r 同时表示响应和交易私钥。必要时将使用上标 'resp' 表示「response」以区分两者。

4. 他们还应解码 output 金额 b_t , 计算 output 掩码 y_t , 并尝试重构对应的 output 承诺⁷

$$C_t^b \stackrel{?}{=} y_t G + b_t H$$

8.1.4 证明 output 的所有权 (InProofV2)

OutProof 展示证明者向某个地址发送了一个 output, 而 InProof 展示某个地址收到了一个 output。它本质上是发送者-接收者共享秘密 rK^v 的另一「面」。这次证明者证明他知道 K^v 中的 k^v , 并且结合交易公钥 rG 后共享秘密 $k^v * rG$ 会出现。

一旦验证者有了 rK^v , 他们就可以通过以下方式检查对应的一次性地址是否由证明者的地址所拥有 $K^o - \mathcal{H}_n(k^v * rG, t) * G \stackrel{?}{=} K^s$ (第 ?? 节)。通过对区块链上所有交易公钥制作 InProof, 证明者将暴露他所有的已拥有的 output。

直接给验证者 view key 会有同样的效果, 但一旦他们拥有了该密钥, 验证者将能够识别未来创建的 output 的所有权。使用 InProof, 证明者可以保留对其私钥的控制权, 代价是证明 (然后验证) 每个 output 是否被拥有所需的时间。

InProof

InProof 的构建方式与 OutProof 相同, 只是基密钥现在是 $\mathcal{J} = \{G, rG\}$, 公钥是 $\mathcal{K} = \{K^v, rK^v\}$, 签名密钥是 k^v 而非 r 。我们将仅展示验证步骤以说明我们的意思。注意密钥前缀的顺序发生了变化 (rG 和 K^v 互换了位置), 以匹配每个密钥的不同角色。

多个 InProof 可以一起发送给验证者, 这些 InProof 与同一地址拥有的多个 output 相关。它们以字符串「InProofV2」为前缀, 每项 (以 base-58 编码) 包含发送者-接收者共享秘密 rK^v (或 $rK^{v,i}$) 及其对应的 $\sigma^{inproof}$ 。

验证 (Verification)

1. 计算挑战

- (a) 普通地址:

$$c' = \mathcal{H}_n(\mathbf{m}, [rK^v], [r^{resp}G + c * K^v], [r^{resp} * rG + c * k^v * rG], [T_{txprf2}], [K^v], [rG], [0])$$

- (b) 子地址:

$$c' = \mathcal{H}_n(\mathbf{m}, [rK^{v,i}], [r^{resp}K^{s,i} + c * K^{v,i}], [r^{resp} * rK^{s,i} + c * k^v * rK^{s,i}], [T_{txprf2}], [K^{v,i}], [rK^{s,i}], [K^{s,i}])$$

2. 若 $c = c'$, 则证明者知道 k^v , 且 $k^v * rG$ 是 K^v 和 rG 之间的合法共享秘密 (概率可忽略不计的情况除外)。

⁷ 有效的 OutProof 签名并不一定意味着所考虑的接收者就是真正的接收者。恶意证明者可以生成随机 view key K'^v , 计算 $K'^s = K^o - \mathcal{H}_n(rK'^v, t) * G$, 并提供 (K'^v, K'^s) 作为名义接收者。通过重新计算 output 承诺, 验证者可以更有信心地确认所讨论的接收者地址是合法的。然而, 证明者和接收者可以合谋使用 K'^v 编码 output 承诺, 而一次性地址使用 (K^v, K^s) 。由于接收者需要知道私钥 k'^v (假设 output 金额仍然可花费), 这种程度的欺骗的实际用处令人质疑。为什么接收者不直接使用 (K'^v, K'^s) (或其他一次性地址) 来接收整个 output 呢? 由于 C_t^b 的计算与接收者相关, 我们认为所述的 OutProof 验证过程是足够的。换言之, 证明者无法在不与接收者合谋的情况下用它来欺骗验证者。

使用一次性地址密钥证明「完全」所有权 (Prove ‘full’ ownership with the one-time address key)

虽然 InProof 表明一次性地址是用特定地址构建的（概率可忽略不计的情况除外），但这并不意味着证明者能够花费该 output。只有能够花费 output 的人才真正拥有它。

在 InProof 完成后，证明所有权就像用 spend key 对消息签名一样简单。⁸

8.1.5 证明已拥有的 output 未在某交易中被花费 (UnspentProof)

证明 output 已花费或未花费看似就像用 $\mathcal{J} = \{G, \mathcal{H}_p(K^o)\}$ 和 $\mathcal{K} = \{K^o, \tilde{K}\}$ 上的多基证明重新创建其 key image 一样简单。虽然这确实有效，但验证者必须知道 key image，这也会暴露未花费的 output 在未来何时被花费。

事实证明，我们可以在不暴露 key image 的情况下证明某个 output 没有在特定交易中被花费。此外，我们可以通过将此 UnspentProof [?] 扩展到「它作为环成员被包含在内的所有交易」来证明它目前未被花费。⁹

更具体地说，我们的 UnspentProof 说明区块链上某交易中的给定 key image 是否与其对应环中的特定一次性地址相对应。顺便提一下，正如我们将看到的，UnspentProof 与 InProof 是相辅相成的。

设置 UnspentProof

UnspentProof 的验证者必须知道 rK^v ，即给定已拥有的 output 的发送者-接收者共享秘密，该 output 的一次性地址为 K^o ，交易公钥为 rG 。他要么知道 view key k^v （这使他能够计算 $k^v * rG$ 并检查 $K^o - \mathcal{H}_n(k^v * rG, t) * G \stackrel{?}{=} K^s$ ，从而知道被测试的 output 属于证明者——回忆第 ?? 节），要么证明者提供了 rK^v 。这就是 InProof 的用武之地，因为有了 InProof，验证者就可以确信 rK^v 合法地来自证明者的 view key，并且对应一个已拥有的 output，而无需了解私有 view key。

在验证 UnspentProof 之前，验证者将知道待测试的 key image $\tilde{K}_?$ ，并检查其对应的环是否包含证明者所拥有 output 的一次性地址 K^o 。然后他计算部分「花费」镜像 $\tilde{K}_?^s$ 。

$$\tilde{K}_?^s = \tilde{K}_? - \mathcal{H}_n(rK^v, t) * \mathcal{H}_p(K^o)$$

如果被测试的 key image 是从 K^o 创建的，则结果点将为 $\tilde{K}_?^s = k^s * \mathcal{H}_p(K^o)$ 。

UnspentProof

证明者创建两个多基证明（回忆第 ?? 节）。他的地址（拥有相关 output）为 (K^v, K^s) 或 $(K^{v,i}, K^{s,i})$ 。¹⁰

⁸ 直接提供这种签名的功能在 Monero 中似乎不可用，不过正如我们将看到的，ReserveProof（第 ?? 节）确实包含了这种签名。

⁹ UnspentProof 尚未在 Monero 中实现。

¹⁰ UnspentProof 对子地址和普通地址的制作方式相同。需要子地址的完整 spend key，例如 $k^{s,i} = k^s + \mathcal{H}_n(k^v, i)$ （第 ?? 节）。

1. 一个 3 基证明, 签名密钥为 k^s ,

$$\begin{aligned}\mathcal{J}_3^{unspent} &= \{[G], [K^s], [\tilde{K}_?^s]\} \\ \mathcal{K}_3^{unspent} &= \{[K^s], [k^s * K^s], [k^s * \tilde{K}_?^s]\}\end{aligned}$$

2. 一个 2 基证明, 签名密钥为 $k^s * k^s$,

$$\begin{aligned}\mathcal{J}_2^{unspent} &= \{[G], [\mathcal{H}_p(K^o)]\} \\ \mathcal{K}_2^{unspent} &= \{[k^s * K^s], [k^s * k^s * \mathcal{H}_p(K^o)]\}\end{aligned}$$

连同证明 $\sigma_3^{unspent}$ 和 $\sigma_2^{unspent}$, 证明者确保传达公钥 $k^s * K^s$ 、 $k^s * \tilde{K}_?^s$ 和 $k^s * k^s * \mathcal{H}_p(K^o)$ 。

验证 (Verification)

1. 确认 $\sigma_3^{unspent}$ 和 $\sigma_2^{unspent}$ 是合法的。
2. 确保两个证明中使用了相同的公钥 $k^s * K^s$ 。
3. 检查 $k^s * \tilde{K}_?^s$ 和 $k^s * k^s * \mathcal{H}_p(K^o)$ 是否相同。如果相同, 则 output 已被花费; 如果不同, 则未被花费 (概率可忽略不计的情况除外)。

原理 (Why it works)

这种看似迂回的方法防止验证者了解未花费 output 的 $k^s * H_p(K^o)$ ——他本可以结合 rK^v 来计算其真实的 key image——同时让他确信被测试的 key image 不对应于该 output。

证明 $\sigma_2^{unspent}$ 可以用于涉及同一 output 的任意数量的 UnspentProof, 尽管如果它确实被花费了, 则只需要一个就足够了 (即 UnspentProof 也可以用来证明一个 output 已被花费)。对给定未花费 output 被引用的所有环签名进行 UnspentProof 在计算上应该不会太昂贵。一个 output 随时间推移只可能作为诱饵被包含在大约 11 个 (当前环大小) 不同的环中。

8.1.6 证明地址拥有最低未花费余额 (ReserveProofV2)

尽管在 output 未被花费时暴露其 key image 会导致隐私泄露, 但这仍然是一种有些用处的方法, 并在 UnspentProof 发明之前就已在 Monero 中实现 [?] [?]. Monero 的所谓「ReserveProof」用于通过为一些未花费的 output 创建 key image 来证明某个地址拥有最低数量的资金。

更具体地说, 给定一个最低余额, 证明者找到足够的未花费 output 来覆盖它, 用 InProof 展示所有权, 为它们创建 key image 并用 2 基证明 (使用不同的密钥前缀格式) 证明它们是基于那些 output 合法创建的, 然后用普通 Schnorr 签名证明对所用私有 spend key 的知识 (如果某些 output 由不同子地址拥有, 则可能有多个签名)。验证者可以检查 key image 是否未出现在区块链上, 因此它们的 output 必定是未花费的。

ReserveProof

ReserveProof 内的所有子证明签名的消息不同于其他证明（例如 OutProof、InProof 或 SpendProof）。这次是 $\mathbf{m} = \mathcal{H}_n(\text{message}, \text{address}, \hat{K}_1^o, \dots, \hat{K}_n^o)$ ，其中 address 是证明者普通地址 (K^v, K^s) 的编码形式（见 [?]），key image 对应要包含在证明中的未花费 output。

1. 每个 output 有一个 InProof，用于展示证明者的地址（或其某个子地址）拥有该 output。
2. 每个 output 的 key image 用 2 基证明签名，挑战格式如下

$$c = \mathcal{H}_n(\mathbf{m}, [rG + c * K^o], [r\mathcal{H}_p(K^o) + c * \tilde{K}])$$
3. 每个拥有至少一个 output 的地址（和子地址）都有一个普通 Schnorr 签名（第 ?? 节），挑战如下（普通地址和子地址相同）

$$c = \mathcal{H}_n(\mathbf{m}, K^{s,i}, [rG + c * K^{s,i}])$$

将 ReserveProof 发送给其他人时，证明者将前缀字符串「ReserveProofV2」与两个以 base-58 编码的列表连接起来（例如「ReserveProofV2, list 1, list 2」）。列表 1 中的每项与特定 output 相关，包含其交易哈希（第 ?? 节）、该交易中的 output 索引（第 ?? 节）、相关共享秘密 rK^v 、其 key image、其 InProof $\sigma^{inproof}$ 和其 key image 证明。列表 2 的项目是拥有这些 output 的地址及其 Schnorr 签名。

验证 (Verification)

1. 检查 ReserveProof 的 key image 是否未出现在区块链中。
2. 验证每个 output 的 InProof，以及所提供的某个地址拥有每个 output。
3. 验证 2 基 key image 签名。
4. 使用发送者-接收者共享秘密解码 output 金额（第 ?? 节）。
5. 检查每个地址的签名。

如果一切合法，则证明者必定拥有且未花费至少 ReserveProof 的 output 中所包含的总金额（概率可忽略不计的情况除外）。¹¹

8.2 Monero 审计框架 (Monero audit framework)

在美国，大多数公司每年对其财务报表进行审计 [?]，其中包括利润表、资产负债表和现金流量表。其中前两者在很大程度上涉及公司的内部记录，而后者涉及影响公司当前持有资金的每一笔交易。

¹¹ ReserveProof 虽然展示了对资金的完全所有权，但不包含证明给定子地址确实对应证明者普通地址的证明。

加密货币是数字现金，因此对加密货币用户现金流量表的任何审计都必须与存储在区块链上的交易相关联。

被审计者的第一个任务是识别他们当前拥有的所有 output（已花费和未花费的）。这可以使用所有地址的 InProof 来完成。大型企业可能有大量的子地址，尤其是在在线市场中运营的零售商（见第 ?? 章）。为每个子地址的所有交易创建 InProof 可能会导致证明者和验证者产生巨大的计算和存储需求。

相反，我们可以仅为证明者的普通地址（对所有交易）制作 InProof。审计者使用那些发送者-接收者共享秘密来检查是否有任何 output 由证明者的主地址或其相关子地址拥有。回忆第 ?? 节，用户的 view key 足以识别地址的子地址拥有的所有 output。

为了确保证明者不会通过隐藏某些子地址的普通地址来欺骗审计者，他还必须证明所有子地址都对应于他已知的某个普通地址。

8.2.1 证明地址和子地址的对应关系 (SubaddressProof)

SubaddressProof 展示普通地址的 view key 可用于识别给定子地址拥有的 output。¹²

SubaddressProof

SubaddressProof 的制作方式与 OutProof 和 InProof 大致相同。此处基密钥为 $\mathcal{J} = \{G, K^{s,i}\}$ ，公钥为 $\mathcal{K} = \{K^v, K^{v,i}\}$ ，签名密钥为 k^v 。同样，我们仅展示验证步骤以说明我们的意思。

验证 (Verification)

验证者知道证明者的地址 (K^v, K^s) 、子地址 $(K^{v,i}, K^{s,i})$ ，并拥有 SubaddressProof $\sigma^{subproof} = (c, r)$ 。

1. 计算挑战

$$c' = \mathcal{H}_n(\mathbf{m}, [K^{v,i}], [rG + c * K^v], [rK^{s,i} + c * K^{v,i}], [T_{txprf2}], [K^v], [K^{s,i}], [0])$$

2. 若 $c = c'$ ，则证明者知道 K^v 的 k^v ，且 $K^{s,i}$ 与该 view key 结合可以生成 $K^{v,i}$ （概率可忽略不计的情况除外）。

8.2.2 审计框架 (The audit framework)

现在我们已经准备好尽可能多地了解一个人的交易历史。¹³

¹² SubaddressProof 尚未在 Monero 中实现。

¹³ 此审计框架在 Monero 中尚未完全可用。SubaddressProof 和 UnspentProof 尚未实现，InProof 尚未为我们所解释的子地址相关优化做好准备，并且没有真正的结构来方便地获取或组织证明者和验证者所需的所有信息。

1. 证明者收集他所有账户的列表，每个账户由一个普通地址和各种子地址组成。他为所有子地址制作 SubaddressProof。与 ReserveProof 类似，他还为每个地址和子地址的 spend key 制作签名，展示他对这些地址所拥有的所有 output 的花费权。
2. 证明者为他的每个普通地址在区块链上的所有交易（即所有交易公钥）生成 InProof。这向审计者揭示了证明者的地址拥有的所有 output，因为他们可以用发送者-接收者共享秘密检查所有一次性地址。他们可以确信子地址拥有的 output 也会被识别，因为有 SubaddressProof。¹⁴
3. 证明者为他的每个已拥有的 output，在它们作为环成员出现的所有交易 input 上生成 UnspentProof。现在审计者将知道证明者的余额，并可以进一步调查已花费的 output。¹⁵
4. 可选：证明者为他花费 output 的每笔交易生成 OutProof，向审计者展示接收者和金额。此步骤仅适用于证明者保存了交易私钥的交易。

重要的是，证明者无法直接展示资金来源。他唯一的办法是向给他汇款的人请求一组证明。

1. 对于向证明者汇款的交易，其作者制作 SpendProof 证明确实发送了它。
2. 证明者的资助者还用了一个标识性公钥（例如其普通地址的 spend key）制作签名。SpendProof 和这个签名都应签名包含该标识性公钥的消息，以确保 SpendProof 没有被窃取，也不是由其他人制作的。

¹⁴ 此步骤也可以通过提供私有 view key 来完成，但这有明显的隐私影响。

¹⁵ 或者，他可以为所有已拥有的 output 制作 ReserveProof。同样，暴露未花费 output 的 key image 有明显的隐私影响。

Monero 中的多重签名 (Multisignatures in Monero)

加密货币交易不可逆转。如果有人窃取了私钥或诈骗成功，损失的资金可能永远无法追回。将签名权分散给多人可以削弱作恶者带来的潜在危险。

假设你将资金存入与一家安全公司共同管理的联名账户，该公司会监控与你账户相关的可疑活动。交易只有在你和公司双方合作的情况下才能签署。如果有人窃取了你的密钥，你可以通知公司存在问题，公司将停止为你的账户签署交易。这通常被称为”托管”服务。¹

加密货币使用”多重签名”技术来实现协作签名，即所谓的”M-of-N multisig”。在 M-of-N 中，N 个人合作生成一个联合密钥，只需要 M 个人 ($M \leq N$) 就可以用该密钥签名。本章首先介绍 N-of-N multisig 的基础知识，然后深入 N-of-N Monero multisig，推广到 M-of-N multisig，最后解释如何将 multisig 密钥嵌套在其他 multisig 密钥内部。

本章重点介绍我们认为多重签名
em 应该如何实现，基于

¹ 多重签名有多种应用场景，从企业账户到报纸订阅再到在线市场。

citeMRL-0009-multisig 中的建议以及关于高效实现的各种观察。我们尽量在脚注中指出当前实现与本文描述之间的差异。² 我们的贡献在于详细阐述 M-of-N multisig, 以及一种嵌套 multisig 密钥的新方法。

9.1 与共同签名者通信 (Communicating with co-signers)

构建联合密钥和联合交易需要在全世界各地的人们之间传递秘密信息。为了保护这些信息不被观察者窃取, 共同签名者需要加密他们发送给彼此的消息。

Diffie-Hellman 交换 (ECDH) 是使用椭圆曲线密码学加密消息的一种非常简单的方法。我们已经在

refsec:pedersen_{monero} *Monero output* $r K^v$ 传达给接收者。它看起来是这样的:

$$amount_t = b_t \oplus_s \mathcal{H}_n(\text{"amount"}, \mathcal{H}_n(r K_B^v, t))$$

我们可以轻松地将此扩展到任何消息。首先将消息编码为一系列比特, 然后将其分割成与 $\mathcal{H}_n$ 输出大小相等的块。生成一个随机数 $r \in \mathbb{Z}_l$, 并使用接收者的公钥 K 对所有消息块执行 Diffie-Hellman 交换。将这些加密块连同公钥 rG 一起发送给目标接收者, 接收者随后可以使用共享秘密 krG 解密消息。消息发送者还应对其加密消息 (或为了简便起见, 仅对加密消息的 hash) 创建签名, 以便接收者可以确认消息未被篡改 (签名仅可在正确的消息 m 上验证)。

由于加密对于 Monero 这样的加密货币的运行并非必需, 我们不认为有必要深入更多细节。好奇的读者可以参考这篇优秀的概念概述

citetutorialspoint-cryptography, 或者在此查看流行的 AES 加密方案的技术描述

citeAES-encryption。此外, Bernstein 博士开发了一种称为 ChaCha 的加密方案

citeBernstein_{chacha, chacha - irtf Monero} *output keyimage*

9.2 地址的密钥聚合 (Key aggregation for addresses)

9.2.1 朴素方法 (Naive approach)

假设 N 个人想要创建一个群组多重签名地址, 我们将其表示为 $(K^{v,grp}, K^{s,grp})$ 。资金可以像发送到任何普通地址一样发送到该地址, 但正如我们稍后将看到的, 要花费这些资金, 所有 N 个人必须合作签署交易。

由于所有 N 个参与者都应该能够查看群组地址收到的资金, 我们可以让每个人都知道群组 view key $k^{v,grp}$ (回顾

² 截至撰写时, 我们了解到有三个 multisig 实现。第一个是使用 CLI (命令行界面) 的非常基础的手动流程 citecli-22multisig-instructions。第二个是真正优秀的 MMS (Multisig Messaging System), 它通过 CLI 实现安全、高度自动化的 multisig

citemms-manual, mms-project-proposal。第三个是商业化的 ‘Exa Wallet’, 其初始发布代码可在其 Github 仓库 url<https://github.com/exantech> 获取 (似乎未更新到当前发布版本)。这三个实现都依赖于同一个核心团队的代码库, 本质上意味着只有一个实现。

refsec:user-keys 和

refsec:one-time-addresses 节)。为了赋予所有参与者平等的权力，view key 可以是所有参与者安全地互相发送的 view key 分量之和。对于参与者 $e \in \{1, \dots, N\}$ ，其基础 view key 分量为 $k_e^{v,base} \in_R \mathbb{Z}_l$ ，所有参与者可以计算群组私有 view key

$$k^{v,grp} = \sum_{e=1}^N k_e^{v,base}$$

以类似的方式，群组 spend key $K^{s,grp} = k^{s,grp}G$ 可以是私有 spend key 基础分量之和。然而，如果有人知道所有的私有 spend key 分量，那么他就知道完整的私有 spend key。再加上私有 view key，他就可以独自签署交易。这就不是多重签名了，只是普通的签名。

相反，如果群组 spend key 是公钥 spend key 之和，我们可以达到同样的效果。假设参与者拥有基础公钥 spend key $K_e^{s,base}$ ，他们安全地互相发送这些密钥。现在让他们各自计算

$$K^{s,grp} = \sum_e K_e^{s,base}$$

显然这等同于

$$K^{s,grp} = \left(\sum_e k_e^{s,base} \right) * G$$

9.2.2 朴素方法的缺陷 (Drawbacks to the naive approach)

使用公钥 spend key 之和是直观且看似直接的，但会导致几个问题。

密钥聚合测试 (Key aggregation test)

知道所有基础公钥 spend key $K_e^{s,base}$ 的外部对手可以通过计算 $K^{s,grp} = \sum_e K_e^{s,base}$ 并检查 $K^s \stackrel{?}{=} K^{s,grp}$ 来轻而易举地测试给定公钥地址 (K^v, K^s) 是否为密钥聚合。这与一个更广泛的要求相关，即聚合密钥应与普通密钥不可区分，这样观察者就无法根据用户发布的地址类型获得对其活动的任何洞察。³

我们可以通过为每个多重签名地址创建新的基础 spend key，或通过遮蔽旧密钥来解决这个问题。前者很简单，但可能不太方便。

第二种方式如下：给定参与者 e 的旧密钥对 (K_e^v, K_e^s) ，其私钥为 (k_e^v, k_e^s) ，随机遮蔽值为 μ_e^v, μ_e^s ，⁴ 让他的群组地址新基础私钥分量为

$$\begin{aligned} k_e^{v,base} &= \mathcal{H}_n(k_e^v, \mu_e^v) \\ k_e^{s,base} &= \mathcal{H}_n(k_e^s, \mu_e^s) \end{aligned}$$

³ 如果至少一个诚实的参与者使用从均匀分布中随机选择的分量，那么通过简单求和聚合的密钥与普通密钥不可区分

citeSCOZZAFAVA1993313。

⁴ 随机遮蔽值可以很容易地从某个密码派生。例如， $\mu^s = \mathcal{H}_n(\text{password})$ 且 $\mu^v = \mathcal{H}_n(\mu^s)$ 。或者，如 Monero 中所做的那样，用一个字符串遮蔽 spend key 和 view key，例如 $\mu^s, \mu^v = \text{"Multisig"}$ 。这意味着 Monero 每个普通地址只支持一个 multisig 基础 spend key，尽管实际上将钱包设为 multisig 会导致用户失去对原始钱包的访问权限 citecli-22multisig-instructions。用户必须用他们的普通地址创建一个新钱包来访问其资金，假设 multisig 不是从一个全新的普通地址创建的。

如果参与者不希望观察者收集新密钥并测试密钥聚合，他们将需要安全地互相通信新密钥分量。⁵

如果密钥聚合测试不是问题，他们可以将其公钥基础分量 ($K_e^{v,base}, K_e^{s,base}$) 作为普通地址发布。任何第三方随后可以从这些单独的地址计算群组地址并向其发送资金，而无需与任何联合接收者交互

citemaxwell2018simple-musig。

密钥抵消 (Key cancellation)

如果群组 spend key 是公钥之和，一个不诚实的参与者如果事先得知其合作者的 spend key 基础分量，就可以抵消它们。

例如，假设 Alice 和 Bob 想要创建一个群组地址。Alice 出于善意，告诉 Bob 她的密钥分量 ($k_A^{v,base}, K_A^{s,base}$)。Bob 私下生成他的密钥分量 ($k_B^{v,base}, K_B^{s,base}$) 但没有立即告诉 Alice。相反，他计算 $K_B^{ts,base} = K_B^{s,base} - K_A^{s,base}$ 并告诉 Alice ($k_B^{v,base}, K_B^{ts,base}$)。群组地址是：

$$\begin{aligned} K^{v,grp} &= (k_A^{v,base} + k_B^{v,base})G \\ &= k^{v,grp}G \\ K^{s,grp} &= K_A^{s,base} + K_B^{ts,base} \\ &= K_A^{s,base} + (K_B^{s,base} - K_A^{s,base}) \\ &= K_B^{s,base} \end{aligned}$$

这留下了一个群组地址 ($k^{v,grp}G, K_B^{s,base}$)，其中 Alice 知道私有群组 view key，而 Bob 同时知道私有 view key

em 和私有 spend key！Bob 可以独自签署交易，欺骗 Alice，而 Alice 可能会相信发送到该地址的资金只有在她的许可下才能被花费。

我们可以通过要求每个参与者在聚合密钥之前，制作一个签名来证明他们知道其 spend key 分量的私钥来解决这个问题

citeold-multisig-mrl-note。⁶ 这既不方便又容易出现实现错误。幸运的是，有一个可靠的替代方案可用。

9.2.3 鲁棒密钥聚合 (Robust key aggregation)

为了轻松抵抗密钥抵消，我们对 spend key 聚合做了一个小改动 (view key 聚合保持不变)。设 N 个签名者的基础 spend key 分量集合为 $\mathbb{S}^{base} = \{K_1^{s,base}, \dots, K_N^{s,base}\}$ ，按照某种约定排序 (例如从小到大，即字典序)。⁷ 鲁棒聚合 spend key 为

⁵ 正如我们将在

refsec:smaller-thresholds 节中看到的，当 $M < N$ 时，由于共享秘密的存在，密钥聚合不适用于 M-of-N multisig。

⁶ Monero 当前 (也是第一个) 的 multisig 迭代于 2018 年 4 月发布

citelithiumluna-v7 (M-of-N 集成随后在 2018 年 10 月跟进

citeberylliumbullet-v8)，使用了这种朴素的密钥聚合方法，并要求用户对其 spend key 分量进行签名。

⁷ $\mathbb{S}^{base}$ 需要一致排序，以确保参与者确信他们在 hash 相同的内容。

citeMRL-0009-multisig^{8,9}

$$K^{s,grp} = \sum_e \mathcal{H}_n(T_{agg}, \mathbb{S}^{base}, K_e^{s,base}) K_e^{s,base}$$

现在如果 Bob 试图抵消 Alice 的 spend key，他将陷入一个非常困难的问题。

$$\begin{aligned} K^{s,grp} &= \mathcal{H}_n(T_{agg}, \mathbb{S}, K_A^s) K_A^s + \mathcal{H}_n(T_{agg}, \mathbb{S}, K_B'^s) K_B'^s \\ &= \mathcal{H}_n(T_{agg}, \mathbb{S}, K_A^s) K_A^s + \mathcal{H}_n(T_{agg}, \mathbb{S}, K_B'^s) K_B^s - \mathcal{H}_n(T_{agg}, \mathbb{S}, K_B'^s) K_A^s \\ &= [\mathcal{H}_n(T_{agg}, \mathbb{S}, K_A^s) - \mathcal{H}_n(T_{agg}, \mathbb{S}, K_B'^s)] K_A^s + \mathcal{H}_n(T_{agg}, \mathbb{S}, K_B'^s) K_B^s \end{aligned}$$

Bob 的沮丧就留给读者自行想象了。

与朴素方法一样，任何知道 $\mathbb{S}^{base}$ 和相应公钥 view key 的第三方都可以计算群组地址。

由于参与者不需要证明他们知道自己的私有 spend key，或者在签署交易之前进行任何交互，我们的鲁棒密钥聚合满足所谓的

em 纯公钥模型，即“唯一的要求是每个潜在签名者拥有一个公钥”[?]。¹⁰

premerge 和 merge 函数

更正式地说，为了后续清晰起见，我们可以说存在一个 premerge 操作，它接收一组基础密钥 $\mathbb{S}^{base}$ ，并输出一组大小相等的聚合密钥 $\mathbb{K}^{agg}$ ，其中元素¹¹

$$\mathbb{K}^{agg}[e] = \mathcal{H}_n(T_{agg}, \mathbb{S}^{base}, K_e^{s,base}) K_e^{s,base}$$

聚合私钥 k_e^{agg} 用于群组签名。¹²

还有另一个 merge 操作，它接收来自 premerge 的聚合并构造群组签名密钥（例如 Monero 的 spend key）

$$K^{grp} = \sum_e \mathbb{K}^{agg}[e]$$

我们在

refsec:n-1-of-n 节中将这些函数推广到 (N-1)-of-N 和 M-of-N，并在

refsubsec:nesting-multisig-keys 节中进一步推广到嵌套 multisig。

⁸ 回顾

refsec:CLSAG 节，hash 函数应通过添加标签前缀进行域分离，例如 T_{agg} = “Multisig_Aggregation”。我们在下一节的 Schnorr 签名等示例中省略了标签。

⁹ 在聚合 hash 中包含 $\mathbb{S}^{base}$ 很重要，以避免涉及 Wagner 对生日问题的广义解 citegeneralized-birthday-wagner 的复杂密钥抵消攻击。

citadam-wagnerian-tragedies

citemaxwell2018simple-musig

¹⁰ 正如我们稍后将看到的，密钥聚合仅对 N-of-N 和 1-of-N multisig 满足纯公钥模型。

¹¹ 记号： $\mathbb{K}^{agg}[e]$ 是集合中的第 e 个元素。

¹² 鲁棒密钥聚合尚未在 Monero 中实现，但由于参与者可以存储和使用私钥 k_e^{agg} （对于朴素密钥聚合， $k_e^{agg} = k_e^{base}$ ），将 Monero 更新为使用鲁棒密钥聚合只会改变 premerge 过程。

9.3 阈值 Schnorr 类签名 (Thresholded Schnorr-like signatures)

多重签名需要一定数量的签名者才能工作，因此我们说存在一个“阈值”，低于该阈值就无法产生签名。一个有 N 个参与者且需要所有 N 个人来构建签名的多重签名，通常称为 m of N multisig，其阈值为 N 。稍后我们将将其扩展到 M -of- N ($M \leq N$) multisig，其中 N 个参与者创建群组地址，但只需要 M 个人来制作签名。

让我们暂时从 Monero 中退一步。本文档中的所有签名方案都源自 Maurer 的一般零知识证明 [citesimple-zk-proof-maurer](#)，因此我们可以使用简单的 Schnorr 类签名来演示阈值签名的基本形式 (回顾

[refsec:signing-messages](#) 节)

[citeold-multisig-mrl-note](#)。

签名 (Signature)

假设有 N 个人，每人拥有集合 $\mathbb{K}^{agg}$ 中的一个公钥，其中每个人 $e \in \{1, \dots, N\}$ 知道私钥 k_e^{agg} 。他们的 N -of- N 群组公钥（将用于签署消息）为 K^{grp} 。假设他们想联合签署一条消息 $\mathbf{m}$ 。他们可以像这样协作完成一个基本的 Schnorr 类签名

1. 每个参与者 $e \in \{1, \dots, N\}$ 执行以下操作：
 - (a) 选择随机分量 $\alpha_e \in_R \mathbb{Z}_l$,
 - (b) 计算 $\alpha_e G$
 - (c) 用 $C_e^\alpha = \mathcal{H}_n(T_{com}, \alpha_e G)$ 对其进行承诺,
 - (d) 并安全地将 C_e^α 发送给其他参与者。
2. 一旦收集到所有承诺 C_e^α ，每个参与者安全地将其 $\alpha_e G$ 发送给其他参与者。他们必须验证所有其他参与者的 $C_e^\alpha \stackrel{?}{=} \mathcal{H}_n(T_{com}, \alpha_e G)$ 。

3. 每个参与者计算

$$\alpha G = \sum_e \alpha_e G$$

4. 每个参与者 $e \in \{1, \dots, N\}$ 执行以下操作：¹³

- (a) 计算挑战 $c = \mathcal{H}_n(\mathbf{m}, [\alpha G])$,
- (b) 定义其响应分量 $r_e = \alpha_e - c * k_e^{agg} \pmod{l}$,
- (c) 并安全地将 r_e 发送给其他参与者。

5. 每个参与者计算

$$r = \sum_e r_e$$

6. 任何参与者都可以发布签名 $\sigma(\mathbf{m}) = (c, r)$ 。

¹³ 如

[refsec:schnorr-fiat-shamir](#) 节所述，不要对不同的挑战 c 重复使用 α_e 。这意味着要重置已发送响应的多重签名流程，应从头开始使用新的 α_e 值。

验证 (Verification)

给定 K^{grp} 、 $\mathbf{m}$ 和 $\sigma(\mathbf{m}) = (c, r)$:

1. 计算挑战 $c' = \mathcal{H}_n(\mathbf{m}, [rG + c * K^{grp}])$ 。
2. 如果 $c = c'$ ，则签名是合法的（除了可忽略的概率外）。

我们为了清晰起见加上了上标 grp ，但实际上验证者无法判断 K^{grp} 是一个合并密钥，除非参与者告诉他，或者他知道基础或聚合密钥分量。

为什么有效 (Why it works)

响应 r 是此签名的核心。参与者 e 在 r_e 中知道两个秘密 (α_e 和 k_e^{agg})，因此他的私钥 k_e^{agg} 对其他参与者来说是信息论安全的（假设他从不重复使用 α_e ）。此外，验证者使用群组公钥 K^{grp} ，因此需要所有密钥分量来构建签名。

$$\begin{aligned}
 rG &= \left(\sum_e r_e \right) G \\
 &= \left(\sum_e (\alpha_e - c * k_e^{agg}) \right) G \\
 &= \left(\sum_e \alpha_e \right) G - c * \left(\sum_e k_e^{agg} \right) G \\
 &= \alpha G - c * K^{grp} \\
 \alpha G &= rG + c * K^{grp} \\
 \mathcal{H}_n(\mathbf{m}, [\alpha G]) &= \mathcal{H}_n(\mathbf{m}, [rG + c * K^{grp}]) \\
 c &= c'
 \end{aligned}$$

额外的提交-揭示步骤 (Additional commit-and-reveal step)

读者可能想知道步骤 2 是从哪里来的。没有提交-揭示 (commit-and-reveal)

[citeMRL-0009-multisig](#)，恶意共同签名者可以在挑战计算

$\mathbf{em}$ 之前了解到所有 $\alpha_e G$ 。这使他能够在一定程度上控制产生的挑战，方法是在发送之前修改自己的 $\alpha_e G$ 。他可以使用从多个受控签名中收集的响应分量，在亚指数时间内推导出其他签名者的私钥 k_e^{agg}

[citecryptoeprint:2018:417](#)，这是一个严重的安全威胁。此威胁依赖于 Wagner 对生日问题

[citebirthday-problem](#) 的广义化

[citegeneralized-birthday-wagner](#)（另见

[citeadam-wagnerian-tragedies](#) 获得更直观的解释）。¹⁴

¹⁴ 提交-揭示未在 Monero multisig 的当前实现中使用，尽管正在为未来版本考虑。
[citemultisig-research-issue-67](#)

9.4 Monero 的 MLSTAG 环保密签名 (MLSTAG Ring Confidential signatures for Monero)

Monero 阈值环保密交易增加了一些复杂性, 因为 MLSTAG (阈值化 MLSAG) 签名密钥是一次性地址和对零的承诺 (用于 input 金额)。

回顾

refsec:multi_{output}ransactions t output (K_t^v, K_t^s) 的人的一次性地址如下:

$$\begin{aligned} K_t^o &= \mathcal{H}_n(rK_t^v, t)G + K_t^s = (\mathcal{H}_n(rK_t^v, t) + k_t^s)G \\ k_t^o &= \mathcal{H}_n(rK_t^v, t) + k_t^s \end{aligned}$$

我们可以更新群组地址 $(K_t^{v,grp}, K_t^{s,grp})$ 接收的 output 的记号:

$$\begin{aligned} K_t^{o,grp} &= \mathcal{H}_n(rK_t^{v,grp}, t)G + K_t^{s,grp} \\ k_t^{o,grp} &= \mathcal{H}_n(rK_t^{v,grp}, t) + k_t^{s,grp} \end{aligned}$$

与之前一样, 任何拥有 $k_t^{v,grp}$ 和 $K_t^{s,grp}$ 的人都可以发现 $K_t^{o,grp}$ 是其地址拥有的 output, 并可以解码 output 金额的 Diffie-Hellman 项并重建相应的承诺遮蔽值 (

refsec:pedersen_{monero}

这也意味着 multisig 子地址是可行的 (

refsec:subaddresses 节)。使用发送到子地址的资金进行 multisig 交易需要对以下算法进行一些相当直接的修改, 我们在脚注中提及。¹⁵

9.4.1 N-of-N multisig 的 RCTTypeBulletproof2

multisig 交易的大部分可以由发起者完成。只有 MLSTAG 签名需要协作。发起者应执行以下操作来准备 RCTTypeBulletproof2 交易 (回顾

refsec:RCTTypeBulletproof2 节):

1. 生成交易私钥 $r \in_R \mathbb{Z}_l$ (refsec:one-time-addresses 节) 并计算相应的公钥 rG (如果处理子地址接收者则生成多个此类密钥; refsec:subaddresses 节)。
2. 决定要花费的 input ($j \in \{1, \dots, m\}$ 个拥有的 output, 一次性地址为 $K_j^{o,grp}$, 金额为 $a_1, \dots, a_m$), 以及接收资金的接收者 ($t \in \{0, \dots, p-1\}$ 个新 output, 金额为 $b_0, \dots, b_{p-1}$, 一次性地址为 K_t^o)。这包括矿工费 f 及其承诺 fH 。决定每个 input 的诱饵环成员集合。
3. 编码每个 output 的金额 $amount_t$ (refsec:pedersen_{monero} output C_t^b 。

¹⁵ Monero 支持 multisig 子地址。

4. 为每个 input $j \in \{1, \dots, m-1\}$ 选择伪 output 承诺遮蔽分量 $x'_j \in_R \mathbb{Z}_l$, 并按如下方式计算第 m 个遮蔽值 (refsec:ringct-introduction 节)

$$x'_m = \sum_t y_t - \sum_{j=1}^{m-1} x'_j$$

计算伪 output 承诺 C_j^a 。

5. 为所有 output 生成聚合 Bulletproof 范围证明。回顾 refsec:rangeproofs $\alpha_j^z \in_R \mathbb{Z}_l$ 并计算 $\alpha_j^z G$ 来准备 MLSTAG 签名。¹⁶

他将所有这些信息安全地发送给其他参与者。现在签名者组已经准备好用他们的私钥 $k_e^{s,agg}$ 和对零的承诺 $C_{\pi,j}^a - C_{\pi,j}^a = z_j G$ 来构建 input 签名。

MLSTAG RingCT

首先, 他们为所有 input $j \in \{1, \dots, m\}$ 构建群组 key image, 一次性地址为 $K_{\pi,j}^{o,grp}$ 。¹⁷

6. 对于每个 input j , 每个参与者 e 执行以下操作:

- 计算部分 key image $\tilde{K}_{j,e}^o = k_e^{s,agg} \mathcal{H}_p(K_{\pi,j}^{o,grp})$,
- 并安全地将 $\tilde{K}_{j,e}^o$ 发送给其他参与者。

2. 每个参与者现在可以计算, 其中 u_j 是交易中 $K_{\pi,j}^{o,grp}$ 被发送到 multisig 地址的 output 索引,¹⁸

$$\tilde{K}_j^{o,grp} = \mathcal{H}_n(k^{v,grp} r G, u_j) \mathcal{H}_p(K_{\pi,j}^{o,grp}) + \sum_e \tilde{K}_{j,e}^o$$

然后他们为每个 input j 构建 MLSTAG 签名。

1. 每个参与者 e 执行以下操作:

- 生成种子分量 $\alpha_{j,e} \in_R \mathbb{Z}_l$ 并计算 $\alpha_{j,e} G$ 和 $\alpha_{j,e} \mathcal{H}_p(K_{\pi,j}^{o,grp})$,
- 为 $i \in \{1, \dots, v+1\}$ ($i = \pi$ 除外) 生成随机分量 $r_{i,j,e}$ 和 $r_{i,j,e}^z$,

¹⁶ 不需要对这些进行提交-揭示, 因为对零的承诺是所有签名者都知道的。

¹⁷ 如果 $K_{\pi,j}^{o,grp}$ 是从 i 索引的 multisig 子地址构建的, 那么 (来自 refsec:subaddresses 节) 其私钥是一个复合值:

$$k_{\pi,j}^{o,grp} = \mathcal{H}_n(k^{v,grp} r_{u_j} K^{s,grp,i}, u_j) + \sum_e k_e^{s,agg} + \mathcal{H}_n(k^{v,grp}, i)$$

¹⁸ 如果一次性地址对应于 i 索引的 multisig 子地址, 则添加

$$\tilde{K}_j^{o,grp} = \dots + \mathcal{H}_n(k^{v,grp}, i) \mathcal{H}_p(K_{\pi,j}^{o,grp})$$

(c) 计算承诺

$$C_{j,e}^\alpha = \mathcal{H}_n(T_{com}, \alpha_{j,e}G, \alpha_{j,e}\mathcal{H}_p(K_{\pi,j}^{o,grp}), r_{1,j,e}, \dots, r_{v+1,j,e}, r_{1,j,e}^z, \dots, r_{v+1,j,e}^z)$$

(d) 并安全地将 $C_{j,e}^\alpha$ 发送给其他参与者。

2. 收到其他参与者的所有 $C_{j,e}^\alpha$ 后, 发送所有 $\alpha_{j,e}G$ 、 $\alpha_{j,e}\mathcal{H}_p(K_{\pi,j}^{o,grp})$ 和 $r_{i,j,e}$ 以及 $r_{i,j,e}^z$, 并验证每个参与者的原始承诺是否有效。

3. 每个参与者可以计算所有

$$\begin{aligned}\alpha_j G &= \sum_e \alpha_{j,e} G \\ \alpha_j \mathcal{H}_p(K_{\pi,j}^{o,grp}) &= \sum_e \alpha_{j,e} \mathcal{H}_p(K_{\pi,j}^{o,grp}) \\ r_{i,j} &= \sum_e r_{i,j,e} \\ r_{i,j}^z &= \sum_e r_{i,j,e}^z\end{aligned}$$

4. 每个参与者可以构建签名环 (参见 refsec:MLSAG 节)。

5. 为了完成签名闭环, 每个参与者 e 执行以下操作:

- (a) 定义 $r_{\pi,j,e} = \alpha_{j,e} - c_\pi k_e^{s,agg} \pmod{l}$,
- (b) 并安全地将 $r_{\pi,j,e}$ 发送给其他参与者。

6. 每个人可以计算 (回忆 $\alpha_{j,e}^z$ 是由发起者创建的) ¹⁹

$$\begin{aligned}r_{\pi,j} &= \sum_e r_{\pi,j,e} - c_\pi * \mathcal{H}_n(k^{v,grp}rG, u_j) \\ r_{\pi,j}^z &= \alpha_{j,e}^z - c_\pi z_j \pmod{l}\end{aligned}$$

input j 的签名为 $\sigma_j(\mathbf{m}) = (c_1, r_{1,j}, r_{1,j}^z, \dots, r_{v+1,j}, r_{v+1,j}^z)$, 附带 $\tilde{K}_j^{o,grp}$ 。

由于在 Monero 中消息 $\mathbf{m}$ 和挑战 c_π 依赖于交易的所有其他部分, 任何试图在发送错误值给其同伴签名者的过程中作弊的参与者, 都会导致签名失败。响应 $r_{\pi,j}$ 仅对其定义时的 $\mathbf{m}$ 和 c_π 有用。

9.4.2 简化通信 (Simplified communication)

构建多重签名 Monero 交易需要很多步骤。我们可以重组和简化其中一些, 使签名者交互被包含在两个部分共五轮中。

¹⁹ 如果一次性地址 $K_{\pi,j}^{o,grp}$ 对应于 i 索引的 multisig 子地址, 则包含

$$r_{\pi,j} = \dots - c_\pi * \mathcal{H}_n(k^{v,grp}, i)$$

1. it 多重签名公钥地址的密钥聚合：任何拥有一组公钥地址的人都可以对其运行 `premerge`，然后 `merge` 一个 N-of-N 地址，但除非参与者了解所有分量，否则不会知道群组 view key，因此群组首先安全地互相发送 k_e^v 和 $K_e^{s,base}$ 。任何参与者都可以 `premerge` 和 `merge` 并发布 $(K^{v,grp}, K^{s,grp})$ ，允许群组向群组地址接收资金。M-of-N 聚合需要更多步骤，我们在 `refsec:smaller-thresholds` 节中描述。
2. it 交易：

- (a) 某个参与者或子联盟（发起者）决定撰写一笔交易。他们选择 m 个 input，一次性地址为 $K_j^{o,grp}$ ，金额承诺为 C_j^a ， m 组 v 个额外的一次性地址和承诺用作环诱饵，选择 p 个 output 接收者，公钥地址为 (K_t^v, K_t^s) ，向他们发送的金额为 b_t ，决定交易费 f ，选择交易私钥 r ，²⁰生成伪 output 承诺遮蔽值 x'_j ($j \neq m$)，为每个 output 构造 ECDH 项 $amount_t$ ，生成聚合范围证明，并为所有 input 的对零承诺生成签名开放值 α_j^z ，以及随机标量 $r_{i,j}$ 和 $r_{i,j}^z$ ($i \neq \pi_j$)。²¹他们还为一轮通信准备自己的贡献。

发起者将所有信息安全地发送给其他参与者。²²其他参与者可以通过发送下一轮通信的部分来表示同意，或协商修改。

- (b) 每个参与者选择其 MLSTAG 签名的开放分量，对其进行承诺，计算其部分 key image，并安全地将这些承诺和部分 image 发送给其他参与者。

MLSTAG 签名：key image $\tilde{K}_{j,e}^o$ ，签名随机性 $\alpha_{j,e}G$ 和 $\alpha_{j,e}\mathcal{H}_p(K_{\pi,j}^{o,grp})$ 。部分 key image 不需要包含在承诺数据中，因为它们不能用于提取签名者的私钥。它们对于查看哪些已拥有的 output 已被花费也很有用，因此出于模块化设计的考虑应单独处理。

- (c) 收到所有签名承诺后，每个参与者安全地将承诺的信息发送给其他参与者。
- (d) 每个参与者完成其 MLSTAG 签名的部分，安全地将所有 $r_{\pi_j,j,e}$ 发送给其他参与者。²³

假设流程顺利，所有参与者都可以自行完成交易的撰写并广播。由多重签名联盟创建的交易与个人创建的交易不可区分。

9.5 重新计算 key image (Recalculating key images)

如果有人丢失了记录并想要计算其地址的余额（收到的资金减去花费的资金），他们需要检查 blockchain。View key 仅对读取收到的资金有用，因此用户需要为所有拥有的 output 计算 key

²⁰ 或者如果发送到至少一个子地址，则为交易私钥 r_t 。

²¹ 注意，我们通过让发起者生成随机标量 $r_{i,j}$ 和 $r_{i,j}^z$ 来简化签名过程，而不是让每个共同签名者生成最终被求和在一起的分量。

²² 他不需要直接发送 output 金额 b_t ，因为它们可以从 $amount_t$ 计算出来。Monero 采用了合理的方法：创建一个由发起者选择的信息填充的部分交易，并将其与其他相关信息（如交易私钥、目标地址、真实 input 等）一起发送给其他共同签名者。

²³ 签名者的每次签名尝试都必须使用唯一的 $\alpha_{j,e}$ ，以避免向其他签名者泄露其私有 spend key（回顾 `refsec: schnorr-fiat-shamir` 节）
citeMRL-0009-multisig。钱包应通过在使用它的响应传输到钱包外部时始终删除 $\alpha_{j,e}$ 来从根本上强制执行此操作。

image, 通过与 blockchain 中存储的 key image 进行比较来查看它们是否已被花费。由于群组地址的成员无法自行计算 key image, 他们需要寻求其他参与者的帮助。

从简单的分量之和计算 key image 可能会因为不诚实的参与者报告虚假密钥而失败。²⁴ 给定一个一次性地址为 $K^{o,grp}$ 的已接收 output, 群组可以生成一个简单的”可链接”Schnorr 类证明 (基本上是单密钥 bLSTAG, 回顾

refsec:proofs-discrete-logarithm-multiple-bases 和

refblsag_{note} $keyimage^{o,grp}$ 是合法的, 而无需透露其私有 spend key 分量或需要彼此信任。

证明 (Proof)

1. 每个参与者 e 执行以下操作:

- 计算 $\tilde{K}_e^o = k_e^{s,agg} \mathcal{H}_p(K^{o,grp})$,
- 生成种子分量 $\alpha_e \in_R \mathbb{Z}_l$ 并计算 $\alpha_e G$ 和 $\alpha_e \mathcal{H}_p(K^{o,grp})$,
- 用 $C_e^\alpha = \mathcal{H}_n(T_{com}, \alpha_e G, \alpha_e \mathcal{H}_p(K^{o,grp}))$ 对数据进行承诺,
- 并安全地将 C_e^α 和 $\tilde{K}_e^o$ 发送给其他参与者。

2. 收到所有 C_e^α 后, 每个参与者发送承诺的信息并验证其他参与者的承诺是否合法。

3. 每个参与者可以计算:²⁵

$$\begin{aligned}\tilde{K}^{o,grp} &= \mathcal{H}_n(k^{v,grp} r G, u) \mathcal{H}_p(K^{o,grp}) + \sum_e \tilde{K}_e^o \\ \alpha G &= \sum_e \alpha_e G \\ \alpha \mathcal{H}_p(K^{o,grp}) &= \sum_e \alpha_e \mathcal{H}_p(K^{o,grp})\end{aligned}$$

4. 每个参与者计算挑战²⁶

$$c = \mathcal{H}_n([\alpha G], [\alpha \mathcal{H}_p(K^{o,grp})])$$

5. 每个参与者执行以下操作:

- 定义 $r_e = \alpha_e - c * k_e^{s,agg} \pmod{l}$,
- 并安全地将 r_e 发送给其他参与者。

6. 每个参与者可以计算²⁷

²⁴ 目前 Monero 似乎使用简单求和。

²⁵ 如果一次性地址对应于 i 索引的 multisig 子地址, 则添加

$$\tilde{K}^{o,grp} = \dots + \mathcal{H}_n(k^{v,grp}, i) \mathcal{H}_p(K^{o,grp})$$

²⁶ 此证明应包括域分离和密钥前缀, 为简洁起见我们省略。

²⁷ 如果一次性地址 $K^{o,grp}$ 对应于 i 索引的 multisig 子地址, 则包含

$$r^{resp} = \dots - c * \mathcal{H}_n(k^{v,grp}, i)$$

$$r^{resp} = \sum_e r_e - c * \mathcal{H}_n(k^{v,grp} rG, u)$$

证明为 (c, r^{resp}) ，附带 $\tilde{K}^{o,grp}$ 。

验证 (Verification)

1. 检查 $l\tilde{K}^{o,grp} \stackrel{?}{=} 0$ 。
2. 计算 $c' = \mathcal{H}_n([r^{resp}G + c * K^{o,grp}], [r^{resp}\mathcal{H}_p(K^{o,grp}) + c * \tilde{K}^{o,grp}])$ 。
3. 如果 $c = c'$ ，则 key image $\tilde{K}^{o,grp}$ 对应于一次性地址 $K^{o,grp}$ （除了可忽略的概率外）。

9.6 更小的阈值 (Smaller thresholds)

在本章开头，我们讨论了托管服务，它使用 2-of-2 multisig 在用户和安全公司之间分配签名权。这种设置并不理想，因为如果安全公司被入侵或拒绝合作，你的资金可能会被冻结。

我们可以通过 2-of-3 multisig 地址来解决这种可能性，它有三个参与者但只需要其中两个来签署交易。托管服务提供一个密钥，用户提供两个密钥。用户可以将一个密钥存储在安全的地方（如保险箱），并将另一个用于日常购买。如果托管服务失败，用户可以使用安全密钥和日常密钥提取资金。

具有低于 N 阈值的多重签名有广泛的用途。

9.6.1 1-of-N 密钥聚合 (1-of-N key aggregation)

假设一群人想要创建一个他们都可以签署的 multisig 密钥 K^{grp} 。解决方案是微不足道的：让每个人都知道私钥 k^{grp} 。有三种方法可以做到这一点。

1. 一个参与者或子联盟选择一个密钥并安全地将其发送给其他所有人。
2. 所有参与者计算私钥分量并安全地发送它们，使用简单求和作为合并密钥。²⁸
3. 参与者将 M-of-N multisig 扩展到 1-of-N。如果对手可以访问群组的通信，这可能有用。

在这种情况下，对于 Monero，每个人都会知道私钥 $(k^{v,grp,1xN}, k^{s,grp,1xN})$ 。在本节之前，所有群组密钥都是 N-of-N 的，但现在我们使用上标 1xN 来表示与 1-of-N 签名相关的密钥。

²⁸ 注意，密钥抵消在这里基本上没有意义，因为每个人都知道完整的私钥。

9.6.2 (N-1)-of-N 密钥聚合 ((N-1)-of-N key aggregation)

在 (N-1)-of-N 群组密钥中，如 2-of-3 或 4-of-5，任何 (N-1) 个参与者的集合都可以签名。我们通过 Diffie-Hellman 共享秘密来实现这一点。假设参与者 $e \in \{1, \dots, N\}$ ，拥有基础公钥 K_e^{base} ，他们都知道这些密钥。

每个参与者 e 对于 $w \in \{1, \dots, N\}$ 但 $w \neq e$ 计算：

$$k_{e,w}^{sh,(N-1) \times N} = \mathcal{H}_n(k_e^{base} K_w^{base})$$

然后他计算所有 $K_{e,w}^{sh,(N-1) \times N} = k_{e,w}^{sh,(N-1) \times N} G$ 并安全地将它们发送给其他参与者。我们现在使用上标 sh 来表示由参与者子群共享的密钥。

每个参与者将有 (N-1) 个共享私钥分量，对应于其他每个参与者，总共有 $N \times (N-1)$ 个密钥。所有密钥由两个 Diffie-Hellman 伙伴共享，因此实际上有 $[N \times (N-1)]/2$ 个唯一密钥。这些唯一密钥组成集合 $\mathbb{S}^{(N-1) \times N}$ 。

推广 premerge 和 merge

这是我们更新

refsec:robust-key-aggregation 节中 **premerge** 定义的地方。其输入将是集合 $\mathbb{S}^{M \times N}$ ，其中 M 是该集合的密钥所准备的“阈值”。当 $M = N$ 时， $\mathbb{S}^{N \times N} = \mathbb{S}^{base}$ ，当 $M < N$ 时它包含共享密钥。输出是 $\mathbb{K}^{agg, M \times N}$ 。

$\mathbb{K}^{agg, (N-1) \times N}$ 中的 $[N \times (N-1)]/2$ 个密钥分量可以送入 **merge**，输出 $K^{grp, (N-1) \times N}$ 。重要的是，所有 $[N \times (N-1)]/2$ 个私钥分量只需 (N-1) 个参与者即可组装，因为每个参与者与第 N 个人共享一个 Diffie-Hellman 秘密。

2-of-3 示例

假设有三个人，公钥为 $\{K_1^{base}, K_2^{base}, K_3^{base}\}$ ，他们各自知道一个私钥，想要创建一个 2-of-3 multisig 密钥。在 Diffie-Hellman 交换和互相发送公钥之后，他们各自知道以下内容：

1. 人物 1: $k_{1,2}^{sh,2 \times 3}$, $k_{1,3}^{sh,2 \times 3}$, $K_{2,3}^{sh,2 \times 3}$
2. 人物 2: $k_{2,1}^{sh,2 \times 3}$, $k_{2,3}^{sh,2 \times 3}$, $K_{1,3}^{sh,2 \times 3}$
3. 人物 3: $k_{3,1}^{sh,2 \times 3}$, $k_{3,2}^{sh,2 \times 3}$, $K_{1,2}^{sh,2 \times 3}$

其中 $k_{1,2}^{sh,2 \times 3} = k_{2,1}^{sh,2 \times 3}$ ，依此类推。集合 $\mathbb{S}^{2 \times 3} = \{K_{1,2}^{sh,2 \times 3}, K_{1,3}^{sh,2 \times 3}, K_{2,3}^{sh,2 \times 3}\}$ 。

执行 **premerge** 和 **merge** 创建群组密钥：²⁹

²⁹ 由于合并密钥由共享秘密组成，仅知道原始基础公钥的观察者将无法聚合它们（refsubsec:drawbacks-naive-aggregation-cancellation 节）并识别合并密钥的成员。

$$K^{grp,2x3} = \mathcal{H}_n(T_{agg}, \mathbb{S}^{2x3}, K_{1,2}^{sh,2x3}) K_{1,2}^{sh,2x3} + \\ \mathcal{H}_n(T_{agg}, \mathbb{S}^{2x3}, K_{1,3}^{sh,2x3}) K_{1,3}^{sh,2x3} + \\ \mathcal{H}_n(T_{agg}, \mathbb{S}^{2x3}, K_{2,3}^{sh,2x3}) K_{2,3}^{sh,2x3}$$

现在假设人物 1 和 2 想要签署一条消息 $\mathbf{m}$ 。我们将使用基本的 Schnorr 类签名来演示。

1. 每个参与者 $e \in \{1, 2\}$ 执行以下操作：

- (a) 选择随机分量 $\alpha_e \in_R \mathbb{Z}_l$,
- (b) 计算 $\alpha_e G$,
- (c) 用 $C_e^\alpha = \mathcal{H}_n(T_{com}, \alpha_e G)$ 进行承诺,
- (d) 并安全地将 C_e^α 发送给其他参与者。

2. 收到所有 C_e^α 后, 每个参与者发出 $\alpha_e G$ 并验证其他承诺是否合法。

3. 每个参与者计算

$$\alpha G = \sum_e \alpha_e G \\ c = \mathcal{H}_n(\mathbf{m}, [\alpha G])$$

4. 人物 1 执行以下操作：

- (a) 计算 $r_1 = \alpha_1 - c * [k_{1,3}^{agg,2x3} + k_{1,2}^{agg,2x3}]$,
- (b) 并安全地将 r_1 发送给人物 2。

5. 人物 2 执行以下操作：

- (a) 计算 $r_2 = \alpha_2 - c * k_{2,3}^{agg,2x3}$,
- (b) 并安全地将 r_2 发送给人物 1。

6. 每个参与者计算

$$r = \sum_e r_e$$

7. 任一参与者都可以发布签名 $\sigma(\mathbf{m}) = (c, r)$ 。

子阈值签名唯一的变化是如何通过定义 $r_{\pi,e}$ 来”闭环”(在环签名的情况下, 秘密索引为 π)。每个参与者必须包含其对应于”缺失者”的共享秘密, 但由于所有其他共享秘密都是成对出现的, 因此有一个技巧。给定所有参与者原始密钥的集合 $\mathbb{S}^{base}$, 只有

em 第一个人——按照 $\mathbb{S}^{base}$ 中的索引排序——拥有某共享秘密副本的人才使用它来计算他的 $r_{\pi,e}$ 。^{30,31}

³⁰ 在实践中, 这意味着发起者应确定哪些 M 个签名者的子集将签署给定消息。如果他发现 O 个签名者愿意签名 ($O \geq M$), 他可以在 O 中的每个 M 大小子集上协调多个并发签名尝试, 以增加成功的可能性。Monero 似乎使用了这种方法。事实还表明(一个深奥的点), 发起者创建的 em 原始输出目标列表是随机打乱的, 该随机列表随后被所有并发签名尝试和所有其他共同签名者使用(这与模糊的 `shuffle_outs` 标志相关)。

³¹ 目前 Monero 似乎使用轮询签名方法, 其中发起者用他的所有私钥签名, 将部分签名的交易传递给另一个签名者, 后者用他的所有 em 尚未用于签名的私钥签名, 然后传递给再一个签名者, 依此类推, 直到最终签名者可以发布交易或将其发送给其他签名者以便他们发布。

在前面的例子中，人物 1 计算：

$$r_1 = \alpha_1 - c * [k_{1,3}^{agg,2x3} + k_{1,2}^{agg,2x3}]$$

而人物 2 只计算

$$r_2 = \alpha_2 - c * k_{2,3}^{agg,2x3}$$

同样的原则适用于在子阈值 Monero multisig 交易中计算群组 key image。

9.6.3 M-of-N 密钥聚合 (M-of-N key aggregation)

我们可以通过调整对 (N-1)-of-N 的视角来理解 M-of-N。在 (N-1)-of-N 中，两个公钥之间的每个共享秘密，例如 K_1^{base} 和 K_2^{base} ，包含两个私钥 $k_1^{base} k_2^{base} G$ 。它是一个秘密，因为只有人物 1 能计算 $k_1^{base} K_2^{base}$ ，只有人物 2 能计算 $k_2^{base} K_1^{base}$ 。

如果存在第三个人拥有 K_3^{base} ，存在共享秘密 $k_1^{base} k_2^{base} G$ 、 $k_1^{base} k_3^{base} G$ 和 $k_2^{base} k_3^{base} G$ ，并且参与者将这些公钥互相发送（使它们不再是秘密）会怎样？他们各自为两个公钥贡献了一个私钥。现在假设他们用第三个公钥创建一个新的共享秘密。

人物 1 计算共享秘密 $k_1^{base} * (k_2^{base} k_3^{base} G)$ ，人物 2 计算 $k_2^{base} * (k_1^{base} k_3^{base} G)$ ，人物 3 计算 $k_3^{base} * (k_1^{base} k_2^{base} G)$ 。现在他们都知道 $k_1^{base} k_2^{base} k_3^{base} G$ ，形成了三方共享秘密（只要没有人发布它）。

群组可以使用 $k^{sh,1x3} = \mathcal{H}_n(k_1^{base} k_2^{base} k_3^{base} G)$ 作为共享私钥，并发布

$$K^{grp,1x3} = \mathcal{H}_n(T_{agg}, \mathbb{S}^{1x3}, K^{sh,1x3}) K^{sh,1x3}$$

作为 1-of-3 multisig 地址。

在 3-of-3 multisig 中，每个人有 1 个秘密；在 2-of-3 multisig 中，每 2 个人的群组有 1 个共享秘密；在 1-of-3 中，每 3 个人的群组有 1 个共享秘密。

现在我们可以推广到 M-of-N：每 (N-M+1) 个人的可能群组都有一个共享秘密 [citeold-multisig-mrl-note](#)。如果 (N-M) 个人缺席，他们所有的共享秘密都至少被剩余 M 个人中的一个拥有，这些人可以协作用群组密钥签名。

M-of-N 算法

给定参与者 $e \in \{1, \dots, N\}$ ，初始私钥为 $k_1^{base}, \dots, k_N^{base}$ ，他们希望生成一个 M-of-N 合并密钥 ($M \leq N$; $M \geq 1$ 且 $N \geq 2$)，我们可以使用一个交互式算法。

我们将使用 $\mathbb{S}_s$ 来表示阶段 $s \in \{0, \dots, (N-M)\}$ 中所有唯一的公钥。最终集合 $\mathbb{S}_{N-M}$ 按照排序约定排序（例如从小到大，即字典序）。这个记号是为了方便， $\mathbb{S}_s$ 与前几节中的 $\mathbb{S}^{(N-s) \times N}$ 相同。

我们将使用 $\mathbb{S}_{s,e}^K$ 来表示每个参与者在算法阶段 s 创建的公钥集合。开始时 $\mathbb{S}_{0,e}^K = \{K_e^{base}\}$ 。

集合 $\mathbb{S}_e^k$ 将在最后包含每个参与者的聚合私钥。

1. 每个参与者 e 安全地将其原始公钥集合 $\mathbb{S}_{0,e}^K = \{K_e^{base}\}$ 发送给其他参与者。

2. 每个参与者通过收集所有 $S_{0,e}^K$ 并去除重复项来构建 S_0 。
3. 对于阶段 $s \in \{1, \dots, (N - M)\}$ (如果 $M = N$ 则跳过)
 - (a) 每个参与者 e 执行以下操作:
 - i. 对于 $S_{s-1} \notin S_{s-1,e}^K$ 的每个元素 g_{s-1} , 计算新的共享秘密
 $k_e^{base} * S_{s-1}[g_{s-1}]$
 - ii. 将所有新的共享秘密放入 $S_{s,e}^K$ 。
 - iii. 如果 $s = (N - M)$, 为 $S_{N-M,e}^K$ 中的每个元素 x 计算共享私钥
 $S_e^k[x] = \mathcal{H}_n(S_{N-M,e}^K[x])$
 并通过设置 $S_{N-M,e}^K[x] = S_e^k[x] * G$ 来覆盖公钥。
 - iv. 将 $S_{s,e}^K$ 发送给其他参与者。
 - (b) 每个参与者通过收集所有 $S_{s,e}^K$ 并去除重复项来构建 S_s 。³²
4. 每个参与者按照约定对 S_{N-M} 进行排序。
5. **premerge** 函数以 $S_{(N-M)}$ 作为输入, 每个聚合密钥为, 对于 $g \in \{1, \dots, (\text{size of } S_{N-M})\}$,

$$\mathbb{K}^{agg, \text{MxN}}[g] = \mathcal{H}_n(T_{agg}, S_{(N-M)}, S_{(N-M)}[g]) * S_{(N-M)}[g]$$
6. **merge** 函数以 $\mathbb{K}^{agg, \text{MxN}}$ 作为输入, 群组密钥为

$$K^{grp, \text{MxN}} = \sum_{g=1}^{\text{size of } S_{N-M}} \mathbb{K}^{agg, \text{MxN}}[g]$$
7. 每个参与者 e 用其聚合私钥覆盖 S_e^k 中的每个元素 x

$$S_e^k[x] = \mathcal{H}_n(T_{agg}, S_{(N-M)}, S_e^k[x]G) * S_e^k[x]$$

注意: 如果用户希望在 multisig 中拥有不平等的签名权, 例如在 3-of-4 中拥有 2 份, 他们应使用多个起始密钥分量而不是重复使用同一个。

9.7 密钥家族 (Key families)

到目前为止, 我们考虑的是一个简单签名者群组之间的密钥聚合。例如, Alice、Bob 和 Carol 各自为一个 2-of-3 multisig 地址贡献密钥分量。

现在假设 Alice 想要签署来自该地址的所有交易, 但不希望 Bob 和 Carol 在没有她的情况下签名。换句话说, (Alice + Bob) 或 (Alice + Carol) 是可接受的, 但 (Bob + Carol) 不行。

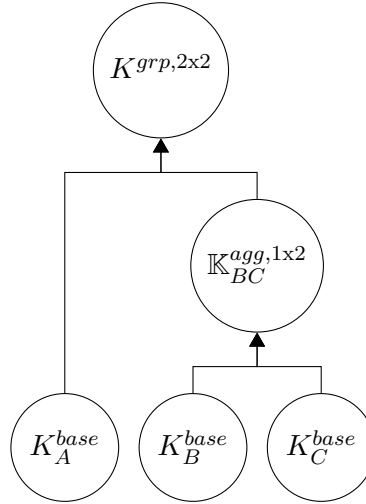
我们可以通过两层密钥聚合来实现这种场景。首先是 Bob 和 Carol 之间的 1-of-2 multisig 聚合 $\mathbb{K}_{BC}^{agg, 1 \times 2}$, 然后是 Alice 和 $\mathbb{K}_{BC}^{agg, 1 \times 2}$ 之间的 2-of-2 群组密钥 $K^{grp, 2 \times 2}$ 。基本上是一个 (2-of-([1-of-1] 和 [1-of-2])) multisig 地址。

这意味着签名权的访问结构可以相当开放。

³² 参与者应在最后阶段 ($s = N - M$) 跟踪谁拥有哪些密钥, 以促进协作签名, 其中只有 S_0 中拥有某私钥的第一个人使用它来签名。参见 refsec:n-1-of-n 节。

9.7.1 家族树 (Family trees)

我们可以像这样图示 (2-of-([1-of-1] 和 [1-of-2])) multisig 地址:



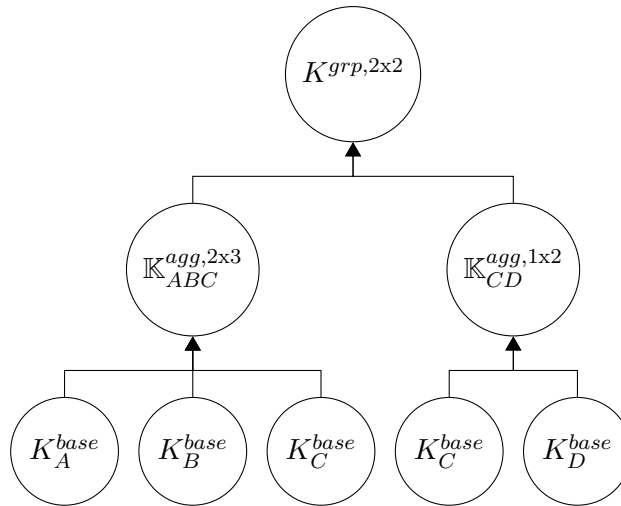
密钥 K_A^{base} , K_B^{base} , K_C^{base} 被视为

em 根祖先, 而 $K_{BC}^{agg,1x2}$ 是

em 父节点 K_B^{base} 和 K_C^{base} 的

em 子节点。父节点可以有多个子节点, 但为了概念清晰, 我们将父节点的每个副本视为不同的。这意味着可以有多个相同的根祖先密钥。

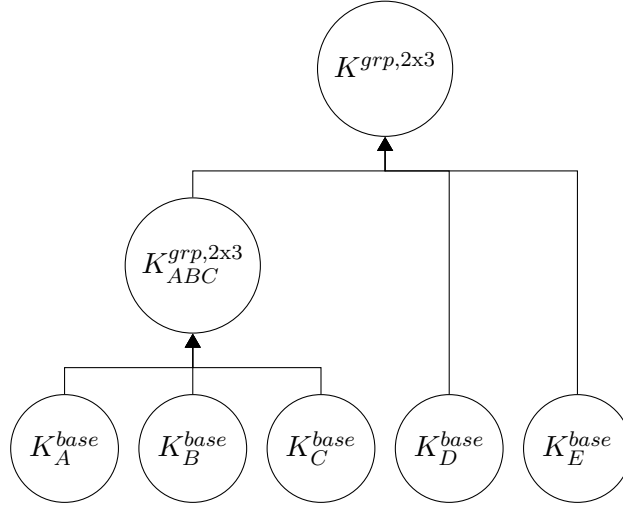
例如, 在这个 2-of-3 和 1-of-2 组合在 2-of-2 中的例子中, Carol 的密钥 K_C^{base} 被使用了两次并显示了两次:



为每个 multisig 子联盟定义了单独的集合 $\mathbb{S}$ 。前面的例子中有三个 premerge 集合: $\mathbb{S}_{ABC}^{2x3} = \{K_{AB}^{sh,2x3}, K_{BC}^{sh,2x3}, K_{AC}^{sh,2x3}\}$ 、 $\mathbb{S}_{CD}^{1x2} = \{K_{CD}^{sh,1x2}\}$ 和 $\mathbb{S}_{final}^{2x2} = \{K_{ABC}^{agg,2x3}, K_{CD}^{agg,1x2}\}$ 。

9.7.2 嵌套 multisig 密钥 (Nesting multisig keys)

假设我们有以下密钥家族



如果我们合并对应于第一个 2-of-3 的 S_{ABC}^{2x3} 中的密钥，我们会在下一层遇到问题。让我们只取一个共享秘密来说明，即 $K_{ABC}^{grp,2x3}$ 和 K_D^{base} 之间的：

$$k_{ABC,D} = \mathcal{H}_n(k_{ABC}^{grp,2x3} K_D^{base})$$

现在，ABC 中的两个人可以轻松贡献聚合密钥分量，使子联盟能够计算

$$k_{ABC}^{grp,2x3} K_D^{base} = \sum k_{ABC}^{agg,2x3} K_D^{base}$$

问题是 ABC 中的每个人都可以计算 $k_{ABC,D}^{sh,2x3} = \mathcal{H}_n(k_{ABC}^{grp,2x3} K_D^{base})$ ！如果低层 multisig 中的每个人都知道高层 multisig 的所有私钥，那么低层 multisig 不如说是 1-of-N。

我们通过不在最终子密钥之前完全合并密钥来解决这个问题。相反，我们只对低层 multisig 输出的所有密钥执行 premerge。

嵌套解决方案 (Solution for nesting)

要在新的 multisig 中使用 $\mathbb{K}^{agg,MxN}$ ，我们像普通密钥一样传递它，但有一个变化。涉及 $\mathbb{K}^{agg,MxN}$ 的操作使用其每个成员密钥，而不是合并的群组密钥。例如， $\mathbb{K}_x^{agg,2x3}$ 和 K_A^{base} 之间共享秘密的公钥”密钥”看起来像

$$\mathbb{K}_{x,A}^{sh,1x2} = \{[\mathcal{H}_n(k_A^{base} \mathbb{K}_x^{agg,2x3}[1]) * G], [\mathcal{H}_n(k_A^{base} \mathbb{K}_x^{agg,2x3}[2]) * G], \dots\}$$

这样 $\mathbb{K}_x^{agg,2x3}$ 的所有成员只知道与他们来自低层 2-of-3 multisig 的私钥对应的共享秘密。大小为二的密钥集 ${}^2\mathbb{K}_A$ 和大小为三的密钥集 ${}^3\mathbb{K}_B$ 之间的操作将产生大小为六的密钥集 ${}^6\mathbb{K}_{AB}$ 。我们可以将密钥家族中的所有密钥推广为密钥集，其中单个密钥表示为 ${}^1\mathbb{K}$ 。密钥集的元素按照某种约定排序（即从小到大），包含密钥集的集合（例如 $\mathbb{S}$ 集合）按照每个密钥集中的第一个元素排序，遵循某种约定。

我们让密钥集通过家族结构传播，每个嵌套的 multisig 群组将其 premerge 聚合集作为下一层的“基础密钥”发送，³³直到最后一个子密钥的聚合集出现，此时才最终使用 merge。

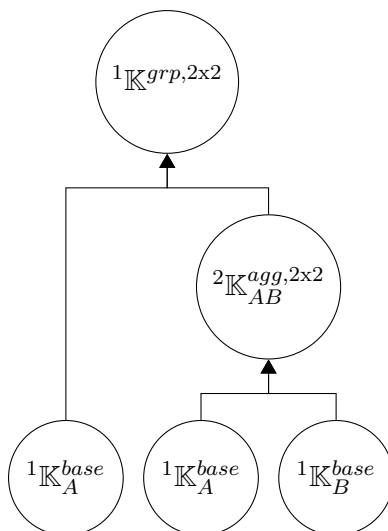
用户应存储其基础私钥、multisig 家族结构所有层级的聚合私钥以及所有层级的公钥。这样做有助于创建新结构、merging 嵌套的 multisig，以及在出现问题时与其他签名者协作重建结构。

9.7.3 对 Monero 的影响 (Implications for Monero)

对最终密钥做出贡献的每个子联盟都需要为 Monero 交易贡献分量（例如开放值 αG ），因此每个子子联盟都需要为其子联盟做出贡献。

这意味着每个根祖先，即使在家族结构中同一个密钥有多个副本，也必须向其子节点贡献一个根分量，每个子节点向其子节点贡献一个分量，依此类推。我们在每一层使用简单求和。

例如，让我们看这个家族



假设他们需要为签名计算某个群组值 x 。根祖先贡献： $x_{A,1}$ 、 $x_{A,2}$ 、 x_B 。总计 $x = x_{A,1} + x_{A,2} + x_B$ 。

目前 Monero 中没有嵌套 multisig 的实现。

³³ 注意，premerge 需要对 em 所有嵌套 multisig 的输出执行，即使 N'-of-N' multisig 嵌套到 N-of-N 中也是如此，因为集合 S 会改变。

第

10

章 10

托管市场中的 Monero

大多数时候，在线购物的体验都很顺利。买家把钱付给卖家，期望的商品就会送到家门口。如果商品有缺陷，或者在某些情况下买家改变了主意，可以退货并获得退款。

要信任一个从未见过面的人或机构并不容易，但很多消费者在购物时感到安心，因为他们知道信用卡公司可以应要求撤销付款 [?]

Cryptocurrency transaction 不可逆转，当出现问题时，买卖双方能够采取的法律手段非常有限，尤其是 Monero 这种不容易进行链上分析的加密货币 [?]。利用加密货币进行稳健在线购物的基石，是基于 2-of-3 multisig 的 escrow 交易，它允许第三方调解纠纷。通过信任这些第三方，即使是完全匿名的卖家和买家也能在无需繁文缛节的情况下进行交易。

由于 Monero multisig 的交互可能相当复杂（回忆第 ?? 节），我们专辟本章来描述一种尽可能高效的 escrow 购物环境。^{1,2}

¹ Monero 贡献者 René “rbrunner7” Brunner 创建了 MMS [?, ?]，他曾研究将 Monero multisig 集成到基于去中心化加密货币的数字市场 OpenBazaar <https://openbazaar.org/> 中。本文所阐述的概念受到了 René 在这一过程中遇到的困难的启发 [?]（他的“早期分析”）。

² 我们的初步印象是，Monero 当前的 multisig 实现已经具备了与 escrow 购物环境所需类似的交易创建流程，这对潜在的实现工作来说是个好消息。读者应注意，Monero multisig 在大量使用之前还需要一些安全更新 [?]

10.1 基本特性

在线买卖双方之间实现流畅交互，有若干基本要求和特性。这些要点取自 René 的 OpenBazaar 研究 [?]³，因为它们既合理又具有扩展性。

- 离线销售：买家应能在卖家离线时访问其在线店铺并下单。当然，卖家必须实际上线才能接收订单并完成履约。³
- 基于购买订单的支付：用于接收资金的卖家地址对每份购买订单都是唯一的，以便匹配订单和付款。
- 高信任度购买：如果买家信任卖家，可以在商品尚未履约或交付之前就付款。
 - 在线直接支付：在确认卖家在线且商品仍可购买后，买家通过卖家提供的地址将资金在单笔 transaction 中发送给卖家，卖家随即确认订单正在履约。
 - 离线支付：如果卖家离线，买家创建并为一个 1-of-2 multisig 地址注入足够的资金以覆盖预期购买。当卖家上线时，可以从 multisig 地址中提取资金（将找零退还给买家），并完成订单。如果卖家再也没有上线（或在合理的等待期之后），或者买家在卖家上线前改变了主意，买家可以将 1-of-2 地址中的资金全部转回自己的钱包。
- 调解购买：在买家、卖家和双方共同认可的调解人之间构建一个 2-of-3 multisig 地址。买家在卖家履约前向该地址注入资金，商品交付后由其中两方合作释放资金。如果卖家和买家无法达成一致，可以请调解人裁决。

我们将不涉及高信任度购买，因为无需任何复杂的通信，这些功能相当简单。⁴

10.1.1 购买 workflow

假设各方都尽到了应尽的义务，所有购买都应遵循同一套步骤。其中若干步骤涉及调解人介入时应有的判断，例如“你在找我之前是否已申请退款？”。

1. 买家访问卖家的在线店铺，确定要购买的商品，选择“我要购买”，为该笔购买准备资金，然后向卖家提交购买订单。
2. 卖家收到购买订单，核实商品有库存且可用资金充足，然后要么退还资金给买家，要么发货并附上收据通知买家。
 - 对于 2-of-3 multisig，买家可以在收到履约通知时选择性地授权付款。
3. 买家要么如预期收到商品，要么未按时收到或收到有缺陷的商品。
 - 商品正常：买家要么给卖家反馈，要么不作反应。
 - 买家发送反馈：买家可以留下好评或差评。

³ 完成购买订单意味着将商品发出交付给购买者。

⁴ 1-of-2 multisig 可以借鉴一些对 2-of-3 multisig 有用的概念，特别是地址的初始构建。

- * 好评：如果这是尚未付款的 2-of-3 multisig，则此步骤为买家确认向卖家付款。否则仅是一条好评。[workflow 结束]
- * 差评：如果这是 2-of-3 multisig，则此步骤将进入”问题商品” workflow。否则仅是一条差评。
- 买家不作反应：要么卖家已收到付款，要么是 2-of-3 multisig 且卖家需要与某人合作才能收到资金。
 - * 卖家已收款：[workflow 结束]
 - * 卖家未收款：卖家追讨付款。
 - (a) 卖家联系买家要求付款（或发送提醒）。
 - (b) 买家作出回应或不回应。
 - 买家回应：买家的回应可以是付款，也可以是要求退款。
 - > 买家付款：[workflow 结束]
 - > 买家要求退款：进入”问题商品” workflow。
 - 买家不回应：卖家请求调解人帮助释放资金。[workflow 结束]
- 未收到商品或商品有问题：买家要求退款。
 - (a) 买家联系卖家要求退款，并可能附上说明。
 - (b) 卖家同意退款或拒绝退款（或置之不理）。
 - 卖家同意：资金退还给买家。[workflow 结束]
 - 卖家拒绝：要么不是 2-of-3 multisig，要么是。
 - * 不是 2-of-3: [workflow 结束]
 - * 是 2-of-3: 买家要么放弃退款要求（明确放弃，或未在合理时间内回应），要么态度坚决。
 - 买家放弃：买家是否已授权卖家的付款。
 - > 买家已授权付款：[workflow 结束]
 - > 买家未授权：卖家联系调解人，由调解人授权付款。[workflow 结束]
 - 买家态度坚决：卖家或买家联系调解人，调解人与各方合作作出裁决。[workflow 结束]

10.2 无缝 Monero multisig

我们可以利用自然的购买订单 workflow，在参与者甚至没有察觉的情况下，将 Monero 2-of-3 multisig 交互的几乎所有部分都嵌入其中。卖家在最后还有一个额外的小步骤——他必须签名并提交最终的 transaction 以收到付款，类似于”清空收银台”。⁵

10.2.1 multisig 交互基础

所有 2-of-3 multisig 交互都包含相同的通信轮次集合，涉及地址设置和交易构建。为了符合正常工作流，我们使用了与 multisig 章节（回忆第 ?? 和 ?? 节）相比重新组织的交易构建流程。⁶

⁵ 将此扩展到 (N-1)-of-N 以上很可能不可行，除非增加更多步骤，因为设置更低阈值的 multisig 地址需要额外的通信轮次。

⁶ 此流程实际上与 Monero 当前组织 multisig 交易的方式非常相似。

1. 地址设置

- (a) 所有用户必须首先了解其他参与者的 base key，用于构建共享秘密。他们将共享秘密的公钥（例如 $K^{sh} = \mathcal{H}_n(k_A^{base} * k_B^{base} G)G$ ）传输给其他用户。
 - (b) 在得知所有共享秘密公钥后，每个用户可以进行 **premerge** 然后 **merge**，将其合并为地址的公 spend key。聚合后的私 spend key 将用于签名交易。主要签名方（买家和卖家）共享秘密私钥的 hash（例如 $k^{sh} = \mathcal{H}_n(k_A^{base} * k_B^{base} G)$ ）将用作 view key（例如 $k^v = \mathcal{H}_n(k^{sh})$ ）。我们稍后会对这些密钥进行更详细的说明（第 ?? 节）。
2. 交易构建：假设该地址已至少拥有一个 output，且 key image 未知。有两个签名方：发起者和协作方。
- (a) 发起交易：发起者决定开始一笔交易。她为所有拥有的 output 生成承诺值（例如 αG ）（她还不确定哪些会被使用），并对它们进行承诺。她还为这些拥有的 output 创建部分 key image，并用合法性证明签名（回忆第 ?? 节）。她将这些信息连同自己用于接收资金的个人地址（例如用于找零等）一起发送给协作方。
 - (b) 制作部分交易：协作方验证发起交易中的所有信息是否有效。他确定 output 接收者和金额（可部分基于发起者的建议）、要使用的 input 及其各自环成员伪装以及交易费用。他生成交易私钥（如有子地址则生成多个私钥），创建一次性地址、output 承诺、编码金额、伪 output 承诺掩码以及零承诺的承诺值。为证明金额在范围内，他为所有 output 构建 Bulletproof 范围证明。他还为自己的签名生成承诺值（但不对其进行承诺），为 MLSAG 签名生成随机标量，为拥有的 output 生成部分 key image 以及这些部分 key image 的合法性证明。所有这些都发送给发起者。
 - (c) 发起者的部分签名：发起者验证部分交易的信息是否有效，并符合她的预期（例如金额和接收者是否正确）。她用私钥完成 MLSAG 签名并签名，然后将部分签名的交易连同她已公开的承诺值一起发送给协作方。
 - (d) 完成交易：协作方完成交易的签名，并将其提交到网络。

单承诺签名

与我们 multisig 章节中的建议不同，每笔部分交易只提供一个承诺（由交易发起者提供），并在协作方明确发出其承诺值后才公开。对承诺值（例如 αG ）进行承诺的目的是防止恶意协作方利用自己的承诺值来影响所产生的挑战值，这可能使他发现其他协作方的聚合密钥（回忆第 ?? 节）。如果恶意行为者在生成自己的承诺值时缺少哪怕一个部分承诺值，那么他就不可能（或概率可忽略不计地）控制挑战值。^{7,8,9}

以这种方式简化可以减少一轮通信轮次，这对买家-卖家交互体验具有重要影响。

⁷ 这也是为什么发起者在所有交易信息确定后才公开她的承诺值，这样任何签名方都无法更改 MLSAG 消息并影响挑战值。

⁸ 单承诺签名可以推广为 (M-1)-承诺签名，即只有部分交易的作者不进行承诺-公开，其他协作签名方只在交易完全确定后才公开。例如，假设有一个 3-of-3 地址，协作方为 (A,B,C)，他们将尝试单承诺签名。签名方 B 和 C 组成针对 A 的恶意联盟，而 C 是发起者，B 是部分交易的作者。C 以承诺发起，然后 A 提供他的承诺值（不承诺）。当 B 构建部分交易时，他可以与 C 共谋控制签名挑战值，以暴露 A 的私钥。另请注意，(M-1)-承诺签名是本文首次提出的原创概念，没有高级研究资料或代码实现的支撑，最终可能完全是错误的。

⁹ 一种理解方式是考虑“承诺”的含义和目的（回忆第 ?? 节）。一旦 Alice 对值 A 进行承诺，她就无法再……

10.2.2 Escrow 用户体验

以下是涉及 Monero 2-of-3 multisig 在线购买中买家-卖家-调解人交互的详细演示。

1. 买家进行购买

- (a) 买家的新购购物会话：购物者进入一个在线市场，她的客户端生成一个新的子地址，用于在她开始新的购买订单时使用。¹⁰ 在该市场中她找到卖家，每个卖家都提供一系列产品和价格。对购物者本人不可见，但对其客户端（即她用来购物的软件）可见的是，每个产品都有一个用于基于 multisig 购买的 base key。与该 base key 一起的是一份预选的调解人列表，每个调解人都有一个 base key 和预先计算的卖家-调解人共享秘密公钥。¹¹
- (b) 买家将产品加入购物车：我们的购物者决定要某样东西，选择支付方式（即直接支付、1-of-2 multisig 或 2-of-3 multisig），如果她选择 2-of-3 multisig，就会看到一份可选的调解人列表。当她将产品加入购物车时，她的客户端在她不知情的情况下（假设她选择了 2-of-3 multisig），使用产品的 base key、调解人的 base key 和卖家-调解人共享秘密公钥，结合她自己购物会话子地址的 spend key（作为她的 base key），构建一个 2-of-3 买家-卖家-调解人 multisig 地址。¹²

view key 是买家-卖家共享秘密私钥的 hash（不是聚合私钥，即在 premerge 之前），买家和卖家之间的通信加密 key 是 view key 的 hash。¹³

- (c) 买家进入结账：购物者查看她购物车中的所有产品，决定进入结账。这是她在最终确定购买订单之前准备资金的地方。她的客户端构建一笔交易（但尚未签名），要么直接付给卖家，要么为 multisig 地址注入资金（稍多一些以支付未来的费用）。如果为 2-of-3 multisig 地址注入资金，客户端还会发起两笔从该地址转出的交易。一笔用于付给卖家，另一笔用于退款给买家。它们 input 的部分 key image 基于尚未签名的注入资金交易。

实际上，她只需要这两笔交易的承诺值，然后分别是一份部分 key image（附合法性证明）和一份她的购物会话子地址。该子地址具有双重用途：它是买家接收退款或找零 output 的地址，其 spend key 是买家的 multisig base key。¹⁴

- (d) 买家授权付款：在查看所有购买订单细节后，购物者授权付款。¹⁵ 她的客户端完成注入资金交易的签名，并将其提交到网络。¹⁶ 它将购买订单连同注入资金交易的 hash 和已

¹⁰ 为每份购买订单使用新的子地址，甚至为每个不同的卖家或卖家产品使用新的子地址，使得卖家更难追踪客户的行为。这也有助于确保每份购买订单的唯一性，以防有人两次购买同一商品。

¹¹ 卖家也可以轻易地（对购物者不可见地）包含交易的承诺值承诺。然而，为了处理同一产品的多份购买订单，他必须为每个潜在买家预先提供许多承诺。我们只能想象那会变得多么混乱。这正是我们单承诺签名简化所带来的效用的一部分。

¹² 市场应该如何实现是开放性的问题，因为例如选择产品的支付类型可以在结账时而非“加入购物车”界面中呈现给用户。

¹³ 1-of-2 multisig 也会进行同样的过程，只是不包含调解人。

¹⁴ 发起独立的交易很重要，因为承诺值只能使用一次。

¹⁵ 如果买家取消购买订单，她的注入资金交易和部分 multisig 交易将被删除。

¹⁶ 如果她的购物车包含多个卖家的产品，那么她的客户端可以创建多份购买订单并分别处理。所有卖家都可以由同一笔注入资金交易来支付。

发起的 multisig 交易以及买家-调解人共享秘密公钥发送给卖家。¹⁷

2. 卖家完成购买订单

- (a) 卖家评估购买订单：卖家审查我们的购物者的购买订单，然后批准发货。如果是直接付款，就不需要再考虑什么；如果是通过 1-of-2 multisig 付款，他可以制作一笔从该地址付给自己的交易。对于 2-of-3 multisig，他的客户端为购买订单生成一个接收资金的子地址，并从买家发起的交易中制作两笔部分交易。付款交易将产品价格发送给卖家，余额作为找零退还给买家，而退款交易则将 multisig 地址中的全部资金退还给买家。¹⁸ 注意，他通过买家-卖家-调解人 base key 结合买家-调解人共享秘密公钥来重建 multisig 地址。
 - (b) 卖家发货：卖家发货，并向买家发送履约通知。该通知包含购买的收据，以及要求用户完成付款的请求（从这里开始都指 2-of-3 multisig）。对用户隐藏的是卖家的购买订单子地址（可用于争议解决）以及两笔部分交易。
3. 买家完成付款或要求退款：买家可以在收到履约通知后立即操作，也可以等到商品送达后再操作。
- (a) 买家提交部分签名的交易：买家决定是为购买付款还是要求退款。她的客户端在相应的部分交易上创建部分签名，并将其发送给卖家。任何退款大概都会附上合理的说明。
 - (b) 卖家完成交易：卖家收到部分签名的交易，完成签名，并将其提交到网络。如有必要，他会向买家发送退款通知及证明。
4. 调解争议：在买家提交购买订单之后、multisig 地址中的资金被清空之前的任何时候，卖家或买家都可以决定请调解人介入。Party

A Party
B ¹⁹

5. Party

A Party
A Party
A — multisig basekey Party
A — Party
B viewkey multisig keyimage

6. 7. 调解人确认争议：调解人确认他们正在调查争议，同时从发起的交易中制作部分交易并发送给 Party

A Party
B Party
B Party
A

¹⁷ 买家的客户端应记录付款订单的细节，如总价格，以便稍后在签名前验证 multisig 交易的内容。

¹⁸ 部分交易很可能共享很多值，因为它们涉及相同的 input，最终只应有一笔被签名。为了模块化和设计的健壮性，我们认为最好分别处理它们。

¹⁹ 我们的争议解决方案假设参与者是善意的。不合作的人，例如拒绝发起或签名不利于自己的交易，无疑会让整个过程对每个人来说都更加繁琐。

8. 争议结束：要么买家和卖家最终自行解决，要么调解人作出裁决并通知双方。

- 注意：如果根据裁决被告方应收到资金，但因任何原因未提供地址，调解人可以尝试联系他们并与他们合作以接收这些资金。由于这不涉及争议方，该联系可以在争议解决后进行（或继续推进）。

Party
A B Party
A - Party
B

Party
A

Party
A

Party
B

Party
B
Party
B
B
B

Party
B *hash*

调解人结束争议：调解人总结争议及其解决方案，并将报告发送给买家和卖家。

这里安装了四个关键的设计优化。

预选调解人

通过提前选择调解人，卖家可以与每个调解人针对每个产品创建一个共享秘密，并将其公钥与产品信息一起发布。²⁰ 这样，当买家决定购买某物时，就可以一步构建完整的合并 multisig 地址，与“离线销售”的要求保持一致。预选多个调解人可以让买家选择更信任的那一个。

如果买家不信任卖家接受的调解人，也可以与他们自己选择的在线调解人合作，使用卖家的产品 base key 创建一个 multisig 地址。在收到购买订单后，卖家可以接受该新调解人或拒绝销售。²¹

²⁰重要的是，这些 multisig 地址仍然能够抵抗密钥聚合测试，因为买家的共享秘密对观察者来说是未知的。

²¹为方便起见，escrow 服务可以“始终在线”，当购买订单创建时，所有 2-of-3 multisig 地址都与该服务一起主动构建，而不是使用预选的调解人。另一种可能是使用嵌套 multisig (第 ?? 节)，其中预选的调解人实际上是一个 1-of-N multisig 组。这样每当出现争议时，该组中任何恰好可用的调解人都可以介入。实现这一点可能需要大量的开发工作。

我们期望买家和卖家双方对好的调解人的共同需求，能够随着时间的推移创造出一个按服务质量和公平性组织的调解人层级。质量较低或声誉较差的调解人可能会赚取更少的钱或服务更少或不太重要的交易。²²

子地址和产品 ID

卖家为每个产品线/ID 创建一个新的 base key，这些 key 用于构建 2-of-3 multisig 地址。²³ 当卖家完成购买订单时，他们会创建一个唯一的子地址用于接收资金，可用于将购买订单与收到的付款进行匹配。

这里高效地满足了“基于购买订单的支付”的要求，特别是因为发送到不同子地址的资金可以从同一个钱包中轻松访问（回忆第 ?? 节）。

预先部分交易

multisig 交易需要多轮通信，因此我们尽早开始制作。为了用户便利，那些很少使用的部分交易（例如退款交易）会提前制作，以便在需要时立即可以签名。

条件性调解人访问

出于效率和隐私的考虑，调解人只需要在解决争议时才能访问交易的细节。为此，我们将 multisig 的私 view key 设为买家-卖家共享秘密私钥的 hash： $k_{purchase-order}^{v,grp} = \mathcal{H}_n(T_{mv}, k_{AB}^{sh,2x3})$ ，其中 T_{mv} 是市场 view key 域分隔符，A 和 B 分别对应卖家和买家， $k_{AB}^{sh,2x3} = \mathcal{H}_n(k_A^{base} * k_B^{base} G)$ 。我们将其“查看”的范围扩展到买家-卖家的通信记录。换言之，通信加密 key 是 $k_{purchase-order}^{ce} = \mathcal{H}_n(T_{me}, k_{purchase-order}^{v,grp})$ (T_{me} 是市场加密 key 域分隔符)。^{24,25}

调解人只有在原始一方之一将 view key 释放给他们时，才能获得买家-卖家通信的访问权和授权付款的能力。²⁶

²²我们尚不清楚调解人最佳或最可能的收费方式。也许他们会从每笔被调解的交易或添加他们为调解人的交易中收取固定或百分比费用（然后如果费用未在原始部分交易中提供，则拒绝在争议中合作），或者用户和/或卖家和/或市场平台会与他们签订合同。

²³此 base key 也用于 1-of-2 multisig 购买。我们认为不在通信渠道中暴露私 spend key 很重要，因此使用买家-卖家共享秘密是非常合理的。

²⁴将 view key 和加密 key 分离，允许仅赋予查看通信记录的权限，而不授予查看 multisig 地址交易历史的权限。

²⁵1-of-2 multisig 地址也使用此方法。

²⁶调解人验证他们收到的通信记录没有被篡改是很重要的。一种方法可能是让每个协作

此外，卖家可以通过检查他们为每个产品发布的 base key 是否与显示的一致，来验证市场主机（根据此概念的实现方式，可能也是唯一可用的调解人）没有对他们的客户对话进行 MITM（“中间人”攻击，即假装是买家或卖家）。由于用于创建 multisig 地址的买家 base key 也是加密 key 的一部分，恶意主机将很难策划 MITM 攻击。

签名方在发送新消息时包含当前消息记录的签名 hash。调解人可以查看来回消息和 hash 记录的序列来识别差异。这也有助于协作签名方识别未能传输的消息，以及替代性地创建特定签名方确实收到了某些消息的证据。

第 11 章 11

托管市场中的 Monero

大多数时候，在线购物的体验都很顺利。买家把钱付给卖家，期望的商品就会送到家门口。如果商品有缺陷，或者在某些情况下买家改变了主意，可以退货并获得退款。

要信任一个从未见过面的人或机构并不容易，但很多消费者在购物时感到安心，因为他们知道信用卡公司可以应要求撤销付款 [?]

Cryptocurrency transaction 不可逆转，当出现问题时，买卖双方能够采取的法律手段非常有限，尤其是 Monero 这种不容易进行链上分析的加密货币 [?]。利用加密货币进行稳健在线购物的基石，是基于 2-of-3 multisig 的 escrow 交易，它允许第三方调解纠纷。通过信任这些第三方，即使是完全匿名的卖家和买家也能在无需繁文缛节的情况下进行交易。

由于 Monero multisig 的交互可能相当复杂（回忆第 ?? 节），我们专辟本章来描述一种尽可能高效的 escrow 购物环境。¹²

11.1 基本特性

在线买卖双方之间实现流畅交互，有若干基本要求和特性。这些要点取自 René 的 OpenBazaar 研究 [?]，因为它们既合理又具有扩展性。

- 离线销售：买家应能在卖家离线时访问其在线店铺并下单。当然，卖家必须实际上线才能接收订单并完成履约。³

¹Monero 贡献者 René “rbrunner7” Brunner 创建了 MMS [?, ?]，他曾研究将 Monero multisig 集成到基于去中心化加密货币的数字市场 OpenBazaar <https://openbazaar.org/> 中。本文所阐述的概念受到了 René 在这一过程中遇到的困难的启发 [?]（他的“早期分析”）。

²我们的初步印象是，Monero 当前的 multisig 实现已经具备了与 escrow 购物环境所需类似的交易创建流程，这对潜在的实现工作来说是个好消息。读者应注意，Monero multisig 在大量使用之前还需要一些安全更新 [?]

³完成购买订单意味着将商品发出交付给购买者。

- 基于购买订单的支付：用于接收资金的卖家地址对每份购买订单都是唯一的，以便匹配订单和付款。
- 高信任度购买：如果买家信任卖家，可以在商品尚未履约或交付之前就付款。
 - 在线直接支付：在确认卖家在线且商品仍可购买后，买家通过卖家提供的地址将资金在单笔 transaction 中发送给卖家，卖家随即确认订单正在履约。
 - 离线支付：如果卖家离线，买家创建并为一个 1-of-2 multisig 地址注入足够的资金以覆盖预期购买。当卖家上线时，可以从 multisig 地址中提取资金（将找零退还给买家），并完成订单。如果卖家再也没有上线（或在合理的等待期之后），或者买家在卖家上线前改变了主意，买家可以将 1-of-2 地址中的资金全部转回自己的钱包。
- 调解购买：在买家、卖家和双方共同认可的调解人之间构建一个 2-of-3 multisig 地址。买家在卖家履约前向该地址注入资金，商品交付后由其中两方合作释放资金。如果卖家和买家无法达成一致，可以请调解人裁决。

我们将不涉及高信任度购买，因为无需任何复杂的通信，这些功能相当简单。⁴

11.1.1 购买 workflow

假设各方都尽到了应尽的义务，所有购买都应遵循同一套步骤。其中若干步骤涉及调解人介入时应有的判断，例如“你在找我之前是否已申请退款？”。

1. 买家访问卖家的在线店铺，确定要购买的商品，选择“我要购买”，为该笔购买准备资金，然后向卖家提交购买订单。
2. 卖家收到购买订单，核实商品有库存且可用资金充足，然后要么退还资金给买家，要么发货并附上收据通知买家。
 - 对于 2-of-3 multisig，买家可以在收到履约通知时选择性地授权付款。
3. 买家要么如预期收到商品，要么未按时收到或收到有缺陷的商品。
 - 商品正常：买家要么给卖家反馈，要么不作反应。
 - 买家发送反馈：买家可以留下好评或差评。

⁴1-of-2 multisig 可以借鉴一些对 2-of-3 multisig 有用的概念，特别是地址的初始构建。

- * 好评：如果这是尚未付款的 2-of-3 multisig，则此步骤为买家确认向卖家付款。否则仅是一条好评。[工作流结束]
- * 差评：如果这是 2-of-3 multisig，则此步骤将进入”问题商品”工作流。否则仅是一条差评。
- 买家不作反应：要么卖家已收到付款，要么是 2-of-3 multisig 且卖家需要与某人合作才能收到资金。
 - * 卖家已收款：[工作流结束]
 - * 卖家未收款：卖家追讨付款。
 - (a) 卖家联系买家要求付款（或发送提醒）。
 - (b) 买家作出回应或不回应。
 - 买家回应：买家的回应可以是付款，也可以是要求退款。
 - > 买家付款：[工作流结束]
 - > 买家要求退款：进入”问题商品”工作流。
 - 买家不回应：卖家请求调解人帮助释放资金。[工作流结束]
- 未收到商品或商品有问题：买家要求退款。
 - (a) 买家联系卖家要求退款，并可能附上说明。
 - (b) 卖家同意退款或拒绝退款（或置之不理）。
 - 卖家同意：资金退还给买家。[工作流结束]
 - 卖家拒绝：要么不是 2-of-3 multisig，要么是。
 - * 不是 2-of-3: [工作流结束]
 - * 是 2-of-3: 买家要么放弃退款要求（明确放弃，或未在合理时间内回应），要么态度坚决。
 - 买家放弃：买家是否已授权卖家的付款。
 - > 买家已授权付款：[工作流结束]
 - > 买家未授权：卖家联系调解人，由调解人授权付款。[工作流结束]
 - 买家态度坚决：卖家或买家联系调解人，调解人与各方合作作出裁决。[工作流结束]

11.2 无缝 Monero multisig

我们可以利用自然的购买订单工作流，在参与者甚至没有察觉的情况下，将 Monero 2-of-3 multisig 交互的几乎所有部分都嵌入其中。卖家在最后还有一

个额外的小步骤——他必须签名并提交最终的 transaction 以收到付款，类似于“清空收银台”。⁵

11.2.1 multisig 交互基础

所有 2-of-3 multisig 交互都包含相同的通信轮次集合，涉及地址设置和交易构建。为了符合正常工作流，我们使用了与 multisig 章节（回忆第 ?? 和 ?? 节）相比重新组织的交易构建流程。⁶

1. 地址设置

- (a) 所有用户必须首先了解其他参与者的 base key，用于构建共享秘密。他们将共享秘密的公钥（例如 $K^{sh} = \mathcal{H}_n(k_A^{base} * k_B^{base} G)G$ ）传输给其他用户。
- (b) 在得知所有共享秘密公钥后，每个用户可以进行 premerge 然后 merge，将其合并为地址的公 spend key。聚合后的私 spend key 将用于签名交易。主要签名方（买家和卖家）共享秘密私钥的 hash（例如 $k^{sh} = \mathcal{H}_n(k_A^{base} * k_B^{base} G)$ ）将用作 view key（例如 $k^v = \mathcal{H}_n(k^{sh})$ ）。我们稍后会对这些密钥进行更详细的说明（第 ?? 节）。

2. 交易构建：假设该地址已至少拥有一个 output，且 key image 未知。有两个签名方：发起者和协作方。

- (a) 发起交易：发起者决定开始一笔交易。她为所有拥有的 output 生成承诺值（例如 αG ）（她还不确定哪些会被使用），并对它们进行承诺。她还为这些拥有的 output 创建部分 key image，并用合法性证明签名（回忆第 ?? 节）。她将这些信息连同自己用于接收资金的个人地址（例如用于找零等）一起发送给协作方。
- (b) 制作部分交易：协作方验证发起交易中的所有信息是否有效。他确定 output 接收者和金额（可部分基于发起者的建议）、要使用的 input 及其各自环成员伪装以及交易费用。他生成交易私钥（如有子地址则生成多个私钥），创建一次性地址、output 承诺、编码金额、伪 output 承诺掩码以及零承诺的承诺值。为证明金额在范围内，他为所有 output 构建 Bulletproof 范围证明。他还为自己的签名生成承诺值（但不对其进行承诺），为 MLSAG 签名生成随机标量，为拥有的 output 生成部分 key image 以及这些部分 key image 的合法性证明。所有这些都发送给发起者。

⁵将此扩展到 (N-1)-of-N 以上很可能不可行，除非增加更多步骤，因为设置更低阈值的 multisig 地址需要额外的通信轮次。

⁶此流程实际上与 Monero 当前组织 multisig 交易的方式非常相似。

- (c) 发起者的部分签名：发起者验证部分交易的信息是否有效，并符合她的预期（例如金额和接收者是否正确）。她用私钥完成 MLSAG 签名并签名，然后将部分签名的交易连同她已公开的承诺值一起发送给协作方。
- (d) 完成交易：协作方完成交易的签名，并将其提交到网络。

单承诺签名

与我们 multisig 章节中的建议不同，每笔部分交易只提供一个承诺（由交易发起者提供），并在协作方明确发出其承诺值后才公开。对承诺值（例如 αG ）进行承诺的目的是防止恶意协作方利用自己的承诺值来影响所产生的挑战值，这可能使他发现其他协作方的聚合密钥（回忆第 ?? 节）。如果恶意行为者在生成自己的承诺值时缺少哪怕一个部分承诺值，那么他就不可能（或概率可忽略不计地）控制挑战值。⁷⁸⁹

以这种方式简化可以减少一轮通信轮次，这对买家-卖家交互体验具有重要影响。

11.2.2 Escrow 用户体验

以下是涉及 Monero 2-of-3 multisig 在线购买中买家-卖家-调解人交互的详细演示。

1. 买家进行购买

- (a) 买家的新购购物会话：购物者进入一个在线市场，她的客户端生成一个新的子地址，用于在她开始新的购买订单时使用。¹⁰ 在该市场中她找到卖家，每个卖家都提供一系列产品价格。对购物者本人不可见，但对其客户端（即她用来购物的软件）可见的是，每个产品都有一个用于基于 multisig 购买的 base key。与该 base

⁷这也是为什么发起者在所有交易信息确定后才公开她的承诺值，这样任何签名方都无法更改 MLSAG 消息并影响挑战值。

⁸单承诺签名可以推广为 (M-1)-承诺签名，即只有部分交易的作者不进行承诺-公开，其他协作签名方只在交易完全确定后才公开。例如，假设有一个 3-of-3 地址，协作方为 (A,B,C)，他们将尝试单承诺签名。签名方 B 和 C 组成针对 A 的恶意联盟，而 C 是发起者，B 是部分交易的作者。C 以承诺发起，然后 A 提供他的承诺值（不承诺）。当 B 构建部分交易时，他可以与 C 共谋控制签名挑战值，以暴露 A 的私钥。另请注意，(M-1)-承诺签名是本文首次提出的原创概念，没有高级研究资料或代码实现的支撑，最终可能完全是错误的。

⁹一种理解方式是考虑“承诺”的含义和目的（回忆第 ?? 节）。一旦 Alice 对值 A 进行承诺，她就无法再……

¹⁰为每份购买订单使用新的子地址，甚至为每个不同的卖家或卖家产品使用新的子地址，使得卖家更难追踪客户的行为。这也有助于确保每份购买订单的唯一性，以防有人两次购买同一商品。

key 一起的是一份预选的调解人列表, 每个调解人都有一个 base key 和预先计算的卖家-调解人共享秘密公钥。¹¹

- (b) 买家将产品加入购物车: 我们的购物者决定要某样东西, 选择支付方式 (即直接支付、1-of-2 multisig 或 2-of-3 multisig), 如果她选择 2-of-3 multisig, 就会看到一份可选的调解人列表。当她将产品加入购物车时, 她的客户端在她不知情的情况下 (假设她选择了 2-of-3 multisig), 使用产品的 base key、调解人的 base key 和卖家-调解人共享秘密公钥, 结合她自己购物会话子地址的 spend key (作为她的 base key), 构建一个 2-of-3 买家-卖家-调解人 multisig 地址。¹²

view key 是买家-卖家共享秘密私钥的 hash (不是聚合私钥, 即在 premerge 之前), 买家和卖家之间的通信加密 key 是 view key 的 hash。¹³

- (c) 买家进入结账: 购物者查看她购物车中的所有产品, 决定进入结账。这是她在最终确定购买订单之前准备资金的地方。她的客户端构建一笔交易 (但尚未签名), 要么直接付给卖家, 要么为 multisig 地址注入资金 (稍多一些以支付未来的费用)。如果为 2-of-3 multisig 地址注入资金, 客户端还会发起两笔从该地址转出的交易。一笔用于付给卖家, 另一笔用于退款给买家。它们 input 的部分 key image 基于尚未签名的注入资金交易。

实际上, 她只需要这两笔交易的承诺值, 然后分别是一份部分 key image (附合法性证明) 和一份她的购物会话子地址。该子地址具有双重用途: 它是买家接收退款或找零 output 的地址, 其 spend key 是买家的 multisig base key。¹⁴

- (d) 买家授权付款: 在查看所有购买订单细节后, 购物者授权付款。¹⁵ 她的客户端完成注入资金交易的签名, 并将其提交到网络。¹⁶ 它将购买订单连同注入资金交易的 hash 和已发起的 multisig 交易以及买家-调解人共享秘密公钥发送给卖家。¹⁷

¹¹ 卖家也可以轻易地 (对购物者不可见地) 包含交易的承诺值承诺。然而, 为了处理同一产品的多份购买订单, 他必须为每个潜在买家预先提供许多承诺。我们只能想象那会变得多么混乱。这正是我们单承诺签名简化所带来的效用的一部分。

¹² 市场应该如何实现是开放性的问题, 因为例如选择产品的支付类型可以在结账时而非“加入购物车”界面中呈现给用户。

¹³ 1-of-2 multisig 也会进行同样的过程, 只是不包含调解人。

¹⁴ 发起独立的交易很重要, 因为承诺值只能使用一次。

¹⁵ 如果买家取消购买订单, 她的注入资金交易和部分 multisig 交易将被删除。

¹⁶ 如果她的购物车包含多个卖家的产品, 那么她的客户端可以创建多份购买订单并分别处理。所有卖家都可以由同一笔注入资金交易来支付。

¹⁷ 买家的客户端应记录付款订单的细节, 如总价格, 以便稍后在签名前验证 multisig 交

2. 卖家完成购买订单

- (a) 卖家评估购买订单：卖家审查我们的购物者的购买订单，然后批准发货。如果是直接付款，就不需要再考虑什么；如果是通过 1-of-2 multisig 付款，他可以制作一笔从该地址付给自己的交易。对于 2-of-3 multisig，他的客户端为购买订单生成一个接收资金的子地址，并从买家发起的交易中制作两笔部分交易。付款交易将产品价格发送给卖家，余额作为找零退还给买家，而退款交易则将 multisig 地址中的全部资金退还给买家。¹⁸ 注意，他通过买家-卖家-调解人 base key 结合买家-调解人共享秘密公钥来重建 multisig 地址。
 - (b) 卖家发货：卖家发货，并向买家发送履约通知。该通知包含购买的收据，以及要求用户完成付款的请求（从这里开始都指 2-of-3 multisig）。对用户隐藏的是卖家的购买订单子地址（可用于争议解决）以及两笔部分交易。
3. 买家完成付款或要求退款：买家可以在收到履约通知后立即操作，也可以等到商品送达后再操作。
 - (a) 买家提交部分签名的交易：买家决定是为购买付款还是要求退款。她的客户端在相应的部分交易上创建部分签名，并将其发送给卖家。任何退款大概都会附上合理的说明。
 - (b) 卖家完成交易：卖家收到部分签名的交易，完成签名，并将其提交到网络。如有必要，他会向买家发送退款通知及证明。
 4. 调解争议：在买家提交购买订单之后、multisig 地址中的资金被清空之前的任何时候，卖家或买家都可以决定请调解人介入。Party

A Party
B 19

5. Party

A Party
A Party
A - multisig basekey Party
A - Party
B viewkey multisig keyimage

易的内容。

¹⁸部分交易很可能共享很多值，因为它们涉及相同的 input，最终只应有一笔被签名。为了模块化和设计的健壮性，我们认为最好分别处理它们。

¹⁹我们的争议解决方案假设参与者是善意的。不合作的人，例如拒绝发起或签名不利于自己的交易，无疑会让整个过程对每个人来说都更加繁琐。

6. 7. 调解人确认争议：调解人确认他们正在调查争议，同时从发起的交易中制作部分交易并发送给 Party

A Party
B Party
B Party
A

8. 争议结束：要么买家和卖家最终自行解决，要么调解人作出裁决并通知双方。
- 注意：如果根据裁决被告方应收到资金，但因任何原因未提供地址，调解人可以尝试联系他们并与合作以接收这些资金。由于这不涉及争议方，该联系可以在争议解决后进行（或继续推进）。

Party
A B Party
A - Party
B
Party
A

Party
A
Party
B
Party
B Party
B Party
B

Party
B hash

调解人结束争议：调解人总结争议及其解决方案，并将报告发送给买家和卖家。

这里安装了四个关键的设计优化。

预选调解人

通过提前选择调解人，卖家可以与每个调解人针对每个产品创建一个共享秘密，并将其公钥与产品信息一起发布。²⁰ 这样，当买家决定购买某物时，就

²⁰重要的是，这些 multisig 地址仍然能够抵抗密钥聚合测试，因为买家的共享秘密对观察者来说是未知的。

可以一步构建完整的合并 multisig 地址，与”离线销售”的要求保持一致。预选多个调解人可以让买家选择更信任的那一个。

如果买家不信任卖家接受的调解人，也可以与他们自己选择的在线调解人合作，使用卖家的产品 base key 创建一个 multisig 地址。在收到购买订单后，卖家可以接受该新调解人或拒绝销售。²¹

我们期望买家和卖家双方对好的调解人的共同需求，能够随着时间的推移创造出按服务质量和公平性组织的调解人层级。质量较低或声誉较差的调解人可能会赚取更少的钱或服务更少或不太重要的交易。²²

子地址和产品 ID

卖家为每个产品线/ID 创建一个新的 base key，这些 key 用于构建 2-of-3 multisig 地址。²³ 当卖家完成购买订单时，他们会创建一个唯一的子地址用于接收资金，可用于将购买订单与收到的付款进行匹配。

这里高效地满足了”基于购买订单的支付”的要求，特别是因为发送到不同子地址的资金可以从同一个钱包中轻松访问（回忆第 ?? 节）。

预先部分交易

multisig 交易需要多轮通信，因此我们尽早开始制作。为了用户便利，那些很少使用的部分交易（例如退款交易）会提前制作，以便在需要时立即可以签名。

条件性调解人访问

出于效率和隐私的考虑，调解人只需要在解决争议时才能访问交易的细节。为此，我们将 multisig 的私 view key 设为买家-卖家共享秘密私钥的 hash： $k_{purchase-order}^{v,grp} = \mathcal{H}_n(T_{mv}, k_{AB}^{sh,2x3})$ ，其中 T_{mv} 是市场 view key 域分隔符，A 和 B 分别对应卖家和买家， $k_{AB}^{sh,2x3} = \mathcal{H}_n(k_A^{base} * k_B^{base} G)$ 。我们将其可”查看”的范围扩展到买家-卖家的通信记录。换言之，通信加密 key 是

²¹为方便起见，escrow 服务可以”始终在线”，当购买订单创建时，所有 2-of-3 multisig 地址都与该服务一起主动构建，而不是使用预选的调解人。另一种可能是使用嵌套 multisig（第 ?? 节），其中预选的调解人实际上是一个 1-of-N multisig 组。这样每当出现争议时，该组中任何恰好可用的调解人都可以介入。实现这一点可能需要大量的开发工作。

²²我们尚不清楚调解人最佳或最可能的收费方式。也许他们会从每笔被调解的交易或添加他们为调解人的交易中收取固定或百分比费用（然后如果费用未在原始部分交易中提供，则拒绝在争议中合作），或者用户和/或卖家和/或市场平台会与他们签订合同。

²³此 base key 也用于 1-of-2 multisig 购买。我们认为不在通信渠道中暴露私 spend key 很重要，因此使用买家-卖家共享秘密是非常合理的。

$k_{purchase-order}^{ce} = \mathcal{H}_n(T_{me}, k_{purchase-order}^{v,grp})$ (T_{me} 是市场加密 key 域分隔符)。
2425

调解人只有在原始一方之一将 view key 释放给他们时, 才能获得买家-卖家通信的访问权和授权付款的能力。²⁶

此外, 卖家可以通过检查他们为每个产品发布的 base key 是否与显示的一致, 来验证市场主机 (根据此概念的实现方式, 可能也是唯一可用的调解人) 没有对他们的客户对话进行 MITM (“中间人”攻击, 即假装是买家或卖家)。由于用于创建 multisig 地址的买家 base key 也是加密 key 的一部分, 恶意主机将很难策划 MITM 攻击。

²⁴将 view key 和加密 key 分离, 允许仅赋予查看通信记录的权限, 而不授予查看 multisig 地址交易历史的权限。

²⁵1-of-2 multisig 地址也使用此方法。

²⁶调解人验证他们收到的通信记录没有被篡改是很重要的。一种方法可能是让每个协作签名方在发送新消息时包含当前消息记录的签名 hash。调解人可以查看来回消息和 hash 记录的序列来识别差异。这也有助于协作签名方识别未能传输的消息, 以及替代性地创建特定签名方确实收到了某些消息的证据。

第 12 章 12

联合 Monero 交易 (Joint Monero Transactions (TxTangle))

不同实体和场景的性质不可避免地产生了一些交易图启发式分析。特别是矿工、矿池（第 5.1 节，[?])、托管市场和交易所的行为在 Monero 基于环签名的协议中仍然有明显可分析的模式。

我们在此描述 TxTangle，类似于 Bitcoin 的 CoinJoin [?]，这是一种混淆那些启发式的方法。¹本质上，多笔交易被压缩成一笔交易，使每个参与者的行为模式融合在一起。

为了实现这种混淆，观察者利用联合交易中的信息来分组 input 和 output、将其与个别参与者关联、以及了解实际有多少参与者，必须是极其困难的。²此外，即使是参与者本身也不应知道参与者的数量，或者无法对其他参与者的 input 和 output 进行分组，除非他们控制了除一个参与者以外的所有参与者。³最后，应能在不依赖中心化机构的情况下构建联合交易 [?]。幸运的是，在 Monero 中所有这些要求都可以实现。

¹本章构成联合交易协议的提案。截至撰写本文时，尚未实现此类协议。之前的提案名为 MoJoin，由化名的合著者 Monero Research Lab (MRL) 研究员 Sarang Noether 创建，需要可信的经纪人才能运作。这种经纪人似乎与 Monero 项目对隐私和可替代性的基本承诺相矛盾，因此 MoJoin 未被进一步推进。

²由于在 Bitcoin 中金额是清晰可见的，通常可以根据金额总和对 CoinJoin 的 input 和 output 进行分组。[?]

³恶意污染联合交易是对这种方法的一种潜在攻击，最早针对 CoinJoin 被发现。[?]

12.1 构建联合交易 (Building joint transactions)

在普通交易中，input 和 output 通过金额平衡的证明绑定在一起。根据第 ?? 节，伪 output 承诺之和等于 output 承诺之和（加上手续费承诺）。

$$\sum_j C_j^a - (\sum_t C_t^b + fH) = 0$$

一个平凡的联合交易可以将多笔交易的所有内容放入一笔交易中。ML-SAG 消息将签名所有子交易的数据，金额平衡也显然成立 ($0 + 0 + 0 = 0$)。⁴然而，input 和 output 的分组同样可以平凡地通过测试 input/output 子集的金额是否平衡来识别。⁵

我们可以通过计算每对参与者之间的共享秘密，然后将这些偏移量添加到他们的伪 output 承诺的掩码中来轻松解决这个问题（第 ?? 节）。在每一对中，一个参与者将共享秘密加到他的一个伪 output 承诺上，另一个参与者从他的一个伪承诺中减去它。当求和时，秘密相互抵消，并且由于每对参与者都有一个共享秘密，金额平衡只有在所有承诺合并后才出现。⁶

共享秘密可能在即时意义上隐藏 input/output 分组，但参与者必须以某种方式了解所有 input 和 output，最简单的方法是各自传达自己的 input/output 分组。这显然违反了初始前提，而且无论如何都意味着参与者知道参与者的数量。

12.1.1 n 路通信通道 (n -way communication channel)

TxTangle 的最大参与者数量是 output 或 input 的数量（取较小者）。我们通过每个真实参与者假装为他发送的每个 output 是不同的人来建模。这主要是为了与其他潜在参与者建立一个群组通信通道，而不暴露参与者的数量。

设想 n ($2 \leq n \leq 16$ ，但建议至少 3 人)⁷个表面上互不相关的人在聊天室中以看似随机的间隔聚集，聊天室计划在时间 t_0 开放、 t_1 关闭（一次只能有 16 个人在聊天室中，聊天室有条件如手续费优先级、每字节基础费用 [用于围绕当前中位数和区块奖励简化共识]，以及可接受的交易类型范围——例如目前 Monero 创世初期的交易不能直接在 RingCT 交易中使用 [?])。在 t_1 时，所有模拟成员通过发布公钥来表示继续的意愿，然后通过在所有模拟成员之间构建共享秘密，聊天室被转换为 n 路通信通道。⁸此共享秘密用于加

⁴由于 Bulletproofs 风格的范围证明实际上是聚合成一个的（第 ?? 节），即使在平凡情况下参与者也需要在一定程度上协作。

⁵参与者可以尝试以花哨的方式分配手续费来混淆观察者，但面对暴力破解时会失败，因为手续费并不是很大（大约 32 位或更少）。

⁶我们偏移的是伪 output 承诺而非 output 承诺，因为 output 承诺掩码是从接收者的地址构建的（第 ?? 节）。

⁷目前，一笔交易最多可以有 16 个 output。

⁸第 ?? 节中的 multisig 方法是一种方式，将 M-of-N 一路扩展到 1-of-N。

密消息内容，而模拟成员使用 SAG 签名（第 ?? 节）签名 input 相关消息，这样就永远不会清楚是谁发送了给定的消息，output 相关消息则用模拟成员公钥集合上的 bLSAG（第 ?? 节）签名，使实际 output 与模拟成员脱钩。

⁹

12.1.2 构建联合交易的消息轮次 (Message rounds to construct a joint transaction)

通道设置完成后，TxTangle 交易可以在五轮通信中构建，下一轮只能在前一轮完成后才能开始，每轮都有一个定时通信间隔，消息应在其中随机发布。这些间隔旨在防止消息聚集暴露 input/output 分组。

1. 每个模拟成员为每个预期 output 私下生成一个随机标量，并用 bLSAG 签名。这些标量的排序列表用于确定 output 索引（回忆第 ?? 节；最小的标量获得索引 $t = 0$ ）。¹⁰他们发布这些 bLSAG，以及签名预期 input 的交易版本号的 SAG。在这一轮之后，参与者可以根据 input 和 output 的数量计算交易权重，并准确估算所需的手续费。¹¹¹²¹³
2. 每个模拟成员使用公钥列表与其他每个成员构建共享秘密来偏移他们的伪 output 承诺，并根据每对中较小的公钥决定谁加谁减。¹⁴每个模拟成员必须支付手续费估算的 $1/n$ （使用整数除法）。拥有最小 output 索引的模拟成员负责支付除法后的余数（这将是一个极其微小的金额，但必须考虑以防止指纹识别 TxTangle 交易）。他们私下为每个 output 生成交易公钥（尚不发送给其他成员），并构建他们的 output 承诺、编码金额和用于聚合 Bulletproof 范围证明的 Part A 部分证明，用 bLSAG 签名所有这些（每个 bLSAG 消息一个承诺、一个编码金额和一个部分证明，key image 将这些与指定 output 索引的原始随机标量列表关联起来）。伪 output 承诺照常生成（第 ?? 节），然后用共享秘

⁹每组独立的 TxTangle bLSAG 应使用相同的 key image，因为与给定 output 相关的所有内容都是链接在一起的。

¹⁰output 索引选择应与其他交易构建实现匹配，以避免对不同软件进行指纹识别。我们使用这种有效随机的方法来与核心实现保持一致，后者也是随机化 output 的。

¹¹如果根据比较 input 和 output 的数量与自己的 input/output 数量发现必须只有两个真实参与者，TxTangle 可以被放弃。建议每个参与者至少有两个 input 和两个 output，以防恶意行为者即使意识到只有两个参与者也不放弃 TxTangle。这个建议有待商榷，因为使用更多 input 和 output 在启发式上不是中性的。

¹²TxTangle 交易不应在 extra 字段中存储多余信息（例如不存储加密支付 ID，除非是只有 2 个 output 的 TxTangle，至少应有一个虚拟加密支付 ID）。

¹³手续费估算应基于标准化方法，使每个参与者计算出相同的结果。否则 output 可能会根据手续费计算方法被聚集。同样的手续费标准应在 TxTangle 之外实施，以促进 TxTangle 交易看起来与普通交易相同。

¹⁴由于点是压缩的（第 ?? 节），只需将密钥解释为 32 字节整数。按惯例，较小密钥的持有者加，较大密钥的持有者减。

密偏移，并用 SAG 签名。bLSAG 和 SAG 发布后，假设参与者以相同方式估算了总手续费，他们现在可以验证金额整体上是平衡。¹⁵

3. 如果金额正确平衡，我们开始一个小的额外轮次来构建聚合 Bulletproof，以证明所有 output 金额在范围内。每个模拟成员使用上一轮的 Part A 部分证明和 output 承诺，并私下计算聚合挑战 A。他们用它们来构建 Part B 部分证明，连同 bLSAG 一起发送到通道。
4. 参与者开始填写将由 MLSAG 签名的消息（回忆第 ?? 节的脚注）。在通信间隔内按随机顺序发布的是两种消息。每个 input 的环成员偏移量和 key image 用 SAG 签名，并与正确的伪 output 承诺关联。每个 output 的一次性地址、交易公钥和 Part C 部分证明（基于 Part B 部分证明和聚合挑战 B 计算）用 bLSAG 签名（这些还可以包含一个随机的基交易公钥分量，正如我们将看到的，它可用于虚假 Janus 缓解）。
5. 参与者使用所有部分证明完成聚合 Bulletproof，并私下应用对数内积技术将其压缩为最终证明以包含在交易数据中。一旦收集了所有将由 MLSAG 签名的信息，每个参与者完成他们的 input 的 MLSAG，并在通信间隔内随机将它们（每个带一个 SAG）发送到通道。任何参与者一旦获得所有片段就可以提交交易。

交易公钥和 Janus 缓解 (Transaction public keys and the Janus mitigation)

如果 TxTangle 的每个参与者都知道交易私钥 r （第 ?? 节），那么他们中的任何一个都可以根据已知地址列表测试其他人的一次性 output 地址。因此有必要将 TxTangle 交易构建为存在子地址接收者（第 ?? 节），包括为每个 output 使用不同的交易公钥。

为了匹配与子地址相关的 Janus 攻击缓解的可能实现——其中在 extra 字段中包含一个额外的「基」交易公钥 [?]，TxTangle 也应该有一个虚假的「基」密钥，由每个模拟成员生成的随机密钥之和组成。¹⁶

许多向子地址汇款的 TxTangle 参与者可能至少有两个 output，其中一个将找零转回参与者。这意味着任何 TxTangle 参与者仍然可以通过将找零的交易公钥也作为子地址接收者的「基」密钥来启用 Janus 缓解。¹⁷子地址

¹⁵我们不发布各自支付的单独手续费金额，以防某个参与者计算错误，这可能由于非标准手续费金额的集合而暴露 output 分组。如果金额不能正确平衡，TxTangle 交易可以被放弃。

¹⁶密钥取消（第 ?? 节）应该不是问题，因为它只是一个虚假密钥，理想情况下应该在交易公钥列表中随机索引。

¹⁷如果向自己的子地址发送，则不需要 Janus 缓解。启用了 Janus 缓解的钱包应该识别出在 TxTangle 中花费的金额等于接收到子地址的金额，因此不会错误地通知用户存在问题。

接收者可能会意识到交易是一个 TxTangle，并且「基」密钥可能对应于发送者的找零 output。¹⁸

12.1.3 弱点 (Weaknesses)

恶意行为者有两种主要方式来破坏 TxTangle 的目的——即对潜在对手/分析师隐藏 input/output 分组。他们可以污染交易，使诚实参与者的子集更小（甚至不存在）[?]。他们也可以导致 TxTangle 尝试失败，并利用相同参与者的后续尝试来估算 input/output 分组。

前者不容易缓解，特别是在非常去中心化的情况下，没有参与者有声誉。TxTangle 的一个可能应用是与协作的矿池一起使用，矿池可以隐藏其矿工属于哪个矿池。这种矿池会知道 input/output 分组，但由于目的是帮助其连接的矿工，保守信息对它们有利。此外，这种 TxTangle 不允许恶意行为者参与，假设矿池在某种程度上是诚实的。

后者可以通过仅尝试几次 TxTangle 后休息一下，并始终为新尝试重新生成交易的大多数随机元素来防御。这些元素包括交易公钥、伪承诺掩码、范围证明标量和 MLSAG 标量。特别是，每个 input 的环诱饵集应保持不变，以防止交叉比较暴露真正的 input。如果可能，不同的 TxTangle 尝试应使用不同的真正 input。由于这种弱点是不可避免的，它使下一节的概念更具吸引力。

12.2 托管 TxTangle (Hosted TxTangle)

真正去中心化的 TxTangle 有一些悬而未决的问题。定时轮次如何启动和执行？聊天室最初是如何创建的，让参与者找到彼此？最直接的方式是通过 TxTangle 主机，由它生成和管理那些聊天室。

这样的主机似乎违背了混淆参与者的目标，因为每个人必须连接并发送可用于关联 input/output 分组的消息（特别是如果主机参与并知道消息内容）。我们可以使用 I2P 等网络¹⁹使主机收到的每条消息看起来像是来自一个独特个体。

¹⁸这是假设交易公钥与 output 是 1:1 的，这在目前看来是这种情况。如果交易公钥在 extra 字段中按随机或排序顺序是标准做法，那么 TxTangle 和非 TxTangle 交易对于子地址接收者来说将是基本无法区分的。在某些特殊情况下，TxTangle 参与者无法包含「基」密钥（例如当他们的所有 output 都是发往子地址时），或者由于子地址接收者收到了大部分或全部 output 而明显不是 TxTangle。注意由于 TxTangle 交易通常比典型交易有更多的 output，此启发式可用于区分 TxTangle 和普通的子地址交易。

¹⁹隐形互联网项目 (<https://geti2p.net/en/>)。

12.2.1 通过 I2P 与主机的基本通信及其他特性 (Basic communication with a host over I2P, and other features)

使用 I2P，用户创建所谓的「隧道」，将重度加密的消息通过其他用户的客户端传送到目的地。据我们理解，这些隧道可以在被销毁和重新创建之前传输多条消息（例如隧道似乎有 10 分钟的定时器）。对于我们的用例，仔细控制新隧道的创建时间以及哪些消息可以从同一隧道传出是至关重要的。²⁰

1. 申请 *TxTangle*：在我们原始的 n 路提案（第 ?? 节）中，参与者在计划关闭之前逐渐将他们的模拟成员添加到可用的 *TxTangle* 房间。然而，如果有足够多的用户同时尝试 *TxTangle*，当用户试图随机将所有预期 output 放入同一个 *TxTangle* 「房间」时，可能会有很高的失败率，然后房间过早满员，他们不得不退出。这会相当混乱。

我们可以通过告诉主机我们有多少个 output（例如给他一个我们的模拟成员公钥列表），让他组装每个 *TxTangle* 的参与者，来实现有影响力的优化。由于我们仍然保留 bLSAG 和 SAG 消息协议，主机将无法识别最终交易中的 output 分组。他只知道参与者的数量以及每个人有多少 output。此外，在这种情况下观察者无法监控开放的 *TxTangle* 房间来推断参与者的信息，这是一个重要的隐私改进。注意主机污染 *TxTangle* 的能力与非主机设计没有显著差异，因此这一变化对该攻击向量是中性的。

2. 通信方法：由于主机已经充当消息传输的中心，最简单的方式是由他管理 *TxTangle* 通信。在每一轮中，主机从模拟成员收集消息（仍然在通信间隔内随机进行），在轮次结束时有一个短暂的数据分发阶段，他将所有收集的数据发送给每个参与者，在下一轮之前有一个缓冲期以确保消息被接收并有时间处理。
3. 隧道和 *input/output* 分组：一旦 *TxTangle* 启动，用户应将他们的模拟成员身份与实际创建的 output 分离。这意味着为 bLSAG 签名消息创建新隧道，每个此类隧道只能传输与特定 output 相关的消息（通过同一隧道传输多条此类消息是可以的，因为显然关于同一 output 的信息来自同一来源）。他们还应为与特定 input 相关的 SAG 签名消息创建新隧道。
4. 主机 MITM 攻击威胁：主机可能通过假装是其他参与者来欺骗某个参与者，因为他控制着发送用于构建 bLSAG 和 SAG 的模拟成员列表。

²⁰在 I2P 中有「出站」和「入站」隧道（见 <https://geti2p.net/en/docs/how/tunnel-routing>）。通过入站隧道接收的所有内容看起来像是来自同一来源，即使来自多个来源，因此表面上看来 *TxTangle* 用户不需要为所有用例创建不同的隧道。然而，如果 *TxTangle* 主机将自己设置为其入站隧道的入口点，那么他就获得了对 *TxTangle* 参与者出站隧道的直接访问权。

换言之，他发送给参与者 A 的列表可能包含参与者 A 的模拟成员，其余的都是他自己的。参与者 B 收到的消息在重新传输给 A 之前使用 A 的列表重新签名。由于 A 的列表签名的所有消息都属于 A，主机将直接了解 A 的 input/output 分组！

我们可以通过修改交易公钥的制作方式来防止主机作为诚实参与者交互的 MITM。参与者照常互相发送他们的预期交易公钥（用 bLSAG），然后，很像第 ?? 节中的稳健密钥聚合，实际包含在交易数据中的密钥（用于制作 output 承诺掩码等）以模拟成员列表的哈希为前缀。换言之， $\mathcal{H}_n(T_{agg}, S_{mock}, r_t G) * r_t G$ 是第 t 个交易公钥。将模拟成员列表烘焙到交易本身中使得没有所有实际参与者之间的直接通信就很难完成 TxTangle。²¹

12.2.2 主机即服务 (Host as a service)

对于可持续性和持续改进，TxTangle 服务以营利方式运营很重要。²²与其用基于账户的模型来损害用户身份，主机可以作为单独的 output 参与每个 TxTangle，并要求参与者为其提供资金。当访问主机的「eepsite」/服务申请 TxTangle 时，用户会收到当前托管费用的通知，该费用应按 output 支付。

参与者将负责支付手续费和主机费用的份额。这次，最小模拟成员的公钥（不包括主机的密钥）将承担手续费和主机费用的余额。²³由于主机没有 input，他没有伪 output 承诺来抵消其 output 的承诺掩码。相反，他照常与其他模拟成员创建共享秘密，然后将他真正的承诺掩码分成随机大小的块分配给每个其他模拟成员，并除以共享秘密。他发布这些标量的列表（根据公钥将它们与其他每个模拟成员对应），用他的模拟成员密钥签名，让参与者知道它来自主机。这个列表的出现标志着第 ?? 节中第 1 轮的开始（即设置轮「0」的结束）。模拟成员将他们的主机标量乘以适当的共享秘密，并加到他们的伪承诺掩码中。这样，即使是主机的 output 在最终交易中也不能被任何参与者识别，除非完全合谋对付他。

为了简化手续费的计算，主机可以在第 1 轮结束时分发交易中使用的总手续费，因为他会较早了解到交易权重。参与者可以验证该金额与预期金额相似，并支付其份额。

如果参与者合谋作弊不提供托管费用，主机可以在第 3 轮终止 TxTangle。如果通道中出现不该出现的或无效的消息，他也可以终止。

在第 5 轮结束时，主机完成交易并提交到网络进行验证，作为其服务的一部分。他在最终分发消息中包含交易哈希。

²¹如果实现了 Janus 缓解，此 MITM 防御应改为用虚假 Janus 基密钥实现。每个模拟成员提供一个随机密钥 $r_{mock}G$ ，然后实际基密钥为 $\sum_{mock} \mathcal{H}_n(T_{agg}, S_{mock}, r_{mock}G) * r_{mock}G$ 。

²²虽然部署的 TxTangle 服务可以营利，但代码本身可以是开源的。这对于审计与 TxTangle 服务交互的钱包软件很重要。

²³此处必须使用模拟成员密钥，因为主机不支付手续费，且他的 output 索引未知。

12.3 使用可信经纪人 (Using a trusted dealer)

去中心化 TxTangle 有一些缺点。它要求所有参与者在严格的时间表内积极通信（并首先找到彼此），而且实现起来具有挑战性。

添加一个负责从每个参与者收集交易信息并混淆 input/output 分组的中心化经纪人可以简化流程。代价是更高的信任要求，因为经纪人必须（至少）了解那些分组。²⁴

12.3.1 基于经纪人的流程 (Dealer-based procedure)

经纪人将宣传他可以管理 TxTangle，并收集潜在参与者的申请（包括预期 input [及其类型] 和 output 的数量）。如果他愿意，他可以用自己的 input/output 集参与。

在组装了接近 16 个 output 的分组后（应有两个或更多参与者，且没有参与者可以拥有除一个以外的所有 output 或 input），经纪人将开始五轮中的第一轮。在每一轮中，他从每个参与者收集信息，做出一些决定，并发出标志新一轮次开始的消息。

1. 首先，经纪人对每对参与者生成一个随机标量，并决定每对中谁应使用正版本或负版本。他使用 input 和 output 的数量和类型来估算所需的总手续费。他将每个参与者的标量求和，并私下向每个人发送其总和，加上他们预期支付的手续费份额，以及（随机选择的）他们的 output 索引。这些消息构成了 TxTangle 开始的信号。
2. 每个参与者照常构建他们的子交易，为他们的 output 生成单独的交易公钥（根据需要进行 Janus 缓解），计算一次性 output 地址并编码 output 金额，创建平衡 output 承诺和手续费份额的伪 output 承诺，组装 MLSAG 签名中使用的环成员偏移量列表及相应的 key image，并将经纪人发送的标量（乘以 G ）加到他们的一个伪 output 承诺上。他们为 output 创建 Part A 部分证明，并将所有这些信息发送给经纪人。经纪人验证 input 和 output 金额是否平衡，并将完整的 Part A 部分证明列表发送给每个参与者。
3. 每个参与者计算聚合挑战 A，并生成 Part B 部分证明发送给经纪人。经纪人收集部分证明并分发给所有其他参与者。
4. 每个参与者计算聚合挑战 B，并生成 Part C 部分证明发送给经纪人。经纪人收集这些证明，并应用对数内积技术将其压缩为最终证明。假设证明按预期验证，他生成一个随机的虚假 Janus「基」交易公钥，并将要由 MLSAG 签名的消息发送给每个参与者。

²⁴本节受 MoJoin 协议大纲的启发。

5. 每个参与者完成他们的 MLSAG 并发送给经纪人。一旦他拥有了所有片段，他就可以完成交易的构建，并提交到区块链。他也可以将交易 ID 发送给每个参与者，以便他们确认交易已发布。

第 13 章 13

联合 Monero 交易 (Joint Monero Transactions (TxTangle))

不同实体和场景的性质不可避免地产生了一些交易图启发式分析。特别是矿工、矿池（第 5.1 节，[?])、托管市场和交易所的行为在 Monero 基于环签名的协议中仍然有明显可分析的模式。

我们在此描述 TxTangle，类似于 Bitcoin 的 CoinJoin [?]，这是一种混淆那些启发式的方法。¹本质上，多笔交易被压缩成一笔交易，使每个参与者的行为模式融合在一起。

为了实现这种混淆，观察者利用联合交易中的信息来分组 input 和 output、将其与个别参与者关联、以及了解实际有多少参与者，必须是极其困难的。²此外，即使是参与者本身也不应知道参与者的数量，或者无法对其他参与者的 input 和 output 进行分组，除非他们控制了除一个参与者以外的所有参与者。³最后，应能在不依赖中心化机构的情况下构建联合交易 [?]。幸运的是，在 Monero 中所有这些要求都可以实现。

¹本章构成联合交易协议的提案。截至撰写本文时，尚未实现此类协议。之前的提案名为 MoJoin，由化名的合著者 Monero Research Lab (MRL) 研究员 Sarang Noether 创建，需要可信的经纪人才能运作。这种经纪人似乎与 Monero 项目对隐私和可替代性的基本承诺相矛盾，因此 MoJoin 未被进一步推进。

²由于在 Bitcoin 中金额是清晰可见的，通常可以根据金额总和对 CoinJoin 的 input 和 output 进行分组。[?]

³恶意污染联合交易是对这种方法的一种潜在攻击，最早针对 CoinJoin 被发现。[?]

13.1 构建联合交易 (Building joint transactions)

在普通交易中，input 和 output 通过金额平衡的证明绑定在一起。根据第 ?? 节，伪 output 承诺之和等于 output 承诺之和（加上手续费承诺）。

$$\sum_j C_j^a - (\sum_t C_t^b + fH) = 0$$

一个平凡的联合交易可以将多笔交易的所有内容放入一笔交易中。ML-SAG 消息将签名所有子交易的数据，金额平衡也显然成立 ($0 + 0 + 0 = 0$)。⁴然而，input 和 output 的分组同样可以平凡地通过测试 input/output 子集的金额是否平衡来识别。⁵

我们可以通过计算每对参与者之间的共享秘密，然后将这些偏移量添加到他们的伪 output 承诺的掩码中来轻松解决这个问题（第 ?? 节）。在每一对中，一个参与者将共享秘密加到他的一个伪 output 承诺上，另一个参与者从他的一个伪承诺中减去它。当求和时，秘密相互抵消，并且由于每对参与者都有一个共享秘密，金额平衡只有在所有承诺合并后才出现。⁶

共享秘密可能在即时意义上隐藏 input/output 分组，但参与者必须以某种方式了解所有 input 和 output，最简单的方法是各自传达自己的 input/output 分组。这显然违反了初始前提，而且无论如何都意味着参与者知道参与者的数量。

13.1.1 n 路通信通道 (n -way communication channel)

TxTangle 的最大参与者数量是 output 或 input 的数量（取较小者）。我们通过每个真实参与者假装为他发送的每个 output 是不同的人来建模。这主要是为了与其他潜在参与者建立一个群组通信通道，而不暴露参与者的数量。

设想 n ($2 \leq n \leq 16$ ，但建议至少 3 人)⁷个表面上互不相关的人在聊天室中以看似随机的间隔聚集，聊天室计划在时间 t_0 开放、 t_1 关闭（一次只能有 16 个人在聊天室中，聊天室有条件如手续费优先级、每字节基础费用 [用于围绕当前中位数和区块奖励简化共识]，以及可接受的交易类型范围——例如目前 Monero 创世初期的交易不能直接在 RingCT 交易中使用 [?])。在 t_1 时，所有模拟成员通过发布公钥来表示继续的意愿，然后通过在所有模拟成员之间构建共享秘密，聊天室被转换为 n 路通信通道。⁸此共享秘密用于加

⁴由于 Bulletproofs 风格的范围证明实际上是聚合成一个的（第 ?? 节），即使在平凡情况下参与者也需要在一定程度上协作。

⁵参与者可以尝试以花哨的方式分配手续费来混淆观察者，但面对暴力破解时会失败，因为手续费并不是很大（大约 32 位或更少）。

⁶我们偏移的是伪 output 承诺而非 output 承诺，因为 output 承诺掩码是从接收者的地址构建的（第 ?? 节）。

⁷目前，一笔交易最多可以有 16 个 output。

⁸第 ?? 节中的 multisig 方法是一种方式，将 M-of-N 一路扩展到 1-of-N。

密消息内容，而模拟成员使用 SAG 签名（第 ?? 节）签名 input 相关消息，这样就永远不会清楚是谁发送了给定的消息，output 相关消息则用模拟成员公钥集合上的 bLSAG（第 ?? 节）签名，使实际 output 与模拟成员脱钩。

9

13.1.2 构建联合交易的消息轮次 (Message rounds to construct a joint transaction)

通道设置完成后，TxTangle 交易可以在五轮通信中构建，下一轮只能在前一轮完成后才能开始，每轮都有一个定时通信间隔，消息应在其中随机发布。这些间隔旨在防止消息聚集暴露 input/output 分组。

1. 每个模拟成员为每个预期 output 私下生成一个随机标量，并用 bLSAG 签名。这些标量的排序列表用于确定 output 索引（回忆第 ?? 节；最小的标量获得索引 $t = 0$ ）。¹⁰他们发布这些 bLSAG，以及签名预期 input 的交易版本号的 SAG。在这一轮之后，参与者可以根据 input 和 output 的数量计算交易权重，并准确估算所需的手续费。¹¹¹²¹³
2. 每个模拟成员使用公钥列表与其他每个成员构建共享秘密来偏移他们的伪 output 承诺，并根据每对中较小的公钥决定谁加谁减。¹⁴每个模拟成员必须支付手续费估算的 $1/n$ （使用整数除法）。拥有最小 output 索引的模拟成员负责支付除法后的余数（这将是一个极其微小的金额，但必须考虑以防止指纹识别 TxTangle 交易）。他们私下为每个 output 生成交易公钥（尚不发送给其他成员），并构建他们的 output 承诺、编码金额和用于聚合 Bulletproof 范围证明的 Part A 部分证明，用 bLSAG 签名所有这些（每个 bLSAG 消息一个承诺、一个编码金额和一个部分证明，key image 将这些与指定 output 索引的原始随机标量列表关联起来）。伪 output 承诺照常生成（第 ?? 节），然后用共享秘

⁹每组独立的 TxTangle bLSAG 应使用相同的 key image，因为与给定 output 相关的所有内容都是链接在一起的。

¹⁰output 索引选择应与其他交易构建实现匹配，以避免对不同软件进行指纹识别。我们使用这种有效随机的方法来与核心实现保持一致，后者也是随机化 output 的。

¹¹如果根据比较 input 和 output 的数量与自己的 input/output 数量发现必须只有两个真实参与者，TxTangle 可以被放弃。建议每个参与者至少有两个 input 和两个 output，以防恶意行为者即使意识到只有两个参与者也不放弃 TxTangle。这个建议有待商榷，因为使用更多 input 和 output 在启发式上不是中性的。

¹²TxTangle 交易不应在 extra 字段中存储多余信息（例如不存储加密支付 ID，除非是只有 2 个 output 的 TxTangle，至少应有一个虚拟加密支付 ID）。

¹³手续费估算应基于标准化方法，使每个参与者计算出相同的结果。否则 output 可能会根据手续费计算方法被聚集。同样的手续费标准应在 TxTangle 之外实施，以促进 TxTangle 交易看起来与普通交易相同。

¹⁴由于点是压缩的（第 ?? 节），只需将密钥解释为 32 字节整数。按惯例，较小密钥的持有者加，较大密钥的持有者减。

密偏移，并用 SAG 签名。bLSAG 和 SAG 发布后，假设参与者以相同方式估算了总手续费，他们现在可以验证金额整体是否平衡。¹⁵

3. 如果金额正确平衡，我们开始一个小的额外轮次来构建聚合 Bulletproof，以证明所有 output 金额在范围内。每个模拟成员使用上一轮的 Part A 部分证明和 output 承诺，并私下计算聚合挑战 A。他们用它们来构建 Part B 部分证明，连同 bLSAG 一起发送到通道。
4. 参与者开始填写将由 MLSAG 签名的消息（回忆第 ?? 节的脚注）。在通信间隔内按随机顺序发布的是两种消息。每个 input 的环成员偏移量和 key image 用 SAG 签名，并与正确的伪 output 承诺关联。每个 output 的一次性地址、交易公钥和 Part C 部分证明（基于 Part B 部分证明和聚合挑战 B 计算）用 bLSAG 签名（这些还可以包含一个随机的基交易公钥分量，正如我们将看到的，它可用于虚假 Janus 缓解）。
5. 参与者使用所有部分证明完成聚合 Bulletproof，并私下应用对数内积技术将其压缩为最终证明以包含在交易数据中。一旦收集了所有将由 MLSAG 签名的信息，每个参与者完成他们的 input 的 MLSAG，并在通信间隔内随机将它们（每个带一个 SAG）发送到通道。任何参与者一旦获得所有片段就可以提交交易。

交易公钥和 Janus 缓解 (Transaction public keys and the Janus mitigation)

如果 TxTangle 的每个参与者都知道交易私钥 r （第 ?? 节），那么他们中的任何一个都可以根据已知地址列表测试其他人的一次性 output 地址。因此有必要将 TxTangle 交易构建为存在子地址接收者（第 ?? 节），包括为每个 output 使用不同的交易公钥。

为了匹配与子地址相关的 Janus 攻击缓解的可能实现——其中在 extra 字段中包含一个额外的「基」交易公钥 [?]，TxTangle 也应该有一个虚假的「基」密钥，由每个模拟成员生成的随机密钥之和组成。¹⁶

许多向子地址汇款的 TxTangle 参与者可能至少有两个 output，其中一个将找零转回参与者。这意味着任何 TxTangle 参与者仍然可以通过将找零的交易公钥也作为子地址接收者的「基」密钥来启用 Janus 缓解。¹⁷子地址

¹⁵我们不发布各自支付的单独手续费金额，以防某个参与者计算错误，这可能由于非标准手续费金额的集合而暴露 output 分组。如果金额不能正确平衡，TxTangle 交易可以被放弃。

¹⁶密钥取消（第 ?? 节）应该不是问题，因为它只是一个虚假密钥，理想情况下应该在交易公钥列表中随机索引。

¹⁷如果向自己的子地址发送，则不需要 Janus 缓解。启用了 Janus 缓解的钱包应该识别出在 TxTangle 中花费的金额等于接收到子地址的金额，因此不会错误地通知用户存在问题。

接收者可能会意识到交易是一个 TxTangle，并且「基」密钥可能对应于发送者的找零 output。¹⁸

13.1.3 弱点 (Weaknesses)

恶意行为者有两种主要方式来破坏 TxTangle 的目的——即对潜在对手/分析师隐藏 input/output 分组。他们可以污染交易，使诚实参与者的子集更小（甚至不存在）[?]。他们也可以导致 TxTangle 尝试失败，并利用相同参与者的后续尝试来估算 input/output 分组。

前者不容易缓解，特别是在非常去中心化的情况下，没有参与者有声誉。TxTangle 的一个可能应用是与协作的矿池一起使用，矿池可以隐藏其矿工属于哪个矿池。这种矿池会知道 input/output 分组，但由于目的是帮助其连接的矿工，保守信息对它们有利。此外，这种 TxTangle 不允许恶意行为者参与，假设矿池在某种程度上是诚实的。

后者可以通过仅尝试几次 TxTangle 后休息一下，并始终为新尝试重新生成交易的大多数随机元素来防御。这些元素包括交易公钥、伪承诺掩码、范围证明标量和 MLSAG 标量。特别是，每个 input 的环诱饵集应保持不变，以防止交叉比较暴露真正的 input。如果可能，不同的 TxTangle 尝试应使用不同的真正 input。由于这种弱点是不可避免的，它使下一节的概念更具吸引力。

13.2 托管 TxTangle (Hosted TxTangle)

真正去中心化的 TxTangle 有一些悬而未决的问题。定时轮次如何启动和执行？聊天室最初是如何创建的，让参与者找到彼此？最直接的方式是通过 TxTangle 主机，由它生成和管理那些聊天室。

这样的主机似乎违背了混淆参与者的目标，因为每个人必须连接并发送可用于关联 input/output 分组的消息（特别是如果主机参与并知道消息内容）。我们可以使用 I2P 等网络¹⁹使主机收到的每条消息看起来像是来自一个独特个体。

¹⁸这是假设交易公钥与 output 是 1:1 的，这在目前看来是这种情况。如果交易公钥在 extra 字段中按随机或排序顺序是标准做法，那么 TxTangle 和非 TxTangle 交易对于子地址接收者来说将是基本无法区分的。在某些特殊情况下，TxTangle 参与者无法包含「基」密钥（例如当他们的所有 output 都是发往子地址时），或者由于子地址接收者收到了大部分或全部 output 而明显不是 TxTangle。注意由于 TxTangle 交易通常比典型交易有更多的 output，此启发式可用于区分 TxTangle 和普通的子地址交易。

¹⁹隐形互联网项目 (<https://geti2p.net/en/>)。

13.2.1 通过 I2P 与主机的基本通信及其他特性 (Basic communication with a host over I2P, and other features)

使用 I2P，用户创建所谓的「隧道」，将重度加密的消息通过其他用户的客户端传送到目的地。据我们理解，这些隧道可以在被销毁和重新创建之前传输多条消息（例如隧道似乎有 10 分钟的定时器）。对于我们的用例，仔细控制新隧道的创建时间以及哪些消息可以从同一隧道传出是至关重要的。²⁰

1. 申请 *TxTangle*：在我们原始的 n 路提案（第 ?? 节）中，参与者在计划关闭之前逐渐将他们的模拟成员添加到可用的 *TxTangle* 房间。然而，如果有足够多的用户同时尝试 *TxTangle*，当用户试图随机将所有预期 output 放入同一个 *TxTangle* 「房间」时，可能会有很高的失败率，然后房间过早满员，他们不得不退出。这会相当混乱。

我们可以通过告诉主机我们有多少个 output（例如给他一个我们的模拟成员公钥列表），让他组装每个 *TxTangle* 的参与者，来实现有影响力的优化。由于我们仍然保留 bLSAG 和 SAG 消息协议，主机将无法识别最终交易中的 output 分组。他只知道参与者的数量以及每个人有多少 output。此外，在这种情况下观察者无法监控开放的 *TxTangle* 房间来推断参与者的信息，这是一个重要的隐私改进。注意主机污染 *TxTangle* 的能力与非主机设计没有显著差异，因此这一变化对该攻击向量是中性的。

2. 通信方法：由于主机已经充当消息传输的中心，最简单的方式是由他管理 *TxTangle* 通信。在每一轮中，主机从模拟成员收集消息（仍然在通信间隔内随机进行），在轮次结束时有一个短暂的数据分发阶段，他将所有收集的数据发送给每个参与者，在下一轮之前有一个缓冲期以确保消息被接收并有时间处理。
3. 隧道和 *input/output* 分组：一旦 *TxTangle* 启动，用户应将他们的模拟成员身份与实际创建的 output 分离。这意味着为 bLSAG 签名消息创建新隧道，每个此类隧道只能传输与特定 output 相关的消息（通过同一隧道传输多条此类消息是可以的，因为显然关于同一 output 的信息来自同一来源）。他们还应为与特定 input 相关的 SAG 签名消息创建新隧道。
4. 主机 MITM 攻击威胁：主机可能通过假装是其他参与者来欺骗某个参与者，因为他控制着发送用于构建 bLSAG 和 SAG 的模拟成员列表。

²⁰在 I2P 中有「出站」和「入站」隧道（见 <https://geti2p.net/en/docs/how/tunnel-routing>）。通过入站隧道接收的所有内容看起来像是来自同一来源，即使来自多个来源，因此表面上看来 *TxTangle* 用户不需要为所有用例创建不同的隧道。然而，如果 *TxTangle* 主机将自己设置为其入站隧道的入口点，那么他就获得了对 *TxTangle* 参与者出站隧道的直接访问权。

换言之，他发送给参与者 A 的列表可能包含参与者 A 的模拟成员，其余的都是他自己的。参与者 B 收到的消息在重新传输给 A 之前使用 A 的列表重新签名。由于 A 的列表签名的所有消息都属于 A，主机将直接了解 A 的 input/output 分组！

我们可以通过修改交易公钥的制作方式来防止主机作为诚实参与者交互的 MITM。参与者照常互相发送他们的预期交易公钥（用 bLSAG），然后，很像第 ?? 节中的稳健密钥聚合，实际包含在交易数据中的密钥（用于制作 output 承诺掩码等）以模拟成员列表的哈希为前缀。换言之， $\mathcal{H}_n(T_{agg}, S_{mock}, r_t G) * r_t G$ 是第 t 个交易公钥。将模拟成员列表烘焙到交易本身中使得没有所有实际参与者之间的直接通信就很难完成 TxTangle。²¹

13.2.2 主机即服务 (Host as a service)

对于可持续性和持续改进，TxTangle 服务以营利方式运营很重要。²²与其用基于账户的模型来损害用户身份，主机可以作为单独的 output 参与每个 TxTangle，并要求参与者为其提供资金。当访问主机的「eepsite」/服务申请 TxTangle 时，用户会收到当前托管费用的通知，该费用应按 output 支付。

参与者将负责支付手续费和主机费用的份额。这次，最小模拟成员的公钥（不包括主机的密钥）将承担手续费和主机费用的余额。²³由于主机没有 input，他没有伪 output 承诺来抵消其 output 的承诺掩码。相反，他照常与其他模拟成员创建共享秘密，然后将他真正的承诺掩码分成随机大小的块分配给每个其他模拟成员，并除以共享秘密。他发布这些标量的列表（根据公钥将它们与其他每个模拟成员对应），用他的模拟成员密钥签名，让参与者知道它来自主机。这个列表的出现标志着第 ?? 节中第 1 轮的开始（即设置轮「0」的结束）。模拟成员将他们的主机标量乘以适当的共享秘密，并加到他们的伪承诺掩码中。这样，即使是主机的 output 在最终交易中也不能被任何参与者识别，除非完全合谋对付他。

为了简化手续费的计算，主机可以在第 1 轮结束时分发交易中使用的总手续费，因为他会较早了解到交易权重。参与者可以验证该金额与预期金额相似，并支付其份额。

如果参与者合谋作弊不提供托管费用，主机可以在第 3 轮终止 TxTangle。如果通道中出现不该出现的或无效的消息，他也可以终止。

在第 5 轮结束时，主机完成交易并提交到网络进行验证，作为其服务的一部分。他在最终分发消息中包含交易哈希。

²¹如果实现了 Janus 缓解，此 MITM 防御应改为用虚假 Janus 基密钥实现。每个模拟成员提供一个随机密钥 $r_{mock}G$ ，然后实际基密钥为 $\sum_{mock} \mathcal{H}_n(T_{agg}, S_{mock}, r_{mock}G) * r_{mock}G$ 。

²²虽然部署的 TxTangle 服务可以营利，但代码本身可以是开源的。这对于审计与 TxTangle 服务交互的钱包软件很重要。

²³此处必须使用模拟成员密钥，因为主机不支付手续费，且他的 output 索引未知。

13.3 使用可信经纪人 (Using a trusted dealer)

去中心化 TxTangle 有一些缺点。它要求所有参与者在严格的时间表内积极通信（并首先找到彼此），而且实现起来具有挑战性。

添加一个负责从每个参与者收集交易信息并混淆 input/output 分组的中心化经纪人可以简化流程。代价是更高的信任要求，因为经纪人必须（至少）了解那些分组。²⁴

13.3.1 基于经纪人的流程 (Dealer-based procedure)

经纪人将宣传他可以管理 TxTangle，并收集潜在参与者的申请（包括预期 input [及其类型] 和 output 的数量）。如果他愿意，他可以用自己的 input/output 集参与。

在组装了接近 16 个 output 的分组后（应有两个或更多参与者，且没有参与者可以拥有除一个以外的所有 output 或 input），经纪人将开始五轮中的第一轮。在每一轮中，他从每个参与者收集信息，做出一些决定，并发出标志新一轮次开始的消息。

1. 首先，经纪人对每对参与者生成一个随机标量，并决定每对中谁应使用正版本或负版本。他使用 input 和 output 的数量和类型来估算所需的总手续费。他将每个参与者的标量求和，并私下向每个人发送其总和，加上他们预期支付的手续费份额，以及（随机选择的）他们的 output 索引。这些消息构成了 TxTangle 开始的信号。
2. 每个参与者照常构建他们的子交易，为他们的 output 生成单独的交易公钥（根据需要进行 Janus 缓解），计算一次性 output 地址并编码 output 金额，创建平衡 output 承诺和手续费份额的伪 output 承诺，组装 MLSAG 签名中使用的环成员偏移量列表及相应的 key image，并将经纪人发送的标量（乘以 G ）加到他们的一个伪 output 承诺上。他们为 output 创建 Part A 部分证明，并将所有这些消息发送给经纪人。经纪人验证 input 和 output 金额是否平衡，并将完整的 Part A 部分证明列表发送给每个参与者。
3. 每个参与者计算聚合挑战 A，并生成 Part B 部分证明发送给经纪人。经纪人收集部分证明并分发给所有其他参与者。
4. 每个参与者计算聚合挑战 B，并生成 Part C 部分证明发送给经纪人。经纪人收集这些证明，并应用对数内积技术将其压缩为最终证明。假设证明按预期验证，他生成一个随机的虚假 Janus「基」交易公钥，并将要由 MLSAG 签名的消息发送给每个参与者。

²⁴本节受 MoJoin 协议大纲的启发。

5. 每个参与者完成他们的 MLSAG 并发送给经纪人。一旦他拥有了所有片段，他就可以完成交易的构建，并提交到区块链。他也可以将交易 ID 发送给每个参与者，以便他们确认交易已发布。

第 14 章 14

RCTTypeBulletproof2 交易结构

我们在本附录中展示一笔真实 Monero 交易（类型 RCTTypeBulletproof2）的转储，并附有相关字段的解释说明。

该转储来自区块浏览器 <https://xmrchain.net>。也可以通过在非分离模式下运行的 monerod 守护进程中执行命令 `print_tx <TransactionID> +hex +json` 来获取。<TransactionID> 是交易的 hash（第 ?? 节）。打印的第一行显示了要执行的实际命令。

要复现我们的结果，读者可以执行以下操作：

1. 我们需要 Monero 命令行工具(CLI),可在 <https://web.getmonero.org/downloads/>（以及其他地方）获取。获取适合你操作系统的”Command Line Tools Only”，将文件移到合适的位置，然后解压。
2. 打开终端/命令行，导航到解压后创建的文件夹。
3. 使用 `./monerod` 运行 `monerod`。Monero blockchain 将开始下载。遗憾的是，目前没有办法在不下载 blockchain 的情况下在自己的系统上打印交易（即不使用区块浏览器）。
4. 同步完成后，`print_tx` 等命令就可以使用了。使用 `help` 了解其他命令。

组件 `rctsig_prunable`，顾名思义，是可以从 blockchain 中修剪的。也就是说，一旦区块达成共识且当前链长度排除了所有双花攻击的可能性，就可以修剪整个字段并用其 hash 替换 Merkle 树。

`key image` 单独存储在交易的不可修剪区域中。这些对于检测双花攻击至关重要，不能被修剪。

我们的示例交易有 2 个 input 和 2 个 output，于 2020-03-02 19:01:10 UTC 的时间戳被添加到 blockchain（由其区块的矿工报告）。

```

1 print_tx 84799c2fc4c18188102041a74cef79486181df96478b717e8703512c7f7f3349
2 Found in blockchain at height 2045821
3 {
4     "version": 2,
5     "unlock_time": 0,
6     "vin": [ {
7         "key": {
8             "amount": 0,
9             "key_offsets": [ 14401866, 142824, 615514, 18703, 5949, 22840, 5572, 16439,
10             983, 4050, 320
11         ],
12         "k_image": "c439b9f0da76ca0bb17920ca1f1f3f1d216090751752b091bef9006918cb3db4"
13     }
14     }, {
15         "key": {
16             "amount": 0,
17             "key_offsets": [ 14515357, 640505, 8794, 1246, 20300, 18577, 17108, 9824, 581
18             637, 1023
19         ],
20         "k_image": "03750c4b23e5be486e62608443151fa63992236910c41fa0c4a0a938bc6f5a37"
21     }
22     }
23 ],
24 "vout": [ {
25     "amount": 0,
26     "target": {
27         "key": "d890ba9ebfa1b44d0bd945126ad29a29d8975e7247189e5076c19fa7e3a8cb00"
28     }
29     }, {
30         "amount": 0,
31         "target": {
32             "key": "dbec330f8a67124860a9bfb86b66db18854986bd540e710365ad6079c8a1c7b0"
33         }
34     }
35 ],
36 "extra": [ 1, 3, 39, 58, 185, 169, 82, 229, 226, 22, 101, 230, 254, 20, 143,
37 37, 139, 28, 114, 77, 160, 229, 250, 107, 73, 105, 64, 208, 154, 182, 158, 200,
38 73, 2, 9, 1, 12, 76, 161, 40, 250, 50, 135, 231
39 ],
40 "rct_signatures": {

```

```

41     "type": 4,
42     "txnFee": 32460000,
43     "ecdhInfo": [ {
44         "amount": "171f967524e29632"
45     }, {
46         "amount": "5c2a1a9f54ccf40b"
47     } ],
48     "outPk": [ "fed8aded6914f789b63c37f9d2eb5ee77149e1aa4700a482aea53f82177b3b41",
49     "670e086e40511a279e0e4be89c9417b4767251c5a68b4fc3deb80fdae7269c17"]
50 },
51 "rctsig_prunable": {
52     "nbp": 1,
53     "bp": [ {
54         "A": "98e5f23484e97bb5b2d453505db79caadf20dc2b69dd3f2b3dbf2a53ca280216",
55         "S": "b791d4bc6a4d71de5a79673ed4a5487a184122321ede0b7341bc3fdc0915a796",
56         "T1": "5d58cfa9b69ecdb2375647729e34e24ce5eb996b5275aa93f9871259f3a1aecb",
57         "T2": "1101994fea209b71a2aa25586e429c4c0f440067e2b197469aa1a9a1512f84b7",
58         "taux": "b0ad39da006404ccacee7f6d4658cf17e0f42419c284bdca03c0250303706c03",
59         "mu": "cacd7ca5404afa28e7c39918d9f80b7fe5e572a92a10696186d029b4923fa200",
60         "L": [ "d06404fc35a60c6c47a04e2e43435cb030267134847f7a49831a61f82307fc32",
61         "c9a5932468839ee0cda1aa2815f156746d4dce79dab3013f4c9946fce6b69eff",
62         "efdae043dcedb79512581480d80871c51e063fe9b7a5451829f7a7824bcc5a0b",
63         "56fd2e74ac6e1766cfd56c8303a90c68165a6b0855fae1d5b51a2e035f333a1d",
64         "81736ed768f57e7f8d440b4b18228d348dce1eca68f969e75fab458f44174c99",
65         "695711950e076f54cf24ad4408d309c1873d0f4bf40c449ef28d577ba74dd86d",
66         "4dc3147619a6c9401fec004652df290800069b776fe31b3c5cf98f64eb13ef2c"
67     ],
68     "R": [ "7650b8da45c705496c26136b4c1104a8da601ea761df8bba07f1249495d8f1ce",
69     "e87789fbe99a44554871fcf811723ee350cba40276ad5f1696a62d91a4363fa6",
70     "ebf663fe9bb580f0154d52ce2a6dae544e7f6fb2d3808531b0b0749f5152ddbf",
71     "5a4152682a1e812b196a265a6ba02e3647a6bd456b7987adff288c5b0b556ec6",
72     "dc0dcb2e696e11e4b62c20b6bfc6b182290748c5de254d64bf7f9e3c38fb46c9",
73     "101e2271ced03b229b88228d74b36088b40c88f26db8b1f9935b85fb3ab96043",
74     "b14aae1d35c9b176ac526c23f31b044559da75cf95bc640d1005bfcc6367040b"
75     ],
76     "a": "4809857de0bd6becdb64b85e9dfbf6085743a8496006b72ceb81e01080965003",
77     "b": "791d8dc3a4ddde5ba2416546127eb194918839ced3dea7399f9c36a17f36150e",
78     "t": "aace86a7a1cbdec3691859fa07fdc83eed9ca84b8a064ca3f0149e7d6b721c03"
79     }
80 ],

```